

MANUAL DE CONSTRUCCIÓN

Cómo construir la app de contenido de Instagram con aprobación

Panel multiusuario · cola de aprobación · programación de huecos editable
Flask · Postgres (Neon) · Render · GitHub Actions · API de Instagram

@trabajoexcel · trabajoexcel.com

Repositorio JAESCOBARC/IG_RRSS · Versión 3

Generado el 21/09/2026 a partir del código del repositorio

Índice

1. Introducción: qué vas a construir	6
1.1 El objetivo en una frase	6
1.2 Qué hace y qué no hace	6
1.3 A quién va dirigido este manual	6
1.4 Cómo usar este manual	7
1.5 Requisitos previos	7
1.6 Mapa de capítulos	8
2. Arquitectura y decisiones de diseño	9
2.1 Las piezas	9
2.2 El recorrido completo de un post	9
2.3 Estados de un post	10
2.4 Decisiones de diseño (y por qué)	11
2.5 Tecnologías	12
3. Preparar el entorno y la estructura del repositorio	13
3.1 Crear el repositorio	13
3.2 Estructura de carpetas	13
3.3 Entorno de Python	13
3.4 El archivo .gitignore	14
3.5 Regla de oro: los secretos	14
3.6 Variables de entorno de la app	15
4. Instagram: cuenta, token y cliente de la API	16
4.1 Qué API se usa	16
4.2 Pasos (orientativos)	16
4.3 Cómo se publica por API: contenedores	17
4.4 El token: 60 días y renovación	18
4.5 El cliente: app/instagram.py	18
5. La base de datos	20
5.1 Un adaptador para dos motores	20
5.2 El esquema	20
5.3 Migraciones	21
5.4 El almacén clave-valor	21
6. La regla de contenido (CTA prohibidos)	24
6.1 La regla	24
6.2 Por qué se aplica en el servidor	24
6.3 Cómo funciona	24

7. La programación: días, horas y huecos	26
7.1 Conceptos	26
7.2 Dos capas de código	26
7.3 schedule.txt: la programación inicial	26
7.4 La lógica de huecos (schedule.py)	27
7.5 La persistencia (schedule_store.py)	30
8. Usuarios, contraseñas y roles	32
8.1 Modelo	32
8.2 Decisiones	32
9. El servidor Flask (app/server.py)	34
9.1 Arranque y configuración	34
9.2 Seguridad	36
9.3 Utilidades	37
9.4 El token de Instagram	38
9.5 Acceso: healthz, login y logout	39
9.6 El panel: lista y detalle	40
9.7 Aprobar y sacar de la cola	41
9.8 El aviso del programador y la publicación	42
9.9 Rechazar, reabrir, marcar como fallido y eliminar	44
9.10 Cuenta, usuarios y programación	45
9.11 La API de borradores	47
10. La interfaz: plantillas y CSS	50
10.1 Principios	50
10.2 Las plantillas	50
10.3 Los estilos	56
11. Pruebas automáticas	59
11.1 Estrategia	59
11.2 Qué cubren	60
11.3 Cómo escribir una prueba nueva	60
12. El generador de imágenes	61
12.1 Qué resuelve	61
12.2 Los tres formatos	61
12.3 Cómo está construido	61
12.4 El archivo de entrada (JSON)	62
12.5 Uso	62
12.6 La guía de estilo	65

13. Scripts de subida, skill de Claude y rutina semanal	66
13.1 La cadena de generación	66
13.2 url.txt y pick_urls.py	66
13.3 upload_draft.py	66
13.4 La skill de Claude (SKILL.md)	68
13.5 La rutina semanal (los lunes)	69
13.6 CLAUDE.md: las reglas del proyecto	70
14. Despliegue: Neon, Render y GitHub Actions	71
14.1 Neon (Postgres)	71
14.2 Render: el archivo render.yaml	71
14.3 El programador: tick.yml	72
14.4 Conectar tu equipo	74
14.5 Pruebas en producción, paso a paso	74
14.6 Actualizar la app	74
15. Operación y mantenimiento	75
15.1 Una semana tipo	75
15.2 Tareas habituales	75
15.3 Rotar claves	75
15.4 Qué mirar cuando algo va mal	75
15.5 Lecciones de incidentes reales	76
15.6 Copias de seguridad	76
16. Seguridad	77
16.1 Qué se protege y cómo	77
16.2 Límites conocidos	77
16.3 Lista de comprobación antes de dar acceso a otra persona	77
17. Solución de problemas	78
18. Ampliaciones y límites	79
18.1 Añadir un tipo de post nuevo	79
18.2 Otras ideas	79
18.3 Límites conocidos del diseño	79
Apéndice A. Pruebas automáticas: código completo	80
Apéndice B. La skill de Claude: archivos completos	92
B.1 SKILL.md	92
B.2 Guía de estilo	95
Apéndice C. Glosario	96

Apéndice D. Lista de comprobación final 97

1. Introducción: qué vas a construir

1.1 El objetivo en una frase

Una aplicación web que recibe **borradores de contenido para Instagram** (carruseles, publicaciones de una sola imagen e historias), los muestra en un **panel con usuarios y contraseña** para que una persona los revise y los apruebe, y los **publica sola en Instagram** en los días y horas que se configuran desde el propio panel. Después de publicar, **borra las imágenes** y conserva solo el texto, el estado y el enlace del post.

El sistema nació para la cuenta **@trabajoexcel** (que promociona trabajoexcel.com), pero nada de lo que se explica aquí depende de esa cuenta: cambiando las URLs, los textos y el estilo visual sirve para cualquier cuenta profesional de Instagram.

1.2 Qué hace y qué no hace

Hace:

- Recibe borradores por una API protegida con clave (los sube Claude, pero podría subirlos cualquier script).
- Valida cada borrador: tipo, número de imágenes, formato JPEG, tamaño, proporción, longitud del texto, hashtags y una **regla de contenido** que prohíbe ciertos llamados a la acción.
- Muestra un panel para revisar (imágenes, texto, hashtags) y aprobar o rechazar. Es multiusuario, con roles de administrador y editor.
- Mantiene una **cola por tipo de post** y publica **un post por hueco de programación**.
- Publica en Instagram a través de la API oficial, sirviendo las imágenes desde la propia app solo durante la publicación.
- Renueva sola el token de Instagram, que caduca.
- Avisa en el panel de los problemas (programador parado, tipos sin hueco, posts fallidos).

No hace (por diseño):

- **No publica nada sin aprobación humana.** Publicar es una acción sobre una cuenta real; la autoriza una persona en el panel.
- **No guarda imágenes** ni en el repositorio, ni en el servidor después de publicar, ni en el equipo tras subir el borrador.
- No genera contenido por sí misma: la generación la hace Claude con una *skill* (capítulos 12 y 13), y la app solo recibe el resultado.
- No publica Reels ni vídeo, ni gestiona comentarios o mensajes.

1.3 A quién va dirigido este manual

A quien quiera **reconstruir el sistema desde cero** o entender a fondo cada pieza para mantenerlo: una persona con nociones básicas de Python, de línea de comandos y de Git. No hace falta experiencia previa con Flask, con la API de Instagram ni con despliegues: cada concepto se explica cuando aparece.

Lo que sí necesitas es paciencia con las cuentas y los permisos de terceros (Meta, Render, Neon, GitHub): son la parte que más tiempo lleva y la que más cambia con los años.

1.4 Cómo usar este manual

El manual sigue el orden en el que conviene construir la app. Cada capítulo explica **qué hace la pieza, por qué está hecha así y cómo funciona**, y después muestra **el código completo del archivo**, tal como está en el repositorio (el PDF se generó leyendo los archivos reales, así que el código que ves aquí es el que funciona).

Puedes trabajar de dos maneras:

1. **Reconstruir:** crear cada archivo con el contenido del listado, en el orden de los capítulos, y ejecutar las pruebas del capítulo 11 al final de cada bloque.
2. **Entender y mantener:** leer las explicaciones y consultar el listado solo cuando necesites el detalle.

Convenciones del texto:

Elemento	Significado
texto en monoespaciado	Nombres de archivos, funciones, variables y comandos
Recuadro NOTA	Información complementaria
Recuadro POR QUÉ	La razón de una decisión de diseño, para que puedas cambiarla con criterio
Recuadro CUIDADO	Algo que, si se hace mal, rompe la app o compromete la seguridad
Recuadro COMPRUEBA	Una verificación concreta antes de seguir
Barra verde sobre un bloque de código	Ruta del archivo dentro del repositorio
↳ al inicio de una línea de código	Continuación de una línea larga (en el archivo real es una sola línea)

NOTA Las rutas que aparecen son relativas a la raíz del repositorio. En el texto, «la raíz» es la carpeta que contiene `app/`, `scripts/`, `.github/` y `.claude/`.

1.5 Requisitos previos

Conocimientos: Python (funciones, diccionarios, excepciones), SQL básico (`SELECT`, `INSERT`, `UPDATE`), HTML y CSS básicos, y manejo de Git.

Herramientas en tu equipo:

Herramienta	Versión	Para qué
Python	3.12 o superior (el proyecto se probó con 3.13 en local y 3.12.7 en Render)	Ejecutar la app, las pruebas y los scripts
Git	cualquiera reciente	Versionar y subir el código
GitHub CLI (<code>gh</code>)	opcional pero recomendable	Crear el repositorio y los <code>secrets</code> desde la terminal
Un editor (VS Code)	cualquiera	Editar el código

Cuentas (todas tienen plan gratuito suficiente):

Servicio	Para qué	Notas
GitHub	Alojar el código y ejecutar el programador (Actions)	El repositorio es público : ver las reglas de secretos en el capítulo 3
Neon	Base de datos Postgres	Plan gratuito
Render	Alojar la app web	Plan gratuito: se duerme tras un rato sin uso

Servicio	Para qué	Notas
Meta for Developers	Registrar una app que use la API de Instagram	Necesitas una cuenta de Facebook/Meta
Instagram	La cuenta donde se publica	Debe ser una cuenta profesional (empresa o creador)

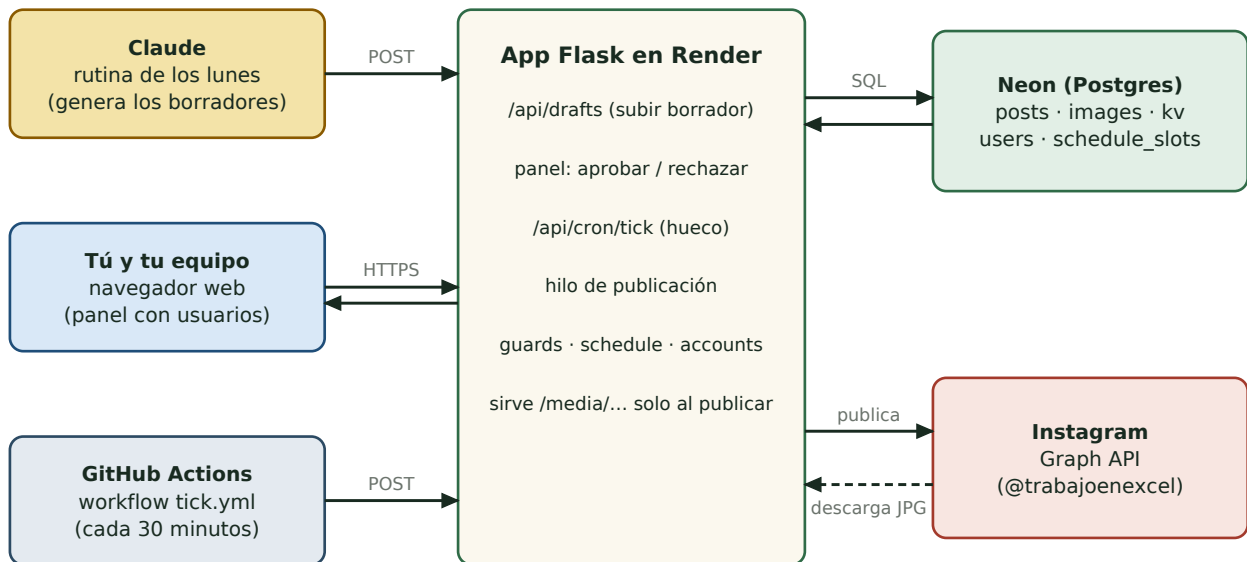
1.6 Mapa de capítulos

Cap.	Contenido	Archivos que se crean
2	Arquitectura, flujo completo y decisiones de diseño	—
3	Entorno, cuentas y estructura del repositorio	<code>.gitignore</code> , <code>requirements.txt</code>
4	Instagram: cuenta, app de Meta, token y cliente de la API	<code>app/instagram.py</code>
5	Base de datos y migraciones	<code>app/db.py</code>
6	Regla de contenido (CTA prohibidos)	<code>app/guards.py</code>
7	Programación: huecos, zona horaria y tolerancia	<code>app/schedule.py</code> , <code>app/schedule_store.py</code> , <code>app/schedule.txt</code>
8	Usuarios y contraseñas	<code>app/accounts.py</code>
9	El servidor Flask, sección por sección	<code>app/server.py</code>
10	Interfaz: plantillas y CSS	<code>app/templates/*</code> , <code>app/static/style.css</code>
11	Pruebas automáticas	<code>app/test_server.py</code>
12	Generador de imágenes	<code>.claude/skills/carruseles-app/scripts/carousel_gen.py</code>
13	Scripts de subida y skill de Claude	<code>scripts/*</code> , <code>.claude/skills/carruseles-app/SKILL.md</code>
14	Despliegue: Neon, Render y GitHub Actions	<code>render.yaml</code> , <code>.github/workflows/tick.yml</code>
15-17	Operación, seguridad y solución de problemas	—
18	Ampliaciones y límites conocidos	—

2. Arquitectura y decisiones de diseño

2.1 Las piezas

El sistema tiene **cinco actores** que se comunican por HTTPS. Ninguno necesita estar encendido a la vez que otro, salvo la app y la base de datos.



Pieza	Qué es	Qué hace
Claude (rutina)	Una rutina programada de Claude en la nube, cada lunes	Genera las imágenes y el texto de 3 borradores y los sube por POST /api/drafts
App Flask	Servidor web en Render	Valida y guarda borradores, sirve el panel, gestiona la cola, publica en Instagram y borra las imágenes
Neon (Postgres)	Base de datos	Guarda posts, imágenes temporales, usuarios, huecos de programación y datos internos (token)
GitHub Actions	Un <i>cron</i> (tick.yml)	Cada 30 minutos llama a POST /api/cron/tick ; la app decide si toca publicar
Instagram Graph API	API oficial de Meta	Crea los contenedores de medios y los publica; descarga las imágenes desde la app
Tú y tu equipo	Navegador	Revisan y aprueban en el panel; administran usuarios y programación

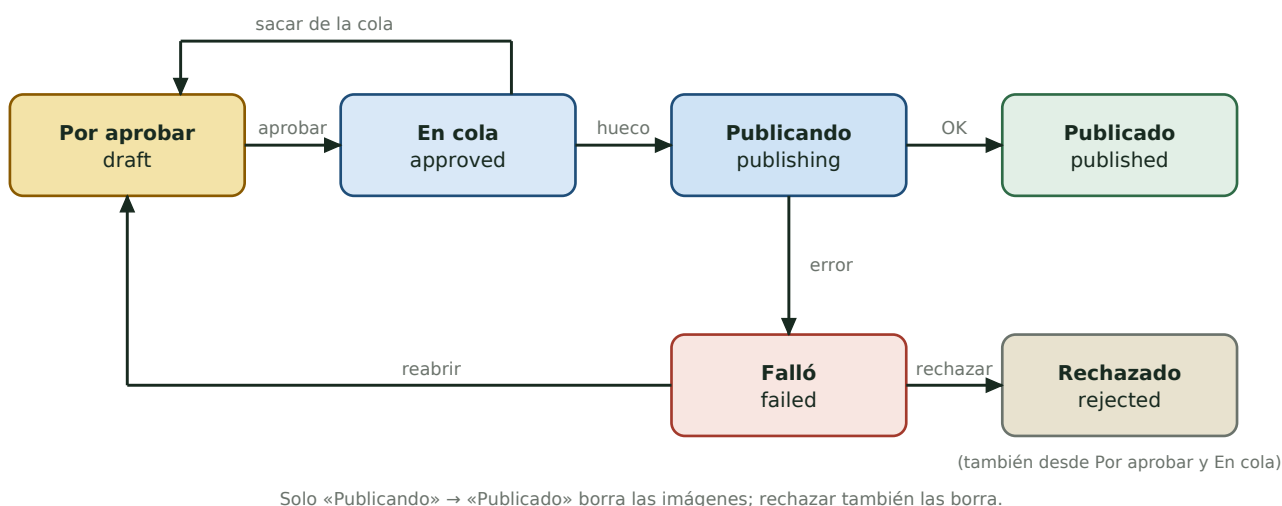
2.2 El recorrido completo de un post

Sigue un post desde que nace hasta que se publica:

1. **El lunes**, la rutina de Claude elige URLs al azar (**pick_urls.py**), genera las imágenes JPG con el generador (capítulo 12) y sube el borrador con **upload_draft.py**. El script envía las imágenes, el título, el texto y los hashtags a **POST /api/drafts**.
2. **La app valida** el borrador (capítulo 9.11). Si algo falla responde 422 con el motivo y no guarda nada. Si todo está bien, guarda el post en estado **draft** y las imágenes en la tabla **images**.

3. **Una persona entra al panel**, ve el borrador en «Por aprobar», revisa las imágenes y el texto, marca la casilla de confirmación y pulsa «Aprobar y poner en cola». El post pasa a **approved**, con la fecha precisa de aprobación y quién lo aprobó.
4. **El programador** (GitHub Actions) llama cada 30 minutos a `/api/cron/tick`. La app consulta la programación: si hay un hueco vigente para un tipo y aún no se ha usado, lo **reclama** de forma atómica y toma el post aprobado **más antiguo** de ese tipo.
5. **La app publica**: el post pasa a **publishing**, un hilo llama a la API de Instagram con las URLs de las imágenes (`/media/<id>/<n>.jpg`) y espera a que Instagram las descargue y procese.
6. **Al terminar**, el post pasa a **published** con su enlace, y la app **borra las imágenes**. Si algo falla, pasa a **failed** con el error (sin credenciales) y conserva las imágenes para poder revisarlo.
7. El workflow de GitHub espera al resultado consultando `GET /api/posts/<id>`; si el post falló, el workflow termina en rojo y GitHub envía un correo.

2.3 Estados de un post



Estado	Significado	Quién lo cambia
draft (Por aprobar)	Subido por la API, pendiente de revisión	La API al crear; «Sacar de la cola» y «Reabrir» lo devuelven aquí
approved (En cola)	Autorizado por una persona; espera su hueco	El botón «Aprobar»
publishing (Publicando)	Un hilo de la app está llamando a Instagram	El aviso del programador
published (Publicado)	Publicado; se conserva el enlace	El hilo de publicación al acabar bien
failed (Falló)	La publicación falló; se conserva el error	El hilo de publicación, o el botón «Marcar como fallido» si se quedó colgado
rejected (Rechazado)	Descartado por una persona; imágenes borradas	El botón «Rechazar»

Las transiciones **se hacen con** `UPDATE ... WHERE status = ...` (comparar y cambiar en una sola sentencia). Así, si dos personas pulsan a la vez, o dos avisos del programador coinciden, **solo uno gana**. Es la base de la seguridad de la cola.

2.4 Decisiones de diseño (y por qué)

Las imágenes no pasan por Git

El repositorio es público. Git conserva el historial: una imagen subida y borrada después sigue estando en el historial para siempre. Por eso las imágenes **nunca** se guardan en el repositorio: se generan en un directorio temporal, se suben a la app y allí viven **solo en la base de datos** hasta que se publican o se rechazan.

La app publica (no GitHub Actions)

Instagram exige una **URL pública** para cada imagen. Si publicara Actions, las imágenes tendrían que estar en algún sitio accesible (por ejemplo, en el repositorio público). Al publicar desde la app, ella misma sirve las imágenes **solo mientras dura la publicación** y luego las borra de verdad.

Aprobar es poner en cola, no publicar al instante

La aprobación y la publicación están separadas por la programación. Esto permite aprobar cuando te venga bien y que el contenido salga a la hora en la que tu audiencia está activa. Además, protege: un error al aprobar no publica nada de inmediato.

Un post por hueco y por tipo

Cada hueco de la programación publica **un** post de **un** tipo: el aprobado más antiguo de ese tipo. Así la cadencia es predecible (por ejemplo, un carrusel a la semana) aunque haya muchos posts en cola. Si en un hueco no hay nada aprobado, **el hueco se pierde**: no se acumula ni se recupera.

Postgres en producción y SQLite en local

La app usa solo SQL estándar con marcadores `%s`. En Neon usa `psycopg`; en local, `sqlite3`. Con un pequeño adaptador (`db.py`, capítulo 5) el mismo código y las mismas pruebas sirven para los dos. Ventaja: las pruebas no necesitan ningún servidor de base de datos.

Un solo worker de gunicorn

El estado de «publicando» vive en un hilo, y el límite de intentos de login en memoria. Con un solo proceso (y varios hilos) no hay que compartir ese estado. Para el volumen de esta app (unos pocos posts a la semana) es más que suficiente.

El token de Instagram vive en la base de datos

Los tokens de larga duración caducan a los 60 días y se pueden renovar. La app renueva el token cada 20 días y guarda el nuevo **en la base de datos**; la variable de entorno de Render solo es el valor inicial. Si cambias la variable de entorno, el valor nuevo manda (capítulo 9.4).

La programación vive en la base de datos

Al principio los horarios estaban en un archivo (`schedule.txt`), lo que obligaba a editar, subir a GitHub y redespigar. Ahora se editan desde el panel y se guardan en la base de datos; `schedule.txt` es solo la programación inicial que se copia la primera vez (capítulo 7).

Los usuarios viven en la base de datos, con una cuenta de rescate

Los usuarios se crean y borran en el panel. La cuenta integrada `admin` usa la variable de entorno `ADMIN_PASSWORD`, no está en la base de datos y no se puede borrar: sirve de recuperación si todo lo demás falla (capítulo 8).

Seguridad por capas

El panel usa sesión con cookie firmada y protección CSRF; la API usa una clave; las imágenes solo son públicas mientras se publica; los errores se **redactan** antes de guardarse (para no exponer el token); y una regla de contenido se aplica tanto al subir como al aprobar y al publicar (capítulo 16).

2.5 Tecnologías

Componente	Tecnología	Por qué
Servidor	Flask 3 + gunicorn	Pequeño, sin magia, fácil de leer
Base de datos	Postgres (Neon) / SQLite	Gratuito y sin mantenimiento / cero configuración en local
Cliente HTTP	<code>requests</code>	Llamar a la API de Instagram
Plantillas	Jinja2 (incluido con Flask)	Autoescapado por defecto (evita XSS)
Contraseñas	<code>werkzeug.security</code>	Hash con sal (scrypt) ya incluido con Flask
Zonas horarias	<code>zoneinfo</code> + <code>tzdata</code>	Cambio de hora verano/invierno correcto
Imágenes (generador)	Pillow	Dibujar texto con medida real de la fuente
Alojamiento	Render (plan gratuito)	Despliegue desde el repositorio con un archivo <code>render.yaml</code>
Programador	GitHub Actions	Gratuito y no requiere servidor propio

3. Preparar el entorno y la estructura del repositorio

3.1 Crear el repositorio

Crea un repositorio en GitHub (público, como el original, o privado si prefieres; si es privado, GitHub Actions tiene una cuota mensual gratuita). Con la CLI:

```
gh repo create IG_RRSS --public --clone
cd IG_RRSS
```

3.2 Estructura de carpetas

Esta es la estructura final. Iremos creando cada archivo en su capítulo.

```
IG_RRSS/
├── app/
│   ├── server.py          ← la aplicación web (lo que se despliega en Render)
│   ├── db.py             ← servidor Flask: panel, cola, programador y API
│   ├── accounts.py       ← acceso a datos (Postgres / SQLite) y migraciones
│   ├── schedule.py       ← usuarios, contraseñas y roles
│   ├── schedule_store.py ← lógica de huecos de publicación (sin base de datos)
│   ├── schedule.txt      ← programación guardada en la base de datos
│   ├── guards.py         ← programación INICIAL (se copia una sola vez)
│   ├── instagram.py      ← regla de contenido (CTA prohibidos)
│   ├── requirements.txt   ← cliente de la API de Instagram
│   ├── test_server.py    ← dependencias de la app
│   ├── templates/       ← pruebas automáticas
│   │   ├── base.html    ← HTML (Jinja2)
│   │   ├── login.html   ← base.html login.html index.html post.html
│   │   ├── account.html ← account.html users.html schedule.html
│   │   └── users.html
│   └── static/style.css  ← estilos
├── scripts/
│   ├── upload_draft.py   ← sube un borrador a la app
│   └── pick_urls.py      ← elige URLs al azar
├── .claude/skills/carruseles-app/ ← skill de Claude: genera el contenido
│   ├── SKILL.md
│   ├── scripts/carousel_gen.py ← generador de imágenes
│   ├── assets/fonts/         ← fuentes incluidas
│   └── references/           ← guía de estilo y ángulos por URL
├── .github/workflows/tick.yml ← el programador (cada 30 min)
├── render.yaml               ← plano de despliegue en Render
├── requirements.txt          ← dependencias de los scripts de la raíz
├── url.txt                   ← URLs a promocionar (una por línea)
├── CLAUDE.md                 ← reglas del proyecto para Claude
├── STATUS.md                 ← bitácora: qué está verificado y qué no
├── README.md
└── .gitignore
```

3.3 Entorno de Python

Crea un entorno virtual y instala las dependencias:

```
python -m venv .venv
# Windows: .venv\Scripts\activate      macOS/Linux: source .venv/bin/activate
pip install -r app/requirements.txt
pip install pillow pyyaml
```

Las dependencias de la app (`app/requirements.txt`):

app/requirements.txt – dependencias de la app

```
flask>=3.0,<4
psycopy[binary]>=3.1,<4
requests>=2.31,<3
gunicorn>=22,<24
tzdata
```

Paquete	Uso
<code>flask</code>	Servidor web
<code>psycopg[binary]</code>	Conexión a Postgres (solo se usa si <code>DATABASE_URL</code> apunta a Postgres)
<code>requests</code>	Llamadas a la API de Instagram
<code>gunicorn</code>	Servidor de producción en Render
<code>tzdata</code>	Base de datos de zonas horarias (Windows y algunos contenedores no la traen)

Los **scripts de la raíz** y el generador usan otras (`requirements.txt` de la raíz):

<code>requirements.txt</code> – dependencias de los scripts
<pre># Scripts de la raíz (upload_draft.py, pick_urls.py) y generador de carruseles. requests pyyaml pillow</pre>

CUIDADO Las **pruebas** importan Pillow (`from PIL import Image`) para fabricar JPEG de prueba, y Pillow no está en `app/requirements.txt` porque la app en producción no lo necesita. Para ejecutar las pruebas instala también pillow (`pip install pillow`).

3.4 El archivo .gitignore

Evita subir por error secretos, bases de datos locales o ajustes personales:

<code>.gitignore</code> – qué no se sube a Git
<pre>__pycache__/ *.pyc .venv/ .env .claude/settings.local.json app/local.db app/*.db .claude/settings.json</pre>

Línea	Por qué
<code>.env</code>	Aquí viven las claves en local. Nunca debe llegar al repositorio
<code>app/*.db, app/local.db</code>	La base de datos SQLite de desarrollo
<code>.claude/settings.json, .claude/settings.local.json</code>	Permisos y ajustes locales de Claude Code
<code>__pycache__/, *.pyc, .venv/</code>	Basura de Python

3.5 Regla de oro: los secretos

El repositorio es público. **Nunca** escribas dentro de la carpeta del proyecto tokens, claves de API, contraseñas ni archivos que las contengan. Los secretos viven únicamente en:

- **Variables de entorno de Render** (para la app en producción).
- **Un archivo `.env` local** (ignorado por Git) para trabajar desde tu equipo.
- **Secrets de GitHub Actions** (para el programador).
- **Las «credenciales de API» del entorno de la rutina de Claude** (no como variable de entorno: esas las ve cualquiera que use el entorno).

CUIDADO Si alguna vez un secreto llega a un commit, **considera que está comprometido** aunque lo borres después: el historial lo conserva. La solución limpia es revocarlo y generar otro. En este proyecto ya ocurrió una vez con un token (por guardar dos secretos cruzados y volcar un error con el token dentro a un archivo público); de ahí que los errores se **redacten** antes de guardarse (capítulo 9.3).

3.6 Variables de entorno de la app

Todas las lee la app al arrancar (`os.environ`). Ninguna va en el código.

Variable	Obligatoria	Qué es
<code>DATABASE_URL</code>	En producción	Cadena de conexión de Neon (<code>postgresql://...</code>). Si falta, se usa SQLite local
<code>ADMIN_PASSWORD</code>	Sí	Contraseña de la cuenta integrada <code>admin</code> . Sin ella, el login responde 503
<code>SECRET_KEY</code>	Sí (en producción)	Firma de las cookies de sesión. Render la genera. Si falta, se genera una aleatoria en cada arranque (y las sesiones se pierden al reiniciar)
<code>API_KEY</code>	Sí	Clave de la API (subir borradores, avisos del programador). Render la genera
<code>IG_USER_ID</code>	Sí	ID numérico de la cuenta de Instagram profesional
<code>IG_ACCESS_TOKEN</code>	Sí	Token de Instagram de larga duración (valor inicial; luego lo renueva la app)
<code>PUBLIC_BASE_URL</code>	No	URL pública de la app, si quieres fijarla en vez de deducirla de la petición
<code>IG_API_VERSION</code>	No	Versión de la API de Instagram (por defecto <code>v23.0</code>)
<code>APP_ENV</code>	No	<code>dev</code> desactiva la cookie «solo HTTPS» para trabajar en local
<code>SQLITE_PATH</code>	No	Ruta de la base SQLite local (por defecto <code>app/local.db</code>)
<code>SCHEDULE_FILE</code>	No	Ruta de la programación inicial (por defecto <code>app/schedule.txt</code>)
<code>PORT</code>	No	Puerto en local (lo pone Render en producción)

4. Instagram: cuenta, token y cliente de la API

Este es el capítulo con más dependencias externas. Conviene hacerlo el primero: sin un token válido no se puede probar la publicación real.

NOTA Meta cambia con frecuencia los nombres de los menús, los casos de uso y los requisitos de revisión de sus aplicaciones. Los pasos de la sección 4.2 son un **mapa orientativo**; contrástalos con la documentación oficial (developers.facebook.com, sección «Instagram Platform», API con inicio de sesión de Instagram) antes de dar por bueno cualquier detalle.

4.1 Qué API se usa

Se usa la **API de Instagram con inicio de sesión de Instagram** («Instagram API with Instagram Login»). Sus llamadas van a `graph.instagram.com`. Tiene dos ventajas frente a la antigua vía por Facebook: no necesitas una Página de Facebook vinculada y los permisos son específicos de Instagram.

Necesitas:

- Una cuenta de Instagram **profesional** (empresa o creador). Las cuentas personales no pueden publicar por API.
- Una **app de Meta** con el producto de Instagram añadido.
- Un **token de acceso** de esa cuenta, con dos permisos: `instagram_business_basic` y `instagram_business_content_publish`.
- El **ID numérico** de la cuenta de Instagram.

4.2 Pasos (orientativos)

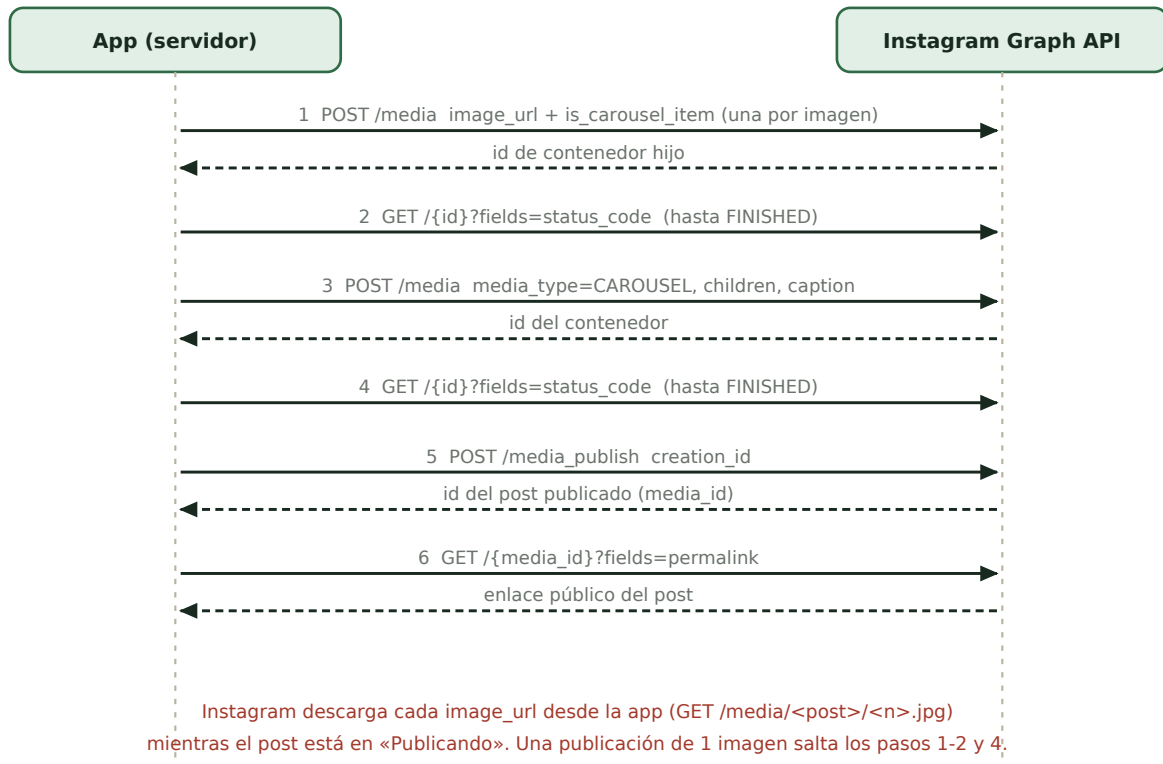
1. **Convierte la cuenta a profesional:** en la app de Instagram, Ajustes → Cuenta → Cambiar a cuenta profesional.
2. **Crea la app de Meta:** en developers.facebook.com → «Mis apps» → «Crear app». Elige el caso de uso relacionado con administrar contenido o mensajes de Instagram y dale un nombre (en el proyecto original se llama «Carruseles y Post»).
3. **Añade el producto Instagram** y abre su sección «API setup with Instagram login».
4. **Da acceso a tu cuenta:** mientras la app está en modo de desarrollo, tu cuenta debe tener un rol en la app (por ejemplo, «Instagram tester»). Añádela en los roles de la app y **acepta la invitación** desde Instagram (Ajustes → Aplicaciones y sitios web → invitaciones de probador).
5. **Genera el token** de esa cuenta desde la misma pantalla de configuración, marcando los dos permisos de arriba. Comprueba en la documentación si el token generado es de larga duración (60 días); si es de corta duración, hay que canjearlo por uno de larga duración.
6. **Obtén el ID de la cuenta:** aparece en la pantalla de configuración, o consultando el perfil con el token (ver el recuadro siguiente).

COMPRUEBA Con el token en una variable de tu terminal (nunca pegado en un archivo del repositorio ni en un chat), esta llamada debe devolver el nombre de usuario y el ID de la cuenta: `curl "https://graph.instagram.com/v23.0/me?fields=user_id,username&access_token=$TOKEN"`. Si responde con un error de token, repite el paso 5.

CUIDADO El ID de la cuenta **no** es un secreto (puede aparecer en documentación), pero el **token sí lo es**. Guardarlos cruzados (el token en la variable del ID y al revés) ya provocó un incidente en este proyecto: la API devolvió un error con el token dentro y acabó escrito en un archivo público. Comprueba siempre qué valor pones en cada variable.

4.3 Cómo se publica por API: contenedores

La API **no** recibe la imagen: recibe **la dirección pública de la imagen** y la descarga ella. Publicar es un proceso en dos fases: primero se crea un **contenedor** (Instagram descarga y procesa la imagen) y después se **publica** el contenedor.



Tipo de post	Llamadas
Publicación de 1 imagen	POST /{id}/media (image_url, caption) → esperar → POST /{id}/media_publish
Carrusel (2 a 10 imágenes)	Un POST /{id}/media por imagen con is_carousel_item=true → esperar a cada uno → POST /{id}/media con media_type=CAROUSEL, children=id1,id2,... y caption → esperar → media_publish
Historia	POST /{id}/media con media_type=STORIES y image_url (sin caption) → esperar → media_publish

Esperar significa consultar `GET /{contenedor}?fields=status_code` hasta que valga `FINISHED`. Si vale `ERROR` o `EXPIRED`, el contenedor no sirve. Instagram tarda unos segundos, por eso el cliente consulta cada 3 segundos hasta 40 veces (2 minutos).

Restricciones que la API impone (y que la app valida antes de aceptar un borrador, capítulo 9.11):

Regla	Valor
Formato de imagen	JPEG (PNG y otros no se admiten)
Tamaño máximo por imagen	8 MB

Regla	Valor
Proporción en el feed	Entre 4:5 (0,8) y 1,91:1
Imágenes de un carrusel	De 2 a 10
Historias	Una imagen, proporción 9:16
Texto de historia	La API no admite texto de publicación en historias: todo el mensaje va en la imagen
URL de la imagen	Pública y accesible desde los servidores de Meta (no vale <code>localhost</code>)
Texto de publicación	Máx. 2200 caracteres y 30 hashtags

NOTA Además, Meta limita cuántas publicaciones por API puede hacer una cuenta cada 24 horas. Con la cadencia de este proyecto (tres posts a la semana) no es un problema, pero consulta el límite vigente en la documentación si piensas publicar mucho más.

4.4 El token: 60 días y renovación

Un token de larga duración **caduca a los 60 días**. Se puede renovar con `GET https://graph.instagram.com/refresh_access_token?grant_type=ig_refresh_token&access_token=...`, pero solo si el token **tiene más de 24 horas y aún no ha caducado**. Si caduca, la única salida es generar uno nuevo a mano (paso 5 de 4.2).

Por eso la app renueva el token **cada 20 días** (`TOKEN_REFRESH_DAYS`, capítulo 9.4): cada vez que alguien inicia sesión o se va a publicar, comprueba cuándo fue la última renovación y, si han pasado 20 días, renueva y guarda el nuevo token en la base de datos. Mientras la app se use al menos una vez al mes, el token no caduca.

4.5 El cliente: `app/instagram.py`

Es un cliente mínimo, sin librerías específicas: usa `requests`. Sus piezas:

- **Constantes.** `API_VERSION` (por defecto `v23.0`, cambiable con `IG_API_VERSION`) y `API`, la URL base. `POLL_SECONDS` y `POLL_ATTEMPTS` fijan la espera de cada contenedor.
- `IGError`. Una excepción propia: todo error de la API se convierte en ella con un mensaje legible.
- `api_call(method, path, token, **params)`. Hace la petición con el token como parámetro, intenta leer JSON (si no puede, guarda el principio del texto) y lanza `IGError` si el código es 400 o superior o si la respuesta trae `error`. Es el **único** sitio donde se llama a Instagram.
- `wait_until_ready(container_id, token)`. El sondeo descrito arriba.
- `publish(image_urls, caption, user_id, token, kind)`. Decide el camino según el tipo (historia, publicación de una imagen o carrusel), espera, publica y pide el enlace público (`permalink`). Si pedir el enlace falla, **no** se considera un error: el post ya está publicado y el enlace es solo informativo. Devuelve `{"media_id", "permalink"}`.
- `refresh(token)`. La renovación del token; devuelve el token nuevo y los segundos de vida.

POR QUÉ Todas las llamadas pasan por `api_call` y `publish` lanza un único tipo de excepción. Así la app (capítulo 9.8) solo tiene que capturar `Exception` una vez, marcar el post como fallido y **redactar** el mensaje (quitar el token) antes de guardarlo. Y en las pruebas basta con sustituir `instagram.publish` por una función falsa.

`app/instagram.py` – cliente de la API de Instagram

```
"""Cliente mínimo de la API de Instagram (Instagram Login) para publicar imágenes y carruseles."""
import os
import time
```

... continuación

```
import requests

API_VERSION = os.environ.get("IG_API_VERSION", "v23.0")
API = f"https://graph.instagram.com/{API_VERSION}"
POLL_SECONDS = 3
POLL_ATTEMPTS = 40

class IGEError(Exception):
    """Error de la API de Instagram."""

def api_call(method: str, path: str, token: str, **params) -> dict:
    params["access_token"] = token
    resp = requests.request(method, f"{API}/{path}", params=params, timeout=60)
    try:
        body = resp.json()
    except ValueError:
        body = {"raw": resp.text[:300]}
    if resp.status_code >= 400 or "error" in body:
        err = body.get("error", body)
        msg = err.get("message") if isinstance(err, dict) else str(err)
        raise IGEError(f"API {resp.status_code}: {msg}")
    return body

def wait_until_ready(container_id: str, token: str) -> None:
    for _ in range(POLL_ATTEMPTS):
        status = api_call("GET", container_id, token, fields="status_code").get("status_code")
        if status == "FINISHED":
            return
        if status in ("ERROR", "EXPIRED"):
            raise IGEError(f"el contenedor {container_id} quedó en estado {status}")
        time.sleep(POLL_SECONDS)
    raise IGEError(f"el contenedor {container_id} no terminó de procesarse a tiempo")

def publish(image_urls: list[str], caption: str, user_id: str, token: str, kind: str = "carrusel") -> dict:
    """Publica una historia (1 imagen), una publicación (1 imagen) o un carrusel (2-10).
    Devuelve {"media_id", "permalink"}. Las historias no admiten texto de publicación."""
    if kind == "historia":
        container = api_call("POST", f"{user_id}/media", token,
                             media_type="STORIES", image_url=image_urls[0])["id"]
    elif len(image_urls) == 1:
        container = api_call("POST", f"{user_id}/media", token,
                             image_url=image_urls[0], caption=caption)["id"]
    else:
        children = [api_call("POST", f"{user_id}/media", token,
                             image_url=url, is_carousel_item="true")["id"] for url in image_urls]
        for child in children:
            wait_until_ready(child, token)
        container = api_call("POST", f"{user_id}/media", token, media_type="CAROUSEL",
                             children=".".join(children), caption=caption)["id"]
    wait_until_ready(container, token)
    media_id = api_call("POST", f"{user_id}/media_publish", token, creation_id=container)["id"]
    permalink = ""
    try:
        permalink = api_call("GET", media_id, token, fields="permalink").get("permalink", "")
    except IGEError:
        pass # ya está publicado; el permalink es solo informativo
    return {"media_id": media_id, "permalink": permalink}

def refresh(token: str) -> tuple[str, int]:
    """Renueva un token de larga duración (debe tener >24 h y no haber caducado)."""
    resp = requests.get("https://graph.instagram.com/refresh_access_token",
                        params={"grant_type": "ig_refresh_token", "access_token": token}, timeout=30)
    try:
        body = resp.json()
    except ValueError:
        body = {}
    if resp.status_code >= 400 or "access_token" not in body:
        err = body.get("error", {})
        raise IGEError(f"no se pudo renovar el token: {err.get('message')} if isinstance(err, dict) else body}")
    return body["access_token"], int(body.get("expires_in", 0))
```

5. La base de datos

5.1 Un adaptador para dos motores

`app/db.py` decide el motor según `DATABASE_URL`: si empieza por `postgres://` o `postgresql://`, usa **Postgres** (`psycopg`); si no, usa **SQLite** en un archivo local. El resto de la app no sabe cuál está usando.

Lo consigue con cuatro reglas:

1. **Solo SQL estándar** y parámetros con `%s`. Para SQLite, `_run` sustituye `%s` por `?` justo antes de ejecutar.
2. **Modo autocommit** en los dos motores: cada sentencia se confirma sola. Las operaciones que necesitan atomicidad se escriben como **una sola sentencia** (`UPDATE ... WHERE ... RETURNING ...`), no como transacciones largas.
3. **Filas como diccionarios** (`dict_row` en Postgres, `sqlite3.Row` en SQLite), y `query/one/execute` devuelven siempre `dict` o el número de filas afectadas.
4. **Fechas como texto ISO en UTC**. Al estar en el mismo formato, se pueden **comparar como texto** (`value < slot_iso`) y ordenar con `ORDER BY`. Esto se usa en la cola y en el reclamo de huecos.

CUIDADO `RETURNING` en `UPDATE/DELETE` funciona en Postgres y en SQLite **3.35 o superior** (Python 3.12/3.13 lo incluyen). `ON CONFLICT` también se usa en ambos. Si en un entorno antiguo SQLite falla, actualiza Python, no cambies el SQL.

5.2 El esquema

Cinco tablas. Se crean solas al arrancar (`CREATE TABLE IF NOT EXISTS`).

`posts` — un registro por post, con su ciclo de vida.

Columna	Tipo	Contenido
<code>id</code>	texto (PK)	UUID sin guiones
<code>title</code>	texto	Nombre corto para el panel (máx. 120 caracteres)
<code>status</code>	texto	<code>draft</code> , <code>approved</code> , <code>publishing</code> , <code>published</code> , <code>failed</code> , <code>rejected</code>
<code>kind</code>	texto	<code>carrusel</code> , <code>publicacion</code> o <code>historia</code>
<code>caption</code> , <code>hashtags</code> , <code>slides_text</code>	texto	Texto de publicación; hashtags y texto de las slides como JSON
<code>source_url</code>	texto	URL de <code>trabajoenexcel.com</code> que promociona
<code>created_at</code> , <code>updated_at</code>	texto	Fechas ISO (UTC)
<code>approved_at</code>	texto	Fecha con microsegundos : ordena la cola sin empates
<code>approved_by</code>	texto	Usuario que lo aprobó
<code>published_at</code> , <code>media_id</code> , <code>permalink</code>	texto	Datos de la publicación en Instagram
<code>error</code>	texto	Motivo del fallo (redactado)

`images` — las imágenes temporales.

Columna	Contenido
<code>post_id</code>	Referencia a <code>posts</code> con <code>ON DELETE CASCADE</code> (al borrar el post, se borran sus imágenes)
<code>position</code>	Orden de la imagen (0, 1, 2...)
<code>data</code>	El JPEG (<code>BYTEA</code> en Postgres, <code>BLOB</code> en SQLite). Clave primaria: (<code>post_id</code> , <code>position</code>)

kv — pares clave-valor para el estado interno: el token renovado, la fecha del último aviso del programador, los huecos ya usados (`last_slot:<tipo>`) y los ajustes de programación.

users — usuarios del panel: `id`, `username` (único), `password_hash`, `role` (admin o editor), `created_at`, `created_by`.

schedule_slots — los huecos de programación: `id`, `weekday` (0 = lunes ... 6 = domingo), `slot_time` (HH:MM), `kind`, `created_at`; con una restricción `UNIQUE (weekday, slot_time, kind)` que impide duplicados.

5.3 Migraciones

Las bases de datos que ya existen no se recrean: `CREATE TABLE IF NOT EXISTS` no añade columnas nuevas. Cuando el modelo crece, `init_db()` comprueba si falta una columna y la añade con `ALTER TABLE`. `_has_column` lo averigua con `information_schema` en Postgres y con `PRAGMA table_info` en SQLite. Así ya se añadieron `kind` (cuando aparecieron los tres tipos) y `approved_by` (cuando llegó el multiusuario). Las tablas nuevas (`users`, `schedule_slots`) no necesitan migración: `IF NOT EXISTS` las crea.

POR QUÉ No se usa un sistema de migraciones (Alembic, etc.) porque el modelo es pequeño y las migraciones son aditivas. Si algún día necesitas cambiar o borrar columnas, ahí sí conviene adoptar uno.

5.4 El almacén clave-valor

`kv_get(conn, key)` y `kv_set(conn, key, value)` guardan pequeños datos sueltos con `INSERT ... ON CONFLICT DO UPDATE` (un «upsert»). Se usa para: `ig_token`, `ig_token_refreshed_at`, `ig_token_tried_at`, `ig_token_env_fp`, `last_tick_at`, `last_slot:<tipo>`, `schedule_seeded`, `schedule_tz` y `schedule_tolerance`.

app/db.py — acceso a datos, esquema y migraciones

```
"""Acceso a datos: Postgres (Neon) en producción, SQLite en local.

Solo se usa SQL estándar y parámetros `s`, para que valga en los dos motores.
Las fechas se guardan como texto ISO (UTC).
"""
import datetime as dt
import os
import sqlite3
from contextlib import contextmanager
from pathlib import Path

DATABASE_URL = os.environ.get("DATABASE_URL", "")
IS_PG = DATABASE_URL.startswith(("postgres://", "postgresql://"))
SQLITE_PATH = os.environ.get("SQLITE_PATH", str(Path(__file__).with_name("local.db")))
BLOB = "BLOB" if IS_PG else "BLOB"

SCHEMA = [
    """CREATE TABLE IF NOT EXISTS posts (
        id TEXT PRIMARY KEY,
        title TEXT NOT NULL,
        status TEXT NOT NULL,
        kind TEXT NOT NULL DEFAULT 'carrusel',
        caption TEXT NOT NULL,
        hashtags TEXT NOT NULL,
        slides_text TEXT NOT NULL,
        source_url TEXT,
        created_at TEXT NOT NULL,
        updated_at TEXT NOT NULL,
        approved_at TEXT,
        published_at TEXT,
        media_id TEXT,
        permalink TEXT,
        error TEXT
    )""",
    f"""CREATE TABLE IF NOT EXISTS images (
        post_id TEXT NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
        position INTEGER NOT NULL,
        data {BLOB} NOT NULL,
        PRIMARY KEY (post_id, position)
    )""",
    """CREATE TABLE IF NOT EXISTS kv (
```

... continuación

```

        key TEXT PRIMARY KEY,
        value TEXT NOT NULL,
        updated_at TEXT NOT NULL
    )""",
    """CREATE TABLE IF NOT EXISTS users (
        id TEXT PRIMARY KEY,
        username TEXT NOT NULL UNIQUE,
        password_hash TEXT NOT NULL,
        role TEXT NOT NULL,
        created_at TEXT NOT NULL,
        created_by TEXT
    )""",
    # created_at: un hueco recién creado no rescata ocurrencias anteriores a su creación
    """CREATE TABLE IF NOT EXISTS schedule_slots (
        id TEXT PRIMARY KEY,
        weekday INTEGER NOT NULL,
        slot_time TEXT NOT NULL,
        kind TEXT NOT NULL,
        created_at TEXT NOT NULL,
        UNIQUE (weekday, slot_time, kind)
    )""",
]

def now() -> str:
    return dt.datetime.now(dt.timezone.utc).isoformat(timespec="seconds")

def now_precise() -> str:
    """Con microsegundos: para ordenar la cola sin empates."""
    return dt.datetime.now(dt.timezone.utc).isoformat(timespec="microseconds")

@contextmanager
def connect():
    if IS_PG:
        import psycopg
        from psycopg.rows import dict_row
        conn = psycopg.connect(DATABASE_URL, autocommit=True, row_factory=dict_row, connect_timeout=15)
    else:
        conn = sqlite3.connect(SQLITE_PATH, timeout=30, isolation_level=None)
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
    try:
        yield conn
    finally:
        conn.close()

def _run(conn, sql, params):
    if not IS_PG:
        sql = sql.replace("%s", "?")
    return conn.execute(sql, params)

def query(conn, sql, params=()):
    """Ejecuta y devuelve todas las filas como dicts (también con RETURNING)."""
    return [dict(r) for r in _run(conn, sql, params).fetchall()]

def one(conn, sql, params=()):
    rows = query(conn, sql, params)
    return rows[0] if rows else None

def execute(conn, sql, params=()) -> int:
    return _run(conn, sql, params).rowcount

def _has_column(conn, table: str, column: str) -> bool:
    if IS_PG:
        return bool(query(conn, (
            "SELECT 1 FROM information_schema.columns "
            "WHERE table_schema = current_schema() AND table_name = %s AND column_name = %s", (table, column)))
        return any(r["name"] == column for r in query(conn, f"PRAGMA table_info({table})"))

def init_db():
    with connect() as conn:
        for stmt in SCHEMA:
            execute(conn, stmt)
    # migraciones: bases de datos creadas antes de que existiera el tipo de post

```

... continuación

```
if not _has_column(conn, "posts", "kind"):
    execute(conn, "ALTER TABLE posts ADD COLUMN kind TEXT NOT NULL DEFAULT 'carrusel'")
if not _has_column(conn, "posts", "approved_by"):
    execute(conn, "ALTER TABLE posts ADD COLUMN approved_by TEXT")

def kv_get(conn, key):
    row = one(conn, "SELECT value FROM kv WHERE key = %s", (key,))
    return row["value"] if row else None

def kv_set(conn, key, value):
    execute(conn,
        "INSERT INTO kv (key, value, updated_at) VALUES (%s, %s, %s) "
        "ON CONFLICT (key) DO UPDATE SET value = excluded.value, updated_at = excluded.updated_at",
        (key, value, now()))
```

6. La regla de contenido (CTA prohibidos)

6.1 La regla

Por decisión de negocio, **ningún slide ni texto de publicación puede pedir que el público comente para recibir un enlace, ni remitir a «el link de la bio»/«la descripción», ni pedir un mensaje directo**. Ejemplos rechazados: «Comenta EXCEL y te dejo el link», «Link en la bio», «Escríbeme por DM». Todo llamado a la acción debe **nombrar la web** con una acción concreta («Haz el test gratis en trabajoenexcel.com»).

6.2 Por qué se aplica en el servidor

La skill de Claude ya conoce la regla, pero **no se confía en el generador**: el servidor la aplica siempre, en tres puntos:

1. Al **subir** un borrador (`POST /api/drafts`) → responde 422.
2. Al **aprobar** un post → si incumple, no se encola.
3. Justo antes de **publicar** → si incumple, el post pasa a `failed`.

De este modo, aunque alguien cambie el generador, edite la base de datos o suba un borrador a mano, el contenido prohibido no llega a Instagram.

6.3 Cómo funciona

`app/guards.py` define una lista de expresiones regulares con su etiqueta. `check(*textos)` las aplica a cada texto y devuelve la lista de incumplimientos (vacía si todo está bien).

Patrón	Detecta
<code>comenta ... link/enlace</code> (hasta 60 caracteres entre medias)	«Comenta y te dejo el link»
<code>comenta + PALABRA EN MAYÚSCULAS</code> (3+ letras)	«Comenta EXCEL»
<code>link/enlace en/de (la/mi/nuestra) bio/biografía/descripción/perfil</code>	«Link en la bio»
<code>(te) (lo) dejo/paso/mando/envío/comparto el/un link/enlace</code>	«Te dejo el link»
<code>por DM / por mensaje privado / directo</code>	«Escríbeme por DM»

Un detalle importante de las expresiones: `(?i:comenta)` aplica «ignorar mayúsculas» **solo a esa palabra**. En el patrón «Comenta PALABRA», lo que sigue `([A-ZÁÉÍÓÚÑ]{3,})` sí distingue mayúsculas: así «Comenta lo que piensas» no se rechaza, pero «Comenta EXCEL» sí.

NOTA Es una defensa razonable, no una garantía absoluta: un texto muy retorcido podría esquivarla. Por eso la revisión humana en el panel sigue siendo la última barrera. Cuando descubras una fórmula nueva que se cuela, añade un patrón y una prueba.

app/guards.py – restricción de contenido

```
"""Restricción de contenido: prohibidos los CTA que dependen de enviar un enlace.
```

```
Se aplica en el servidor al subir un borrador y otra vez al aprobarlo, para que no dependa de que el generador se acuerde de la regla.
```

```
"""
import re
```

```
_PATTERNS = [
    (re.compile(r"(?i:comenta)\b.{0,60}?\\b(?:link|enlace)\b", re.S), "«Comenta ... link»"),
    (re.compile(r"(?i:comenta)\s+[\s'"]?[A-ZÁÉÍÓÚÑ]{3,}"), "«Comenta PALABRA»"),
    (re.compile(r"(?i)\b(link|enlace)s?\s+(en|de)\s+(la\s+|mi\s+|nuestra\s+)?(bio|biograf[í]a|descripci[ó]n|per
```

... continuación

```
↳ fil)),
    "«link en la bio/descripción»"),
    (re.compile(r"(?i)\b(te\s+)?(lo\s+)?(dejo|paso|mando|env[ií]o|comparto)\s+(el|un)\s+(link|enlace)"),
    "«te dejo/paso el link»"),
    (re.compile(r"(?i)\bpor\s+(dm|md|mensaje\s+(privado|directo))\b"), "«por DM»"),
]

def check(*texts: str) -> list[str]:
    """Devuelve una lista de incumplimientos (vacía si el texto es válido)."""
    found = []
    for text in texts:
        for pattern, label in _PATTERNS:
            m = pattern.search(text or "")
            if m:
                snippet = " ".join(m.group(0).split())[:60]
                found.append(f"{label}: «{snippet}»")
    return list(dict.fromkeys(found))
```

7. La programación: días, horas y huecos

7.1 Conceptos

Un **hueco** (*slot*) es una combinación **día de la semana + hora + tipo de post**. Ejemplo: «martes 15:30 carrusel». Reglas:

- En cada hueco se publica **un** post de ese tipo: el **aprobado más antiguo** de la cola de ese tipo.
- Si en el momento del hueco no hay ningún post aprobado de ese tipo, el hueco **se pierde**. No se acumula ni se recupera.
- Dos huecos a la misma hora con tipos distintos salen juntos (por ejemplo, publicación e historia el jueves a las 19:00).
- Las horas son de una **zona horaria** configurable (por defecto `Europe/Madrid`), con cambio de hora verano/invierno automático.
- La **tolerancia** (por defecto 120 minutos) es cuánto puede retrasarse el aviso del programador y aún publicar.

Programación inicial del proyecto: **martes 15:30** → carrusel; **jueves 19:00** → publicación e historia.

7.2 Dos capas de código

- `schedule.py` contiene la **lógica pura**: no toca la base de datos. Se puede probar solo. Sabe leer un texto de programación, construir una programación a partir de filas y calcular qué huecos están vigentes.
- `schedule_store.py` es la **capa de persistencia**: lee y escribe los huecos en las tablas `schedule_slots` y `kv`.

7.3 schedule.txt: la programación inicial

Un archivo de texto sencillo, con comentarios `#`:

- Ajustes opcionales: `zona: Europe/Madrid` y `tolerancia_minutos: 120`.
- Un hueco por línea: `martes 15:30 carrusel`.
- Días: `lunes martes miércoles jueves viernes sábado domingo` (con o sin tilde).
- Tipos: `carrusel`, `publicacion` (o `publicación`) y `historia`.

Solo se lee una vez: la primera vez que la app necesita la programación y la base de datos aún no la tiene, se copia a `schedule_slots` y `kv`, y se marca como «sembrada» (`schedule_seeded`). A partir de ahí manda la base de datos y editar el archivo no cambia nada.

app/schedule.txt – programación inicial

```
# Programación INICIAL de publicaciones
#
# OJO: este archivo solo se lee la primera vez que arranca la app, para copiar estos huecos a la
# base de datos. Después, los horarios se cambian en el panel (Programación) y editar este
# archivo ya no tiene ningún efecto.
#
# Cada línea es un hueco: día HH:MM tipo
#
# Días: lunes martes miércoles jueves viernes sábado domingo
# Tipos: carrusel | publicacion | historia
#
# En cada hueco se publica UN post de ese tipo: el más antiguo de los aprobados
# (la cola de ese tipo). Si a esa hora no hay ninguno aprobado, el hueco se pierde
# y no se recupera después. Dos huecos a la misma hora (con tipos distintos) salen juntos.
#
# Las horas son de la zona indicada abajo (con cambio de hora verano/invierno automático).
#
# Un aviso del programador llega cada 30 minutos, así que un post sale como mucho
```

... continuación

```
# unos 30 min después de la hora indicada.

zona: Europe/Madrid

# Minutos de margen tras la hora del hueco durante los que aún se puede usar
# (cubre retrasos del aviso de GitHub Actions).
tolerancia_minutos: 120

martes 15:30 carrusel
jueves 19:00 publicacion
jueves 19:00 historia
```

7.4 La lógica de huecos (schedule.py)

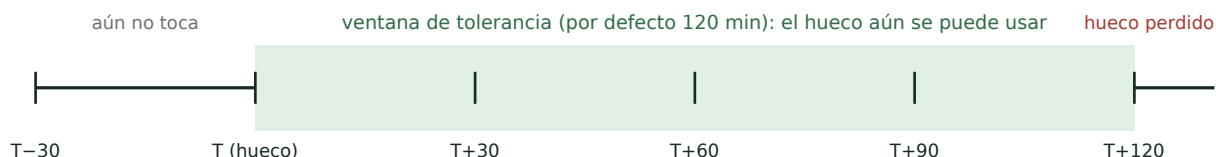
Schedule

Guarda la zona horaria, la tolerancia y la lista de huecos como tuplas (día, hora, tipo). Métodos:

- `slot_times(start, end, kind)` — todas las ocurrencias de huecos de un tipo entre dos instantes (en UTC). Recorre los días desde el día anterior a `start` hasta el posterior a `end`; para cada día de la semana coincidente construye la hora **en la zona horaria elegida** y la convierte a UTC. Como la conversión la hace `zoneinfo` para esa fecha concreta, el **cambio de hora** verano/invierno sale bien sin cálculos manuales.
- `due_slots(now)` — por cada tipo, el hueco **más reciente** que ya empezó y sigue dentro de la tolerancia. Es lo que consulta el aviso del programador.
- `upcoming(now, last_used, n, kind)` — los próximos `n` huecos aprovechables de un tipo, descartando los ya usados. Alimenta la «hora prevista» («Sale: martes 22 sep, 15:30») que muestra el panel.
- `label(slot)` y `describe()` — textos legibles para el panel.

La tolerancia

Cada marca es un aviso del programador (cada 30 min). El primero desde T reclama el hueco y publica.



Con horas que no son :00 o :30, el post sale en el primer aviso posterior (hasta ~30 min de retraso).

El programador (GitHub Actions) avisa cada 30 minutos, pero GitHub no despierta a la hora exacta: puede retrasarse minutos. Por eso un hueco no es «el instante T», sino la **ventana [T, T + tolerancia]**. El primer aviso que caiga dentro de la ventana publica; si pasa la ventana sin aviso (por una caída), el hueco se pierde. Con horas que no son :00 ni :30, el post sale en el primer aviso posterior, es decir, hasta ~30 minutos más tarde.

El campo «desde cuándo vale» (`since`)

Cada hueco recuerda **cuándo se creó** (`created_at`). Una ocurrencia solo cuenta si es **posterior a la creación del hueco**. Los huecos sembrados desde `schedule.txt` llevan fecha 1970, así que siempre valen.

POR QUÉ Sin esto, si a las 11:00 añades el hueco «hoy 10:00» (dentro de la tolerancia de 120 minutos), el siguiente aviso publicaría inmediatamente algo que **no esperabas**, en una cuenta real. Con `since`, un hueco nuevo solo cuenta a partir de ahora.

Funciones auxiliares

`parse(text)` interpreta un `schedule.txt` (con errores claros por línea); `parse_day`, `parse_time` y `parse_kind` validan los datos del formulario del panel; `build_tz` valida la zona horaria; `check_tolerance` limita la tolerancia a **30-720 minutos** (menos de 30 no tendría sentido porque el programador avisa cada 30); y `build(tz_name, tolerance, rows)` construye una `Schedule` a partir de filas de la base de datos (puede no tener ningún hueco). `load(path)` lee y `parsea` un archivo.

app/schedule.py – lógica de huecos (sin base de datos)

```
"""Huecos de publicación. Cada hueco es «día HH:MM tipo» y en cada uno se publica UN post de ese tipo.
```

```
La programación vigente vive en la base de datos y se edita desde el panel (schedule_store.py).
schedule.txt (formato explicado en ese archivo) solo es la programación inicial: se copia a la
base de datos la primera vez que arranca la app.
"""
```

```
import datetime as dt
import os
import re
from pathlib import Path
from zoneinfo import ZoneInfo, ZoneInfoNotFoundError
```

```
SCHEDULE_FILE = os.environ.get("SCHEDULE_FILE", str(Path(__file__).with_name("schedule.txt")))
UTC = dt.timezone.utc
EPOCH = dt.datetime(1970, 1, 1, tzinfo=UTC)
MIN_TOLERANCE, MAX_TOLERANCE = 30, 720 # minutos; el programador avisa cada 30
```

```
KINDS = ("carrusel", "publicacion", "historia")
KIND_ALIASES = {"carrusel": "carrusel", "publicacion": "publicacion", "publicación": "publicacion",
                "historia": "historia"}
KIND_LABELS = {"carrusel": "carrusel", "publicacion": "publicación", "historia": "historia"}
```

```
DAYS = {"lunes": 0, "martes": 1, "miercoles": 2, "miércoles": 2, "jueves": 3,
        "viernes": 4, "sabado": 5, "sábado": 5, "domingo": 6}
DAY_NAMES = ["lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo"]
MONTHS = ["ene", "feb", "mar", "abr", "may", "jun", "jul", "ago", "sep", "oct", "nov", "dic"]
```

```
class ScheduleError(Exception):
    """schedule.txt no es válido."""
```

```
class Schedule:
```

```
    def __init__(self, tz: ZoneInfo, tz_name: str, tolerance: dt.timedelta,
                 slots: list[tuple[int, dt.time, str]],
                 since: dict[tuple[int, dt.time, str], dt.datetime] | None = None):
        self.tz, self.tz_name, self.tolerance, self.slots = tz, tz_name, tolerance, slots
        # desde cuándo vale cada hueco: uno recién creado no publica a posteriori una hora ya pasada
        self.since = since or {}
```

```
    def kinds(self) -> set[str]:
        return {kind for _, _, kind in self.slots}
```

```
    def slot_times(self, start: dt.datetime, end: dt.datetime, kind: str) -> list[dt.datetime]:
        """Huecos de ese tipo (en UTC) con start <= hueco <= end, ordenados."""
        out = []
        day = (start.astimezone(self.tz) - dt.timedelta(days=1)).date()
        last_day = (end.astimezone(self.tz) + dt.timedelta(days=1)).date()
        while day <= last_day:
            for weekday, at, slot_kind in self.slots:
                if slot_kind == kind and day.weekday() == weekday:
                    slot = dt.datetime.combine(day, at, tzinfo=self.tz).astimezone(UTC)
                    if start <= slot <= end and slot >= self.since.get((weekday, at, kind), EPOCH):
                        out.append(slot)
            day += dt.timedelta(days=1)
        return sorted(out)
```

```
    def due_slots(self, now: dt.datetime) -> dict[str, dt.datetime]:
        """Por tipo, el hueco más reciente que ya empezó y sigue dentro de la tolerancia."""
        due = {}
        for kind in KINDS:
            candidates = self.slot_times(now - self.tolerance, now, kind)
            if candidates:
                due[kind] = candidates[-1]
        return due
```

```
    def upcoming(self, now: dt.datetime, last_used: dt.datetime | None, n: int, kind: str) -> list[dt.datetime]:
        """Próximos n huecos aprovechables de ese tipo (los ya usados no cuentan)."""
        found = []
        for slot in self.slot_times(now - self.tolerance, now + dt.timedelta(days=60), kind):
            if last_used and slot <= last_used:
```

... continuación

```

        continue
    found.append(slot)
    if len(found) == n:
        break
    return found

def label(self, slot: dt.datetime) -> str:
    local = slot.astimezone(self.tz)
    return f"{DAY_NAMES[local.weekday()]} {local.day} {MONTHS[local.month - 1]}, {local:%H:%M}"

def describe(self) -> str:
    if not self.slots:
        return "ninguno"
    return ", ".join(f"{DAY_NAMES[d]} {t:%H:%M} ({KIND_LABELS[k]})" for d, t, k in sorted(self.slots))

def parse(text: str) -> Schedule:
    tz_name, tolerance, slots = "Europe/Madrid", dt.timedelta(minutes=120), []
    for number, raw in enumerate(text.splitlines(), start=1):
        line = raw.split("#", 1)[0].strip()
        if not line:
            continue
        if ":" in line and line.split(":", 1)[0].strip().lower() in ("zona", "tolerancia_minutos"):
            key, value = (p.strip() for p in line.split(":", 1))
            if key.lower() == "zona":
                tz_name = value
            else:
                if not value.isdigit():
                    raise ScheduleError(f"línea {number}: tolerancia_minutos debe ser un número")
                tolerance = dt.timedelta(minutes=int(value))
            continue
        m = re.fullmatch(r"([A-Za-záéíóúÁÉÍÓÚñÑ]+)\s+(\d{1,2}):(\d{2})\s+([A-Za-záéíóúÁÉÍÓÚñÑ]+)", line)
        if not m or m.group(1).lower() not in DAYS:
            raise ScheduleError(
                f"línea {number}: no entiendo «{raw.strip()}» (usa «martes 15:30 carrusel»)"
            )
        kind = KIND_ALIASES.get(m.group(4).lower())
        if not kind:
            raise ScheduleError(
                f"línea {number}: tipo desconocido «{m.group(4)}» (usa carrusel, publicacion o historia)"
            )
        hour, minute = int(m.group(2)), int(m.group(3))
        if hour > 23 or minute > 59:
            raise ScheduleError(f"línea {number}: hora no válida")
        slots.append((DAYS[m.group(1).lower()], dt.time(hour, minute), kind))
    if not slots:
        raise ScheduleError("no hay ningún hueco definido")
    tz = build_tz(tz_name)
    return Schedule(tz, tz_name, tolerance, sorted(set(slots)))

def build_tz(tz_name: str) -> ZoneInfo:
    try:
        return ZoneInfo(tz_name)
    except (ZoneInfoNotFoundError, ValueError):
        raise ScheduleError(f"zona horaria desconocida: {tz_name}")

def check_tolerance(minutes) -> int:
    if not isinstance(minutes, int) or not MIN_TOLERANCE <= minutes <= MAX_TOLERANCE:
        raise ScheduleError(f"la tolerancia debe ser un número de minutos entre {MIN_TOLERANCE} y {MAX_TOLERANCE}")
    return minutes

def parse_day(text) -> int:
    text = str(text).strip().lower()
    day = int(text) if text.isdigit() else DAYS.get(text)
    if day is None or not 0 <= day <= 6:
        raise ScheduleError("día no válido")
    return day

def parse_time(text: str) -> dt.time:
    m = re.fullmatch(r"(\d{1,2}):(\d{2})", (text or "").strip())
    if not m or int(m.group(1)) > 23 or int(m.group(2)) > 59:
        raise ScheduleError("hora no válida (usa HH:MM)")
    return dt.time(int(m.group(1)), int(m.group(2)))

def parse_kind(text: str) -> str:
    kind = KIND_ALIASES.get((text or "").strip().lower())
    if not kind:
        raise ScheduleError("tipo no válido (carrusel, publicación o historia)")

```

... continuación

```

    return kind

def build(tz_name: str, tolerance_minutes: int, rows: list[dict]) -> Schedule:
    """Programación a partir de filas de la base de datos (puede no tener ningún hueco)."""
    slots, since = [], {}
    for r in rows:
        key = (r["weekday"], parse_time(r["slot_time"]), r["kind"])
        slots.append(key)
        since[key] = dt.datetime.fromisoformat(r["created_at"])
    return Schedule(build_tz(tz_name), tz_name, dt.timedelta(minutes=tolerance_minutes), sorted(set(slots)),
        ↳ since)

def load(path: str | None = None) -> Schedule:
    try:
        text = Path(path or SCHEDULE_FILE).read_text(encoding="utf-8")
    except OSError as exc:
        raise ScheduleError(f"no se puede leer schedule.txt: {exc}")
    return parse(text)

```

7.5 La persistencia (schedule_store.py)

- `load(conn)` — si aún no se ha sembrado, lo hace (`_seed`); después lee `schedule_slots` y los ajustes de `kv` y construye la `Schedule`. El resto de la app **solo** usa esta función.
- `list_slots(conn)` — los huecos ordenados por día y hora, con el nombre del día y la etiqueta del tipo, para la pantalla.
- `add_slot` — valida día, hora y tipo; inserta con `created_at` preciso; si el `UNIQUE` falla, lo traduce a «Ese hueco ya existe».
- `delete_slot` — elimina un hueco por su id.
- `save_settings` — valida y guarda la zona horaria y la tolerancia.

La sembradura usa `ON CONFLICT DO NOTHING`: si dos peticiones simultáneas intentan sembrar a la vez, no se duplican los huecos gracias a la restricción `UNIQUE`.

NOTA Quién reclama el hueco. `schedule.py` solo dice qué huecos están vigentes. Que **cada hueco se use una sola vez** lo garantiza el servidor con `_claim_slot` (capítulo 9.8): guarda en `kv` el último hueco usado de cada tipo con un `UPDATE ... WHERE value < nuevo` atómico. Solo el primer aviso que lo consigue publica.

app/schedule_store.py – programación en la base de datos

```

"""Programación guardada en la base de datos (se edita desde el panel).

La primera vez que arranca la app, se copia la de schedule.txt; a partir de ahí manda la base
de datos y schedule.txt ya no se lee.
"""
import datetime as dt
import uuid

import db
import schedule
from schedule import Schedule, ScheduleError

KEY_SEEDED, KEY_TZ, KEY_TOLERANCE = "schedule_seeded", "schedule_tz", "schedule_tolerance"

def _seed(conn) -> None:
    cfg = schedule.load() # ScheduleError si schedule.txt no es válido
    for weekday, at, kind in cfg.slots:
        db.execute(conn, ("INSERT INTO schedule_slots (id, weekday, slot_time, kind, created_at) "
            "VALUES (%s, %s, %s, %s, %s) ON CONFLICT DO NOTHING",
            (uuid.uuid4().hex, weekday, f"{at:%H:%M}", kind, schedule.EPOCH.isoformat())))
    db.kv_set(conn, KEY_TZ, cfg.tz_name)
    db.kv_set(conn, KEY_TOLERANCE, str(int(cfg.tolerance.total_seconds() // 60)))
    db.kv_set(conn, KEY_SEEDED, db.now())

def load(conn) -> Schedule:

```

... continuación

```

if db.kv_get(conn, KEY_SEEDED) is None:
    _seed(conn)
rows = db.query(conn, "SELECT weekday, slot_time, kind, created_at FROM schedule_slots")
return schedule.build(db.kv_get(conn, KEY_TZ), int(db.kv_get(conn, KEY_TOLERANCE)), rows)

def list_slots(conn) -> list[dict]:
    """Huecos ordenados por día y hora, con el nombre del día y la etiqueta del tipo."""
    load(conn) # asegura la carga inicial
    rows = db.query(conn, "SELECT id, weekday, slot_time, kind FROM schedule_slots")
    rows.sort(key=lambda r: (r["weekday"], r["slot_time"].zfill(5), r["kind"]))
    for r in rows:
        r["day"] = schedule.DAY_NAMES[r["weekday"]]
        r["kind_label"] = schedule.KIND_LABELS[r["kind"]]
    return rows

def add_slot(conn, day, time_text: str, kind_text: str) -> None:
    load(conn)
    weekday, at, kind = schedule.parse_day(day), schedule.parse_time(time_text), schedule.parse_kind(kind_text)
    try:
        db.execute(conn, ("INSERT INTO schedule_slots (id, weekday, slot_time, kind, created_at) "
                          "VALUES (%s, %s, %s, %s, %s)",
                          (uuid.uuid4().hex, weekday, f"{at:%H:%M}", kind, db.now_precise())))
    except Exception as exc:
        if db.one(conn, "SELECT 1 AS x FROM schedule_slots WHERE weekday = %s AND slot_time = %s AND kind = %s",
                   (weekday, f"{at:%H:%M}", kind)):
            raise ScheduleError("Ese hueco ya existe.") from exc
        raise

def delete_slot(conn, slot_id: str) -> bool:
    return bool(db.query(conn, "DELETE FROM schedule_slots WHERE id = %s RETURNING id", (slot_id,)))

def save_settings(conn, tz_name: str, tolerance_text: str) -> None:
    load(conn)
    tz_name = (tz_name or "").strip()
    schedule.build_tz(tz_name)
    try:
        minutes = int(str(tolerance_text).strip())
    except ValueError:
        minutes = None
    schedule.check_tolerance(minutes)
    db.kv_set(conn, KEY_TZ, tz_name)
    db.kv_set(conn, KEY_TOLERANCE, str(minutes))

```

8. Usuarios, contraseñas y roles

8.1 Modelo

Hay **dos orígenes** de cuenta:

- **admin (integrada).** Su contraseña es la variable de entorno `ADMIN_PASSWORD`. **No** está en la base de datos, no se puede borrar y no se puede cambiar desde el panel. Sirve de rescate: si todos los usuarios se bloquean, se entra con ella.
- **Cuentas del panel.** Se crean en «Usuarios», se guardan en la tabla `users` con la contraseña **cifrada (hash)** y tienen un rol.

Roles:

Rol	Puede
editor	Entrar, ver borradores, aprobar, sacar de la cola, rechazar, reabrir, eliminar posts, cambiar su contraseña
admin	Todo lo anterior + crear/eliminar usuarios + editar la programación

8.2 Decisiones

- **Nombres de usuario:** 3–32 caracteres (`a-z 0-9 . _ -`), guardados en minúsculas para que «Ana» y «ana» sean la misma cuenta. El nombre `admin` está reservado.
- **Contraseñas:** entre 8 y 128 caracteres. Se guardan con `generate_password_hash` de Werkzeug (hash con sal; en las versiones actuales usa *scrypt*). Nunca se guardan ni se muestran en claro.
- **Comprobar sin delatar:** si el usuario no existe, se comprueba igualmente la contraseña contra un hash falso (`_DUMMY_HASH`). Así el tiempo de respuesta no revela si un usuario existe.
- **Condición de carrera al crear:** si dos altas simultáneas usan el mismo nombre, la restricción `UNIQUE` hace que solo una gane y la otra recibe «Ya existe un usuario».
- **Sin «olvidé mi contraseña»:** no hay correo ni recuperación por enlace. Si alguien olvida su contraseña, un administrador elimina la cuenta y la crea de nuevo.
- **Baja inmediata:** la sesión guarda solo el identificador del usuario, y **en cada petición** se comprueba que siga existiendo (capítulo 9.2). Al eliminar una cuenta, deja de funcionar al instante aunque tenga la sesión abierta.

app/accounts.py – usuarios y contraseñas

```
"""Usuarios del panel: alta, baja, contraseñas y comprobación de acceso.

Hay dos orígenes de cuenta:
- «admin»: la cuenta integrada, con la contraseña de ADMIN_PASSWORD (variable de entorno de
  Render). No vive en la base de datos, no se puede borrar y sirve de recuperación.
- Las cuentas creadas desde el panel, guardadas con la contraseña cifrada (hash).
"""
import re
import uuid

from werkzeug.security import check_password_hash, generate_password_hash

import db

ENV_USERNAME = "admin"
ENV_ID = "env"
ROLES = {"admin": "Administrador", "editor": "Editor"}
MIN_PASSWORD, MAX_PASSWORD = 8, 128
_DUMMY_HASH = generate_password_hash("sin-usuario") # para no delatar por el tiempo si un usuario existe

class AccountError(Exception):
    """Dato no válido al crear o cambiar una cuenta (el mensaje se muestra tal cual)."""

def env_user() -> dict:
```

... continuación

```

    return {"id": ENV_ID, "username": ENV_USERNAME, "role": "admin", "builtin": True}

def normalize_username(name: str) -> str:
    name = (name or "").strip().lower()
    if not re.fullmatch(r"[a-z0-9._-]{3,32}", name):
        raise AccountError("El usuario debe tener entre 3 y 32 caracteres: letras, números, punto, guion o guion
        ↳ bajo.")
    if name == ENV_USERNAME:
        raise AccountError("«admin» está reservado para la cuenta integrada.")
    return name

def check_password_rules(password: str) -> None:
    if not MIN_PASSWORD <= len(password or "") <= MAX_PASSWORD:
        raise AccountError(f"La contraseña debe tener entre {MIN_PASSWORD} y {MAX_PASSWORD} caracteres.")

def create(conn, username: str, password: str, role: str, created_by: str) -> dict:
    name = normalize_username(username)
    check_password_rules(password)
    if role not in ROLES:
        raise AccountError("Rol no válido.")
    if db.one(conn, "SELECT 1 AS x FROM users WHERE username = %s", (name,)):
        raise AccountError(f"Ya existe un usuario «{name}».")
    uid = uuid.uuid4().hex
    try:
        db.execute(conn, ("INSERT INTO users (id, username, password_hash, role, created_at, created_by) "
            "VALUES (%s, %s, %s, %s, %s, %s)",
            (uid, name, generate_password_hash(password), role, db.now(), created_by))
    except Exception as exc: # dos altas simultáneas con el mismo nombre: gana la primera
        if db.one(conn, "SELECT 1 AS x FROM users WHERE username = %s", (name,)):
            raise AccountError(f"Ya existe un usuario «{name}».") from exc
        raise
    return {"id": uid, "username": name, "role": role}

def get(conn, uid: str):
    return db.one(conn, "SELECT id, username, role, created_at, created_by FROM users WHERE id = %s", (uid,))

def list_all(conn) -> list[dict]:
    return db.query(conn, "SELECT id, username, role, created_at, created_by FROM users ORDER BY created_at ASC")

def delete(conn, uid: str) -> bool:
    return bool(db.query(conn, "DELETE FROM users WHERE id = %s RETURNING id", (uid,)))

def set_password(conn, uid: str, password: str) -> None:
    check_password_rules(password)
    db.execute(conn, "UPDATE users SET password_hash = %s WHERE id = %s", (generate_password_hash(password),
    ↳ uid))

def verify_password(conn, uid: str, password: str) -> bool:
    row = db.one(conn, "SELECT password_hash FROM users WHERE id = %s", (uid,))
    return bool(row) and check_password_hash(row["password_hash"], password)

def authenticate(conn, username: str, password: str, env_password: str, same) -> dict | None:
    """Usuario si las credenciales son correctas. `same` compara textos en tiempo constante."""
    name = (username or "").strip().lower()
    if name == ENV_USERNAME:
        return env_user() if env_password and same(password or "", env_password) else None
    row = db.one(conn, "SELECT id, username, role, password_hash FROM users WHERE username = %s", (name,))
    ok = check_password_hash(row["password_hash"] if row else _DUMMY_HASH, password or "")
    return {"id": row["id"], "username": row["username"], "role": row["role"]} if row and ok else None

```

9. El servidor Flask (app/server.py)

`server.py` es el corazón de la app: unas 800 líneas que unen todas las piezas anteriores. Se explica **sección por sección**, en el mismo orden en que aparecen en el archivo, con el listado de cada sección justo después. Si reconstruyes el archivo, ve pegando los listados en orden: el resultado es el `server.py` completo.

NOTA Todas las rutas que cambian datos son **POST** y comprueban el token CSRF. Las rutas de lectura son **GET**. Las rutas de la API (`/api/...`) usan una clave en lugar de sesión.

Mapa de rutas:

Ruta	Método	Acceso	Qué hace
<code>/healthz</code>	GET	público	Responde ok (Render lo usa para saber que la app vive)
<code>/login</code> , <code>/logout</code>	GET/POST, POST	público	Entrar y salir
<code>/</code>	GET	sesión	Panel: posts agrupados por estado
<code>/post/<id></code>	GET	sesión	Detalle de un post
<code>/media/<id>/<n>.jpg</code>	GET	sesión, o público solo mientras se publica	Sirve una imagen
<code>/post/<id>/approve</code>	POST	sesión	Aprobar y poner en cola
<code>/post/<id>/unqueue</code>	POST	sesión	Sacar de la cola
<code>/post/<id>/reject</code> , <code>/reopen</code> , <code>/unstick</code> , <code>/delete</code>	POST	sesión	Rechazar, reabrir, marcar fallido, eliminar
<code>/account</code>	GET/POST	sesión	Cambiar la propia contraseña
<code>/admin/users</code> (+ <code>/<id>/delete</code>)	GET/POST	administrador	Gestión de usuarios
<code>/admin/schedule</code> (+ <code>/slots</code> , <code>/slots/<id>/delete</code> , <code>/settings</code>)	GET/POST	administrador	Gestión de la programación
<code>/api/drafts</code>	POST	clave de API	Subir un borrador
<code>/api/posts</code> , <code>/api/posts/<id></code>	GET	clave de API	Listar posts / estado de uno
<code>/api/cron/tick</code>	POST	clave de API	Aviso del programador: publicar lo que toque

9.1 Arranque y configuración

Lo primero es la configuración y la creación de la aplicación.

- **Constantes de negocio.** Límites (2200 caracteres, 10 slides, 30 hashtags, 8 MB por imagen), las proporciones aceptadas y `KIND_RULES`, un diccionario que resume las reglas de cada tipo: cuántas imágenes admite, qué proporción y si lleva texto de publicación. **Añadir un tipo de post nuevo empieza aquí.**
- `utcnow()`. Una función propia para «la hora actual». Las pruebas la sustituyen para simular cualquier momento sin esperar.
- `ProxyFix`. Render pone un intermediario (proxy) delante de la app y termina el HTTPS. `ProxyFix` hace que Flask lea la IP real, el protocolo (`https`) y el host de las cabeceras del proxy. Sin él, las URLs de las imágenes que se dan a Instagram saldrían con `http://` y la IP de todos los visitantes sería la del proxy (lo que rompería el límite de intentos de login).

- **Cookies.** `HttpOnly` (JavaScript no puede leerla), `SameSite=Lax` (no se envía en peticiones cruzadas peligrosas), `Secure` (solo por HTTPS, salvo en modo desarrollo) y duración de 30 días.
- `MAX_CONTENT_LENGTH = 60 MB`. Tope del cuerpo de una subida (un carrusel de 10 imágenes de hasta ~6 MB cada una cabe).
- `db.init_db()`. Crea tablas y aplica migraciones al importar el módulo, es decir, al arrancar.

app/server.py – cabecera, configuración y arranque (líneas 1-68)

```
"""Panel de aprobación de posts de Instagram.

Flujo: Claude sube un borrador (POST /api/drafts) -> el usuario lo revisa en el panel y
lo aprueba (pasa a la cola) -> un aviso periódico (POST /api/cron/tick, desde GitHub
Actions) publica el post más antiguo de la cola en cada hueco de la programación (guardada en
la base de datos y editable desde el panel), sirviendo el servidor las imágenes, y después
BORRA los JPG. Solo se conserva el texto, el estado y el enlace del post. El acceso es por
usuarios (roles «admin» y «editor»); la cuenta integrada «admin» usa ADMIN_PASSWORD.
"""
import datetime as dt
import hashlib
import hmac
import json
import os
import re
import secrets
import threading
import time
import uuid
from functools import wraps

from flask import (Flask, Response, abort, flash, g, jsonify, redirect, render_template,
                  request, session, url_for)
from werkzeug.middleware.proxy_fix import ProxyFix

import accounts
import db
import guards
import instagram
import schedule
import schedule_store
from schedule import KINDS

DEV = os.environ.get("APP_ENV") == "dev"
ADMIN_PASSWORD = os.environ.get("ADMIN_PASSWORD", "") # contraseña de la cuenta integrada «admin»
API_KEY = os.environ.get("API_KEY", "")
MAX_CAPTION = 2200
MAX_SLIDES = 10
MAX_HASHTAGS = 30
MAX_IMAGE_BYTES = 8 * 1024 * 1024 # límite de Instagram por imagen
RATIO_TOL = 0.005
FEED_RATIO = (0.8, 1.91) # ancho/alto admitido en el feed (4:5 a 1,91:1)
STORY_RATIO = (0.5, 0.6) # historias: 9:16 (0,5625)
# reglas por tipo: nº de imágenes (mín, máx), rango de proporción y si lleva texto de publicación
KIND_RULES = {
    "carrusel": {"count": (2, MAX_SLIDES), "ratio": FEED_RATIO, "caption": True},
    "publicacion": {"count": (1, 1), "ratio": FEED_RATIO, "caption": True},
    "historia": {"count": (1, 1), "ratio": STORY_RATIO, "caption": False},
}
TOKEN_REFRESH_DAYS = 20
STUCK_MINUTES = 10
TICK_STALE_HOURS = 2 # sin avisos del programador durante más de esto -> se avisa en el panel

def utcnow() -> dt.datetime:
    return dt.datetime.now(dt.timezone.utc)

app = Flask(__name__)
app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1, x_host=1)
app.config.update(
    SECRET_KEY=os.environ.get("SECRET_KEY") or secrets.token_hex(32),
    MAX_CONTENT_LENGTH=60 * 1024 * 1024,
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE="Lax",
    SESSION_COOKIE_SECURE=not DEV,
    PERMANENT_SESSION_LIFETIME=dt.timedelta(days=30),
)
db.init_db()
```

9.2 Seguridad

Toda la protección transversal está junta.

- `safe_eq` compara textos en **tiempo constante** (`hmac.compare_digest`). Una comparación normal termina antes cuanto antes difiere, y esa diferencia de tiempo permitiría adivinar una clave carácter a carácter.
- **CSRF**. Un ataque CSRF engaña a tu navegador para que envíe un formulario a la app usando tu sesión. La defensa: cada sesión tiene un token aleatorio (`csrf_token()`), que las plantillas incluyen como campo oculto en **todo formulario POST**, y `check_csrf()` lo compara con el de la sesión. Si no coincide, responde 400.
- `current_user()`. Devuelve el usuario de la sesión y **lo comprueba contra la base de datos en cada petición** (con caché en `g`, el objeto de la petición). Si la sesión es de la cuenta integrada, devuelve un usuario `admin` (siempre que `ADMIN_PASSWORD` exista). Si la cuenta se eliminó, devuelve `None`. Acepta una conexión ya abierta para no abrir otra.
- `login_required` y `admin_required`. Decoradores. El primero redirige a `/login?next=...` si no hay usuario; el segundo, además, responde 403 si el rol no es administrador.
- `api_key_required`. Exige la cabecera `Authorization: Bearer <API_KEY>`. Si `API_KEY` no está definida, **nadie** entra.
- **Cabeceras de seguridad** (`after_request`). Entre ellas, una **Content-Security-Policy** muy restrictiva: `default-src 'none'`, solo imágenes y hojas de estilo propias, formularios solo hacia la propia app y ningún *frame*. Como consecuencia, **la app no tiene nada de JavaScript**: todo es HTML y CSS. Además `Cache-Control: no-store` (salvo estáticos e imágenes) para que las páginas privadas no queden en cachés.
- **Límite de intentos de login**. `attempts` guarda, por IP, la hora de cada intento fallido; con 5 fallos en 15 minutos se bloquea esa IP. Vive en memoria: por eso hay un solo *worker*.

app/server.py – seguridad y acceso (líneas 71-174)

```
# ----- seguridad -----

def safe_eq(a: str, b: str) -> bool:
    """Comparación en tiempo constante (admite texto no ASCII)."""
    return hmac.compare_digest(a.encode(), b.encode())

def csrf_token() -> str:
    if "csrf" not in session:
        session["csrf"] = secrets.token_urlsafe(24)
    return session["csrf"]

def check_csrf():
    sent = request.form.get("csrf", "")
    if not sent or not safe_eq(sent, session.get("csrf", "")):
        abort(400, "Formulario caducado. Recarga la página e inténtalo de nuevo.")

app.jinja_env.globals["csrf_token"] = csrf_token

@app.template_filter("fmt")
def fmt_date(value):
    return f"{value[:10]} {value[11:16]} UTC" if value else ""

def current_user(conn=None):
    """Usuario de la sesión, comprobado en cada petición: si se borra la cuenta, pierde el acceso
    al momento. None si no ha iniciado sesión."""
    if "user" not in g:
        uid, user = session.get("uid"), None
        if uid == accounts.ENV_ID:
            user = accounts.env_user() if ADMIN_PASSWORD else None
        elif uid:
            if conn is None:
                with db.connect() as own:
                    user = accounts.get(own, uid)
            else:
                user = accounts.get(conn, uid)
```

... continuación

```

        g.user = user
    return g.user

app.jinja_env.globals["current_user"] = current_user

@app.context_processor
def inject_user():
    return {"user": current_user()}

def login_required(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        if not current_user():
            session.pop("uid", None)
            return redirect(url_for("login", next=request.path))
        return fn(*args, **kwargs)
    return wrapper

def admin_required(fn):
    @wraps(fn)
    @login_required
    def wrapper(*args, **kwargs):
        if current_user()["role"] != "admin":
            abort(403, "Esta sección es solo para administradores.")
        return fn(*args, **kwargs)
    return wrapper

def api_key_required(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        header = request.headers.get("Authorization", "")
        sent = header[7:] if header.startswith("Bearer ") else ""
        if not API_KEY or not sent or not safe_eq(sent, API_KEY):
            return jsonify(error="no autorizado"), 401
        return fn(*args, **kwargs)
    return wrapper

@app.after_request
def security_headers(resp):
    resp.headers["X-Content-Type-Options"] = "nosniff"
    resp.headers["X-Frame-Options"] = "DENY"
    resp.headers["Referrer-Policy"] = "no-referrer"
    resp.headers["Content-Security-Policy"] = (
        "default-src 'none'; img-src 'self' data:; style-src 'self'; "
        "form-action 'self'; frame-ancestors 'none'; base-uri 'none'"
    )
    if request.endpoint not in ("static", "media"):
        resp.headers["Cache-Control"] = "no-store"
    return resp

_attempts: dict[str, list[float]] = {}

def _too_many_attempts(ip: str) -> bool:
    now = time.time()
    recent = [t for t in _attempts.get(ip, []) if now - t < 900]
    _attempts[ip] = recent
    return len(recent) >= 5

```

9.3 Utilidades

Funciones pequeñas que usan varias rutas:

- `build_caption(caption, hashtags)` — une el texto y los hashtags (con `#`) en el texto final de Instagram.
- `post_view(row)` — convierte los campos JSON de un post en listas y calcula `caption_full` (vacío en las historias).
- `jpeg_size(data)` — lee el **ancho y el alto directamente de la cabecera del JPEG**, recorriendo sus **marcadores** hasta el de inicio de trama (SOF). Así se valida la proporción **sin depender de Pillow**

en el servidor.

- `is_stuck(post)` — un post lleva más de 10 minutos en «publicando»: probablemente el proceso se interrumpió.
- `KIND y STATUS` — diccionarios de etiquetas en español, expuestos a las plantillas.
- `redact(text, hidden)` — sustituye por `***` cualquier valor secreto (token, ID) dentro de un texto. **Es la barrera que evita que un error de la API con el token dentro llegue a la base de datos o al panel.**

app/server.py – utilidades (líneas 177-231)

```
# ----- utilidades -----

def build_caption(caption: str, hashtags: list[str]) -> str:
    tags = " ".join("#" + h for h in hashtags)
    return caption.strip() + ("\n\n" + tags if tags else "")

def post_view(row: dict) -> dict:
    row["hashtags"] = json.loads(row["hashtags"])
    row["slides_text"] = json.loads(row["slides_text"])
    # las historias no llevan texto de publicación
    row["caption_full"] = "" if row.get("kind") == "historia" else build_caption(row["caption"], row["hashtags"])
    return row

def jpeg_size(data: bytes):
    """(ancho, alto) de un JPEG leyendo su cabecera, o None si no se puede."""
    i = 2
    while i + 9 < len(data):
        if data[i] != 0xFF:
            i += 1
            continue
        marker = data[i + 1]
        if marker == 0xFF:
            i += 1
            continue
        if marker in (0x01, 0xD8) or 0xD0 <= marker <= 0xD7:
            i += 2
            continue
        if marker in (0xC0, 0xC1, 0xC2, 0xC3, 0xC5, 0xC6, 0xC7, 0xC9, 0xCA, 0xCB, 0xCD, 0xCE, 0xCF):
            return int.from_bytes(data[i + 7:i + 9], "big"), int.from_bytes(data[i + 5:i + 7], "big")
        i += 2 + int.from_bytes(data[i + 2:i + 4], "big")
    return None

def is_stuck(post: dict) -> bool:
    if post["status"] != "publishing":
        return False
    updated = dt.datetime.fromisoformat(post["updated_at"])
    return dt.datetime.now(dt.timezone.utc) - updated > dt.timedelta(minutes=STUCK_MINUTES)

app.jinja_env.globals["is_stuck"] = is_stuck
app.jinja_env.globals["KIND"] = {"carrusel": "Carrusel", "publicacion": "Publicación", "historia": "Historia"}
app.jinja_env.globals["STATUS"] = {
    "draft": "Por aprobar", "approved": "En cola", "publishing": "Publicando...",
    "failed": "Falló", "published": "Publicado", "rejected": "Rechazado",
}

def redact(text: str, hidden: list[str]) -> str:
    for value in hidden:
        if value:
            text = text.replace(value, "***")
    return text
```

9.4 El token de Instagram

La app tiene que usar siempre **el token vigente**: el renovado, no el inicial.

- `get_token(conn)` — devuelve el token guardado en la base de datos, **salvo que hayas cambiado la variable de entorno**. Para detectar el cambio guarda una **huella** (`_fp`, los primeros 12 caracteres de un SHA-256) de la variable con la que se renovó por última vez; si la huella actual difiere, manda el valor de la variable. Así puedes pegar un token nuevo en Render sin tocar la base de datos.

- `maybe_refresh_token(conn)` — renueva si han pasado 20 días desde la última renovación. Tiene un segundo freno: no reintenta más de una vez cada ~6 horas (`ig_token_tried_at`), para no martillear la API si la renovación falla. **Nunca lanza excepciones hacia fuera**: si falla, se reintenta en el siguiente inicio de sesión o publicación.

app/server.py – el token de Instagram (líneas 234-267)

```
# ----- token ----

def _fp(token: str) -> str:
    return hashlib.sha256(token.encode()).hexdigest()[:12]

def get_token(conn) -> str:
    """Token vigente: el renovado guardado en la base de datos, salvo que se haya
    cambiado el de la variable de entorno (entonces manda el nuevo)."""
    env_token = os.environ.get("IG_ACCESS_TOKEN", "")
    stored = db.kv_get(conn, "ig_token")
    if stored and db.kv_get(conn, "ig_token_env_fp") == _fp(env_token):
        return stored
    return env_token

def maybe_refresh_token(conn) -> None:
    """Renueva el token si hace más de TOKEN_REFRESH_DAYS. Nunca falla hacia fuera."""
    try:
        now = dt.datetime.now(dt.timezone.utc)
        for key, days in (("ig_token_refreshed_at", TOKEN_REFRESH_DAYS), ("ig_token_tried_at", 0.25)):
            last = db.kv_get(conn, key)
            if last and now - dt.datetime.fromisoformat(last) < dt.timedelta(days=days):
                return
        token = get_token(conn)
        if not token:
            return
        db.kv_set(conn, "ig_token_tried_at", db.now())
        new_token, _ = instagram.refresh(token)
        db.kv_set(conn, "ig_token", new_token)
        db.kv_set(conn, "ig_token_env_fp", _fp(os.environ.get("IG_ACCESS_TOKEN", "")))
        db.kv_set(conn, "ig_token_refreshed_at", db.now())
    except Exception:
        pass # se reintenta en el próximo inicio de sesión o publicación
```

9.5 Acceso: healthz, login y logout

- `/healthz` responde `ok` sin tocar la base de datos. Render lo consulta para saber que el servicio está sano.
- `/login` (GET muestra el formulario; POST lo procesa):
 1. Si falta `ADMIN_PASSWORD`, responde 503: la app no arranca «abierta» por descuido.
 2. Comprueba el CSRF y el límite de intentos por IP.
 3. Llama a `accounts.authenticate`. Si acierta, `session.clear()` (descarta cualquier sesión previa, para evitar la fijación de sesión) y guarda solo `uid`.
 4. Aprovecha el inicio de sesión para intentar renovar el token.
 5. Redirige a `next` **solo si es una ruta interna** (empieza por `/` y no por `//`), para que nadie pueda usar la página como trampolín hacia otro sitio.
 6. Si falla, cuenta el intento y muestra el mismo mensaje genérico («Usuario o contraseña incorrectos»).
- `/logout` (POST, con CSRF) limpia la sesión.

app/server.py – acceso (líneas 270-308)

```
# ----- acceso ---

@app.get("/healthz")
def healthz():
    return "ok"

@app.route("/login", methods=["GET", "POST"])
def login():
    if not ADMIN_PASSWORD:
```

... continuación

```

    return "Servidor sin ADMIN_PASSWORD configurada.", 503
if request.method == "POST":
    check_csrf()
    ip = request.remote_addr or "?"
    user = None
    if _too_many_attempts(ip):
        flash("Demasiados intentos. Espera 15 minutos.", "error")
    else:
        with db.connect() as conn:
            user = accounts.authenticate(conn, request.form.get("username", ""),
                                       request.form.get("password", ""), ADMIN_PASSWORD, safe_eq)

            if user:
                maybe_refresh_token(conn)
        if user:
            session.clear()
            session["uid"] = user["id"]
            session.permanent = True
            target = request.args.get("next", "")
            return redirect(target if target.startswith("/") and not target.startswith("//") else
                           url_for("index"))
        _attempts.setdefault(ip, []).append(time.time())
        flash("Usuario o contraseña incorrectos.", "error")
    return render_template("login.html")

@app.post("/logout")
def logout():
    check_csrf()
    session.clear()
    return redirect(url_for("login"))

```

9.6 El panel: lista y detalle

- `_queue_context(conn)` prepara lo que el panel muestra sobre la cola: para cada tipo con posts aprobados calcula la **hora prevista** de cada uno (`cfg.upcoming` a partir del último hueco usado), detecta los **tipos aprobados sin ningún hueco** (`no_slot`), detecta si el **programador lleva más de 2 horas sin avisar** (`tick_stale`) y captura un error de programación (`schedule_error`) para mostrarlo en vez de romper la página.
- `index` — los 100 posts más recientes con su número de imágenes (una subconsulta), agrupados por estado; la cola ordenada por fecha de aprobación.
- `_load_post` — carga un post o responde 404.
- `post_page` — el detalle: imágenes, texto, hashtags, los posibles incumplimientos de la regla de contenido y las acciones según el estado. Si el post está publicándose (y no atascado), la plantilla añade un refresco automático.
- `media` — sirve una imagen JPEG. **Regla de acceso:** si el post está en «publicando», la imagen es pública (Instagram tiene que poder descargarla); en cualquier otro estado solo la ve quien ha iniciado sesión; y si no existe, 404. Las cabeceras impiden que se cachee.

POR QUÉ Las imágenes solo son públicas durante unos segundos (lo que dura la publicación) y sus URLs contienen un UUID imposible de adivinar. Fuera de esa ventana, quien no tiene sesión recibe 404, como si no existieran.

app/server.py – panel, detalle y servir imágenes (líneas 311-385)

```

# ----- panel -----

def _queue_context(conn) -> dict:
    """Programación, hora prevista de cada post aprobado y salud del programador."""
    approved = db.query(conn, "SELECT id, kind FROM posts WHERE status = 'approved' ORDER BY approved_at ASC")
    last_tick = db.kv_get(conn, "last_tick_at")
    ctx = {"eta": {}, "schedule_text": "", "schedule_error": None, "last_tick": last_tick, "no_slot": [],
          "tick_stale": (not last_tick) or utcnow() - dt.datetime.fromisoformat(last_tick)
          > dt.timedelta(hours=TICK_STALE_HOURS)}
    try:
        cfg = schedule_store.load(conn)
        for kind in KINDS:
            ids = [r["id"] for r in approved if r["kind"] == kind]

```

... continuación

```

        if not ids:
            continue
        if kind not in cfg.kinds():
            ctx["no_slot"].append(kind)
            continue
        last = db.kv_get(conn, f"last_slot:{kind}")
        used = dt.datetime.fromisoformat(last) if last else None
        for pid, slot in zip(ids, cfg.upcoming(utcnow(), used, len(ids), kind)):
            ctx["eta"][pid] = cfg.label(slot)
        ctx["schedule_text"] = f"{cfg.describe()} · hora de {cfg.tz_name}"
    except schedule.ScheduleError as exc:
        ctx["schedule_error"] = str(exc)
    return ctx

@app.get("/")
@login_required
def index():
    with db.connect() as conn:
        rows = db.query(conn, (
            "SELECT p.id, p.title, p.status, p.kind, p.created_at, p.approved_at, p.published_at, p.permalink, "
            "p.updated_at, "
            "(SELECT COUNT(*) FROM images i WHERE i.post_id = p.id) AS n_images "
            "FROM posts p ORDER BY p.created_at DESC LIMIT 100"))
        refreshed = db.kv_get(conn, "ig_token_refreshed_at")
        queue = _queue_context(conn)
        groups = {s: [r for r in rows if r["status"] == s]
                   for s in ("draft", "approved", "publishing", "failed", "published", "rejected")}
        groups["approved"].sort(key=lambda r: r["approved_at"] or "")
    return render_template("index.html", groups=groups, refreshed=refreshed, queue=queue)

def _load_post(conn, pid: str) -> dict:
    row = db.one(conn, "SELECT * FROM posts WHERE id = %s", (pid,))
    if not row:
        abort(404)
    return post_view(row)

@app.get("/post/<pid>")
@login_required
def post_page(pid):
    with db.connect() as conn:
        post = _load_post(conn, pid)
        n_images = db.one(conn, "SELECT COUNT(*) AS n FROM images WHERE post_id = %s", (pid,))["n"]
        queue = _queue_context(conn)
        violations = guards.check(post["caption_full"], *post["slides_text"])
    return render_template("post.html", post=post, n_images=n_images, violations=violations, queue=queue,
                          refresh=post["status"] == "publishing" and not is_stuck(post))

@app.get("/media/<pid>/<int:n>.jpg")
def media(pid, n):
    """Imagen de un borrador. Pública SOLO mientras se publica (Instagram las descarga);
    fuera de esa ventana solo la ve quien ha iniciado sesión."""
    with db.connect() as conn:
        row = db.one(conn, (
            "SELECT p.status, i.data FROM images i JOIN posts p ON p.id = i.post_id "
            "WHERE i.post_id = %s AND i.position = %s", (pid, n))
        logged_in = bool(row) and row["status"] != "publishing" and bool(current_user(conn))
    if not row or (row["status"] != "publishing" and not logged_in):
        abort(404)
    return Response(bytes(row["data"]), mimetype="image/jpeg", headers={"Cache-Control": "private, no-store"})

```

9.7 Aprobar y sacar de la cola

- `_set_status(conn, pid, allowed_from, status, error)` — la primitiva de cambio de estado: `UPDATE ... WHERE id = ... AND status IN (...) RETURNING id`. Devuelve `True` solo si cambió algo. Como comprueba el estado anterior **en la misma sentencia**, dos peticiones simultáneas no pueden pisarse.
- `approve` — antes de encolar comprueba, en este orden: la **casilla de confirmación**; que el contenido **cumpla la regla de CTA**; que el post **tenga imágenes**; y que el servidor tenga **IG_USER_ID y un token**. Después hace `UPDATE ... WHERE status = 'draft'` guardando la fecha con microsegundos (`approved_at`) y **quién aprobó** (`approved_by`). Si ya no estaba en borrador (alguien lo aprobó antes), avisa. **Aprobar no publica**: solo encola.

- `unqueue` — devuelve un post de la cola a borrador (y borra `approved_at` y `approved_by`).

app/server.py – estados, aprobar y sacar de la cola (líneas 388-438)

```
def _set_status(conn, pid: str, allowed_from: tuple[str, ...], status: str, error=None) -> bool:
    marks = ", ".join("%s" for _ in allowed_from)
    rows = db.query(conn, (
        f"UPDATE posts SET status = %s, updated_at = %s, error = %s "
        f"WHERE id = %s AND status IN ({marks}) RETURNING id",
        (status, db.now(), error, pid, *allowed_from)
    ))
    return bool(rows)

@app.post("/post/<pid>/approve")
@login_required
def approve(pid):
    """Autoriza el post: pasa a la cola. Se publica en el próximo hueco libre de schedule.txt."""
    check_csrf()
    if request.form.get("confirm") != "on":
        flash("Marca la casilla de confirmación para autorizar la publicación.", "error")
        return redirect(url_for("post_page", pid=pid))
    with db.connect() as conn:
        post = _load_post(conn, pid)
        n_images = db.one(conn, "SELECT COUNT(*) AS n FROM images WHERE post_id = %s", (pid,))["n"]
        if guards.check(post["caption_full"], *post["slides_text"]):
            flash("El contenido incumple la restricción de CTA. No se puede publicar.", "error")
            return redirect(url_for("post_page", pid=pid))
        if not n_images:
            flash("Este borrador no tiene imágenes.", "error")
            return redirect(url_for("post_page", pid=pid))
        if not (os.environ.get("IG_USER_ID") and get_token(conn)):
            flash("Faltan IG_USER_ID / IG_ACCESS_TOKEN en el servidor.", "error")
            return redirect(url_for("post_page", pid=pid))
        rows = db.query(conn, (
            "UPDATE posts SET status = 'approved', approved_at = %s, approved_by = %s, updated_at = %s "
            "WHERE id = %s AND status = 'draft' RETURNING id",
            (db.now_precise(), current_user()["username"], db.now(), pid)
        ))
        if not rows:
            flash("Este post ya no está en borrador.", "error")
        else:
            flash("Autorizado. Está en la cola y se publicará en el próximo hueco libre.", "ok")
        return redirect(url_for("post_page", pid=pid))

@app.post("/post/<pid>/unqueue")
@login_required
def unqueue(pid):
    """Saca un post de la cola (vuelve a borrador) antes de que llegue su turno."""
    check_csrf()
    with db.connect() as conn:
        rows = db.query(conn, (
            "UPDATE posts SET status = 'draft', approved_at = NULL, approved_by = NULL, updated_at = %s "
            "WHERE id = %s AND status = 'approved' RETURNING id",
            (db.now(), pid)
        ))
        flash("Sacado de la cola." if rows else "Este post ya no está en la cola.", "ok" if rows else "error")
        return redirect(url_for("post_page", pid=pid))
```

9.8 El aviso del programador y la publicación

Es la parte más delicada. Tres funciones pequeñas y una ruta.

- `_start_publish(pids, base_url)` — lanza **un hilo** que publica los posts indicados uno tras otro. La ruta responde de inmediato y el hilo trabaja en segundo plano (publicar tarda decenas de segundos por la espera a Instagram). El hilo **no es daemon**: el proceso no termina hasta que acabe.
- `_claim_slot(conn, kind, slot_iso)` — «reclama» el hueco de un tipo. Guarda en `kv` la clave `last_slot:<tipo>` con la hora del hueco, mediante `UPDATE ... WHERE value < nuevo RETURNING`. Solo **un** aviso consigue subir el valor; los demás (por ejemplo, un reintento a los 30 minutos, todavía dentro de la tolerancia) ven que el hueco ya está usado. Es lo que garantiza **un post por hueco**.
- `_take_oldest(conn, kind)` — pasa a «publicando» el aprobado **más antiguo** del tipo, con una única sentencia (subconsulta con `ORDER BY approved_at LIMIT 1` dentro del `UPDATE ... RETURNING`). Sin carreras.
- `api_tick (POST /api/cron/tick)` — el flujo completo:
 1. Anota la hora del último aviso (`last_tick_at`): el panel avisa si pasan más de 2 horas sin él.

2. En modo normal, pregunta a la programación qué huecos están vigentes (`due_slots`). Si no hay ninguno, responde `idle`.
3. Para cada hueco vigente, intenta reclamarlo. Los ya usados se listan como `slot-used`.
4. Para cada hueco reclamado, toma el post más antiguo de ese tipo. **Si no hay ninguno, el hueco queda gastado igualmente** (se pierde) y la respuesta es `empty-queue`.
5. Si hay posts, arranca el hilo y responde `publishing` con los identificadores.
6. El modo `?force=1[&kind=...]` publica ya el más antiguo **sin gastar hueco**: es solo para pruebas manuales del usuario. **Claude no lo usa nunca**.
- `_publish_worker(pid, base_url)` — el trabajo real:
 1. Intenta renovar el token si toca y carga el post y sus imágenes.
 2. **Vuelve a comprobar** la regla de contenido y que haya imágenes.
 3. Construye las URLs públicas `.../media/<id>/<n>.jpg` (usando `PUBLIC_BASE_URL` o la URL de la petición) y llama a `instagram.publish`.
 4. **Cualquier excepción** deja el post en `failed`, con el mensaje **redactado** (sin token ni ID) y **conservando las imágenes**, para que puedas revisar y reabrir.
 5. Si va bien: pasa a `published` con `media_id`, `permalink` y fecha, y **borra las imágenes** (`DELETE FROM images`).

CUIDADO Un post en `failed` **no se reintentará a ciegas**: puede haberse publicado ya (por ejemplo, si el fallo ocurrió al pedir el enlace o al confirmar). Antes de reabrirlo, **mira el perfil de Instagram**. Este criterio nació de un fallo real: una publicación falló por un ID mal copiado y, al corregirlo, la primera pregunta fue si la anterior había llegado a salir.

app/server.py – cola, programador y publicación (líneas 441-511)

```
def _start_publish(pids: list[str], base_url: str) -> None:
    """Publica en orden, en un solo hilo, los posts indicados."""
    def run():
        for pid in pids:
            _publish_worker(pid, base_url)
    threading.Thread(target=run, daemon=False).start()

def _claim_slot(conn, kind: str, slot_iso: str) -> bool:
    """Marca el hueco de ese tipo como usado. Atómico: solo un aviso del programador lo consigue."""
    key = f"last_slot:{kind}"
    db.execute(conn, ("INSERT INTO kv (key, value, updated_at) VALUES (%s, '', %s) "
        "ON CONFLICT (key) DO NOTHING"), (key, db.now()))
    return bool(db.query(conn, (
        "UPDATE kv SET value = %s, updated_at = %s WHERE key = %s AND value < %s RETURNING key"),
        (slot_iso, db.now(), key, slot_iso)))

def _take_oldest(conn, kind):
    """Pasa a «publicando» el post aprobado más antiguo (del tipo dado, o de cualquiera)."""
    where, params = ("AND kind = %s ", (kind,)) if kind else ("", ())
    rows = db.query(conn, (
        "UPDATE posts SET status = 'publishing', updated_at = %s, error = NULL "
        f"WHERE id = (SELECT id FROM posts WHERE status = 'approved' {where}ORDER BY approved_at ASC LIMIT 1) "
        "AND status = 'approved' RETURNING id, kind"), (db.now(), *params))
    return rows[0] if rows else None

@app.post("/api/cron/tick")
@api_key_required
def api_tick():
    """Aviso del programador (GitHub Actions, cada ~30 min). Por cada tipo con un hueco de
    schedule.txt vigente y sin usar, publica el post más antiguo de la cola de ese tipo (o deja
    el hueco vacío). `?force=1[&kind=historia]` publica ya el más antiguo, sin gastar hueco:
    solo para pruebas manuales."""
    force = request.args.get("force") == "1"
    only_kind = request.args.get("kind") or None
    if only_kind and only_kind not in KINDS:
        return _bad(f"kind debe ser uno de: {', '.join(KINDS)}", 400)
    targets: list[tuple[str | None, str | None]] = [] # (tipo, hueco)
    used_slots: list[str] = []
    with db.connect() as conn:
        db.kv_set(conn, "last_tick_at", utcnow().isoformat(timespec="seconds"))
```

... continuación

```

if force:
    targets = [(only_kind, None)]
else:
    try:
        due = schedule_store.load(conn).due_slots(utcnow())
    except schedule.ScheduleError as exc:
        return jsonify(error=str(exc)), 500
    if not due:
        return jsonify(action="idle")
    for kind, slot in due.items():
        slot_iso = slot.astimezone(dt.timezone.utc).isoformat(timespec="seconds")
        if _claim_slot(conn, kind, slot_iso):
            targets.append((kind, slot_iso))
        else:
            used_slots.append(f"{kind}@{slot_iso}")
    if not targets:
        return jsonify(action="slot-used", slots=used_slots)
    picked = []
    for kind, slot_iso in targets:
        row = _take_oldest(conn, kind)
        if row:
            picked.append({"post_id": row["id"], "kind": row["kind"], "slot": slot_iso})
    if not picked:
        return jsonify(action="empty-queue", slots=[t[1] for t in targets])
    base = os.environ.get("PUBLIC_BASE_URL", "").rstrip("/") or request.url_root.rstrip("/")
    _start_publish([p["post_id"] for p in picked], base)
    return jsonify(action="publishing", post_id=picked[0]["post_id"],
                  post_ids=[p["post_id"] for p in picked], posts=picked)

```

app/server.py – el hilo que publica (líneas 514-539)

```

def _publish_worker(pid: str, base_url: str) -> None:
    hidden: list[str] = []
    try:
        with db.connect() as conn:
            maybe_refresh_token(conn)
            post = _load_post(conn, pid)
            n = db.one(conn, "SELECT COUNT(*) AS n FROM images WHERE post_id = %s", (pid,))["n"]
            token = get_token(conn)
            if guards.check(post["caption_full"], *post["slides_text"]):
                raise ValueError("el contenido incumple la restricción de CTA")
            if not n:
                raise ValueError("el post no tiene imágenes")
            user_id = os.environ.get("IG_USER_ID", "")
            hidden = [token, user_id]
            urls = [f"{base_url}/media/{pid}/{i}.jpg" for i in range(n)]
            result = instagram.publish(urls, post["caption_full"], user_id, token, kind=post["kind"])
    except Exception as exc: # noqa: BLE001 - cualquier fallo deja el post en "failed"
        with db.connect() as conn:
            _set_status(conn, pid, ("publishing",), "failed", redact(str(exc), hidden)[:500])
        return
    with db.connect() as conn:
        db.execute(conn, (
            "UPDATE posts SET status = 'published', updated_at = %s, published_at = %s, "
            "media_id = %s, permalink = %s, error = NULL WHERE id = %s",
            (db.now(), db.now(), result["media_id"], result["permalink"], pid))
        db.execute(conn, "DELETE FROM images WHERE post_id = %s", (pid,)) # no se guarda nada

```

9.9 Rechazar, reabrir, marcar como fallido y eliminar

- **reject** — desde borrador, cola o fallido pasa a **rejected** y **borra las imágenes**.
- **reopen** — un post **failed** vuelve a borrador (conserva las imágenes) para revisarlo de nuevo.
- **unstuck** — si un post lleva más de 10 minutos en «publicando», permite marcarlo como fallido con un mensaje que recuerda comprobar Instagram.
- **delete** — elimina el registro (y, por el **CASCADE**, sus imágenes), salvo que se esté publicando.

app/server.py – rechazar, reabrir y eliminar (líneas 542-587)

```

@app.post("/post/<pid>/reject")
@login_required
def reject(pid):
    check_csrf()
    with db.connect() as conn:
        if _set_status(conn, pid, ("draft", "approved", "failed"), "rejected"):
            db.execute(conn, "DELETE FROM images WHERE post_id = %s", (pid,))

```

... continuación

```

        flash("Post rechazado. Las imágenes se han borrado.", "ok")
    else:
        flash("Este post no se puede rechazar en su estado actual.", "error")
    return redirect(url_for("index"))

@app.post("/post/<pid>/reopen")
@login_required
def reopen(pid):
    """Un post fallido vuelve a borrador (conserva las imágenes) para revisarlo de nuevo."""
    check_csrf()
    with db.connect() as conn:
        if not _set_status(conn, pid, ("failed",), "draft"):
            flash("Solo se pueden reabrir posts fallidos.", "error")
    return redirect(url_for("post_page", pid=pid))

@app.post("/post/<pid>/unstuck")
@login_required
def unstick(pid):
    check_csrf()
    with db.connect() as conn:
        post = _load_post(conn, pid)
        if is_stuck(post):
            _set_status(conn, pid, ("publishing",), "failed",
                        "Interrumpido. Comprueba en Instagram si llegó a publicarse antes de reintentar.")
        else:
            flash("El post sigue publicándose; espera un poco.", "error")
    return redirect(url_for("post_page", pid=pid))

@app.post("/post/<pid>/delete")
@login_required
def delete(pid):
    check_csrf()
    with db.connect() as conn:
        rows = db.query(conn, "DELETE FROM posts WHERE id = %s AND status <> 'publishing' RETURNING id", (pid,))
    flash("Post eliminado." if rows else "No se puede eliminar un post que se está publicando.", "ok" if rows
        else "error")
    return redirect(url_for("index"))

```

9.10 Cuenta, usuarios y programación

Rutas del panel de administración. Todas comprueban CSRF.

- `/account` — el usuario cambia su propia contraseña: pide la actual, la nueva y su repetición. La cuenta integrada `admin` no puede (su contraseña es una variable de entorno) y la plantilla se lo explica.
- `/admin/users` — lista, alta y baja. **No puedes eliminar tu propia cuenta.** Al crear, se muestra un aviso para que pases la contraseña «por un canal privado».
- `/admin/schedule` — muestra los huecos y los ajustes; añadir un hueco, eliminarlo y guardar zona horaria y tolerancia. Los errores de validación se muestran como avisos, no como páginas de error.

app/server.py – cuenta, usuarios y programación (líneas 590-704)

```

# ----- cuenta y usuarios ---

@app.route("/account", methods=["GET", "POST"])
@login_required
def account():
    """Cada usuario cambia su propia contraseña (la de «admin» es la variable ADMIN_PASSWORD de Render)."""
    user = current_user()
    if request.method == "POST" and not user.get("builtin"):
        check_csrf()
        new, again = request.form.get("new_password", ""), request.form.get("again", "")
        with db.connect() as conn:
            if not accounts.verify_password(conn, user["id"], request.form.get("current_password", "")):
                flash("La contraseña actual no es correcta.", "error")
            elif new != again:
                flash("Las contraseñas nuevas no coinciden.", "error")
            else:
                try:
                    accounts.set_password(conn, user["id"], new)
                    flash("Contraseña cambiada.", "ok")

```

... continuación

```

        except accounts.AccountError as exc:
            flash(str(exc), "error")
        return redirect(url_for("account"))
    return render_template("account.html", roles=accounts.ROLES, min_password=accounts.MIN_PASSWORD)

@app.get("/admin/users")
@admin_required
def users_page():
    with db.connect() as conn:
        users = accounts.list_all(conn)
    return render_template("users.html", users=users, roles=accounts.ROLES, min_password=accounts.MIN_PASSWORD)

@app.post("/admin/users")
@admin_required
def users_create():
    check_csrf()
    with db.connect() as conn:
        try:
            created = accounts.create(conn, request.form.get("username", ""), request.form.get("password", ""),
                                     request.form.get("role", ""), current_user()["username"])
            flash(f"Usuario «{created['username']}» creado. Pásale su contraseña por un canal privado.", "ok")
        except accounts.AccountError as exc:
            flash(str(exc), "error")
    return redirect(url_for("users_page"))

@app.post("/admin/users/<uid>/delete")
@admin_required
def users_delete(uid):
    check_csrf()
    if uid == current_user()["id"]:
        flash("No puedes eliminar tu propia cuenta.", "error")
    else:
        with db.connect() as conn:
            gone = accounts.get(conn, uid)
            if gone and accounts.delete(conn, uid):
                flash(f"Usuario «{gone['username']}» eliminado: ya no puede entrar.", "ok")
            else:
                flash("Ese usuario ya no existe.", "error")
    return redirect(url_for("users_page"))

# ----- programación -----

@app.get("/admin/schedule")
@admin_required
def schedule_page():
    with db.connect() as conn:
        error, slots, cfg = None, [], None
        try:
            cfg = schedule_store.load(conn)
            slots = schedule_store.list_slots(conn)
        except schedule.ScheduleError as exc:
            error = str(exc)
    return render_template("schedule.html", slots=slots, cfg=cfg, error=error, days=schedule.DAY_NAMES,
                          kinds=[(k, schedule.KIND_LABELS[k].capitalize()) for k in KINDS],
                          tolerance_range=(schedule.MIN_TOLERANCE, schedule.MAX_TOLERANCE))

@app.post("/admin/schedule/slots")
@admin_required
def schedule_add():
    check_csrf()
    with db.connect() as conn:
        try:
            schedule_store.add_slot(conn, request.form.get("day", ""), request.form.get("time", ""),
                                   request.form.get("kind", ""))
            flash("Hueco añadido. Solo cuenta a partir de ahora: no publica una hora que ya pasó hoy.", "ok")
        except schedule.ScheduleError as exc:
            flash(str(exc), "error")
    return redirect(url_for("schedule_page"))

@app.post("/admin/schedule/slots/<sid>/delete")
@admin_required
def schedule_delete(sid):
    check_csrf()
    with db.connect() as conn:
        removed = schedule_store.delete_slot(conn, sid)
    flash("Hueco eliminado." if removed else "Ese hueco ya no existe.", "ok" if removed else "error")

```

... continuación

```

    return redirect(url_for("schedule_page"))

@app.post("/admin/schedule/settings")
@admin_required
def schedule_settings():
    check_csrf()
    with db.connect() as conn:
        try:
            schedule_store.save_settings(conn, request.form.get("tz", ""), request.form.get("tolerance", ""))
            flash("Ajustes guardados.", "ok")
        except schedule.ScheduleError as exc:
            flash(str(exc), "error")
    return redirect(url_for("schedule_page"))

```

9.11 La API de borradores

`POST /api/drafts` es la puerta por la que entra el contenido. Recibe un formulario `multipart` con `title`, `caption`, `hashtags` (JSON), `slides_text` (JSON), `source_url`, `kind` y una o varias imágenes en el campo `slides`. **Valida todo antes de guardar nada**, en este orden, y cada fallo responde 422 con un mensaje claro:

1. `kind` debe ser uno de los tres tipos.
2. `hashtags` y `slides_text` deben ser JSON válido y ser listas.
3. `title` obligatorio (máx. 120 caracteres); `caption` obligatorio salvo en historias.
4. Historias: se **ignoran** `caption` y `hashtags` (la API de Instagram no los admite).
5. Hashtags: solo caracteres de palabra, sin duplicados (sin distinguir mayúsculas), como máximo 30.
6. Texto final: máximo 2200 caracteres.
7. Número de imágenes según el tipo (`KIND_RULES`).
8. **Cada imagen**: debe empezar por la firma de un JPEG (`FF D8 FF`), no pasar de 8 MB, tener dimensiones legibles y una **proporción válida** para el tipo (con una pequeña tolerancia).
9. **Regla de contenido** (capítulo 6) sobre el texto y los slides.

Solo entonces genera un UUID, inserta el post y sus imágenes (si falla al guardar las imágenes, borra el post para no dejar restos) y responde 201 con el identificador y la URL del panel.

`GET /api/posts` devuelve los posts recientes **sin imágenes ni texto** (solo título, estado, tipo, URL y fechas): lo usa Claude para **no repetir temas**. `GET /api/posts/<id>` devuelve el estado de uno: lo usa el workflow del programador para esperar el resultado de una publicación.

app/server.py – API: subir borradores (líneas 707-789)

```

# ----- API -----

def _bad(message: str, code: int = 422):
    return jsonify(error=message), code

@app.post("/api/drafts")
@api_key_required
def api_create_draft():
    title = (request.form.get("title") or "").strip()
    caption = (request.form.get("caption") or "").strip()
    source_url = (request.form.get("source_url") or "").strip()[:300]
    kind = (request.form.get("kind") or "carrusel").strip()
    if kind not in KIND_RULES:
        return _bad(f"kind debe ser uno de: {' '.join(KINDS)}")
    rules = KIND_RULES[kind]
    try:
        hashtags = json.loads(request.form.get("hashtags") or "[]")
        slides_text = json.loads(request.form.get("slides_text") or "[]")
    except ValueError:
        return _bad("hashtags y slides_text deben ser JSON")
    if not isinstance(hashtags, list) or not isinstance(slides_text, list):
        return _bad("hashtags y slides_text deben ser listas")
    if not title or len(title) > 120:
        return _bad("title es obligatorio (máx. 120 caracteres)")

```

... continuación

```

if rules["caption"] and not caption:
    return _bad("caption es obligatorio")
if not rules["caption"]:
    caption, hashtags = "", [] # las historias no llevan texto de publicación

tags = []
for h in hashtags:
    h = str(h).strip().rstrip("#")
    if not re.fullmatch(r"\w+", h):
        return _bad(f"hashtag no válido: {h!r}")
    if h.lower() not in [t.lower() for t in tags]:
        tags.append(h)
if len(tags) > MAX_HASHTAGS:
    return _bad(f"demasiados hashtags ({len(tags)}); el máximo es {MAX_HASHTAGS}")
full = "" if kind == "historia" else build_caption(caption, tags)
if len(full) > MAX_CAPTION:
    return _bad(f"caption demasiado largo ({len(full)} > {MAX_CAPTION})")

files = request.files.getlist("slides")
lo, hi = rules["count"]
if not lo <= len(files) <= hi:
    return _bad(f"un {kind} lleva " + (f"{lo} imagen" if lo == hi == 1 else f"entre {lo} y {hi} imágenes")
                + f" (recibidas: {len(files)})")

images = []
for f in files:
    data = f.read()
    if not data.startswith(b"\xff\xd8\xff"):
        return _bad(f"{f.filename}: no es un JPEG (Instagram solo acepta JPEG)")
    if len(data) > MAX_IMAGE_BYTES:
        return _bad(f"{f.filename}: supera los 8 MB de Instagram")
    size = jpeg_size(data)
    if not size:
        return _bad(f"{f.filename}: no se pudo leer el tamaño de la imagen")
    ratio, (rmin, rmax) = size[0] / size[1], rules["ratio"]
    if not rmin - RATIO_TOL <= ratio <= rmax + RATIO_TOL:
        expected = "9:16 (1080x1920)" if kind == "historia" else "entre 4:5 y 1,91:1 (por ejemplo 1080x1350)"
        return _bad(f"{f.filename}: proporción {size[0]}x{size[1]} no válida para {kind}; debe ser
                    ↳ {expected}")
    images.append(data)

violations = guards.check(full, *[str(t) for t in slides_text])
if violations:
    return _bad("contenido no permitido (CTA con enlace): " + "; ".join(violations))

pid = uuid.uuid4().hex
stamp = db.now()
with db.connect() as conn:
    db.execute(conn, (
        "INSERT INTO posts (id, title, status, kind, caption, hashtags, slides_text, source_url, created_at,
        ↳ updated_at) "
        "VALUES (%s, %s, 'draft', %s, %s, %s, %s, %s, %s, %s)",
        (pid, title, kind, caption, json.dumps(tags, ensure_ascii=False),
        json.dumps([str(t) for t in slides_text], ensure_ascii=False), source_url or None, stamp, stamp))
    try:
        for i, data in enumerate(images):
            db.execute(conn, "INSERT INTO images (post_id, position, data) VALUES (%s, %s, %s)", (pid, i,
            ↳ data))
    except Exception:
        db.execute(conn, "DELETE FROM posts WHERE id = %s", (pid,))
        raise
return jsonify(id=pid, url=url_for("post_page", pid=pid, _external=True)), 201

```

app/server.py – API: listar y consultar (líneas 792-812)

```

@app.get("/api/posts")
@api_key_required
def api_list_posts():
    """Posts recientes (sin imágenes ni texto): sirve para no repetir temas al generar."""
    try:
        limit = max(1, min(int(request.args.get("limit", 30)), 100))
    except ValueError:
        return _bad("limit debe ser un número", 400)
    with db.connect() as conn:
        rows = db.query(conn, (
            "SELECT id, title, status, kind, source_url, created_at, published_at FROM posts "
            "ORDER BY created_at DESC LIMIT %s", (limit,))
        return jsonify(rows)

@app.get("/api/posts/<pid>")
@api_key_required

```

... continuación

```
def api_post_status(pid):  
    with db.connect() as conn:  
        row = db.one(conn, "SELECT id, status, kind, permalink, error, approved_at, published_at FROM posts  
        ↳ WHERE id = %s", (pid,))  
    return (jsonify(row), 200) if row else (jsonify(error="no existe"), 404)
```

10. La interfaz: plantillas y CSS

10.1 Principios

- **Sin JavaScript.** La política de seguridad de contenido lo prohíbe. Todo se resuelve con HTML, formularios y CSS. El único «dinamismo» es una etiqueta `<meta http-equiv="refresh">` que recarga la página mientras un post se está publicando.
- **Plantillas Jinja2**, que **escapan automáticamente** el contenido (un título como `<script>` se muestra como texto, no se ejecuta).
- **Formularios POST con campo oculto csrf.** Sin él, el servidor responde 400.
- **Adaptable al móvil** (rejillas fluidas, botones grandes), porque el panel se usa mucho desde el teléfono.
- **Estilo de marca:** crema `#F4EFE1`, verde bosque `#2F6B47`, texto casi negro `#182A20`, definidos como variables CSS.

10.2 Las plantillas

Archivo	Qué muestra
<code>base.html</code>	Estructura común: cabecera con marca y menú (Programación y Usuarios solo para administradores, nombre de usuario y «Salir»), avisos (<i>flash</i>) y el bloque de contenido
<code>login.html</code>	Usuario y contraseña
<code>index.html</code>	Panel: avisos (programación inválida, tipos sin hueco, programador sin avisar) y tarjetas de posts agrupadas por estado (macro <code>card</code>), con la hora prevista de los encolados
<code>post.html</code>	Detalle: imágenes en tira horizontal, texto, hashtags, aviso de regla incumplida, tarjeta de estado según el post (en cola, publicando, fallido, publicado) y botones de acción
<code>account.html</code>	Cambio de contraseña propia
<code>users.html</code>	Tabla de usuarios y formulario de alta
<code>schedule.html</code>	Tabla de huecos, formulario para añadir y ajustes de zona horaria y tolerancia

Detalles que conviene notar:

- **El menú lo decide el rol.** `base.html` recibe `user` (lo inyecta un *context processor* de `server.py`) y solo pinta los enlaces de administración si `user.role == 'admin'`. Aun así, el servidor comprueba el rol en cada ruta: ocultar un enlace no es seguridad.
- **La casilla de confirmación** de `post.html` usa el atributo `required`: el navegador no envía el formulario sin marcarla, y el servidor lo comprueba de nuevo.
- **Estados de las tarjetas:** la clase CSS del estado (`pill approved`, `pill failed`...) da el color.

app/templates/base.html – estructura común
<pre> <!doctype html> <html lang="es"> <head> <meta charset="utf-8"> <meta name="viewport" content="width=device-width, initial-scale=1"> <meta name="robots" content="noindex, nofollow"> <title>{% block title %}Aprobaciones{% endblock %} · Trabajo en Excel</title> <link rel="icon" href="data:,"> <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}"> {% block head %}{% endblock %} </head> <body> <header class="top"> Trabajo en Excel · aprobaciones {% if user %} <nav class="nav" aria-label="Principal"> {% if user.role == 'admin' %} </pre>

... continuación

```

<a href="{{ url_for('schedule_page') }}">Programación</a>
<a href="{{ url_for('users_page') }}">Usuarios</a>
{% endif %}
<a href="{{ url_for('account') }}">{{ user.username }}</a>
<form method="post" action="{{ url_for('logout') }}">
  <input type="hidden" name="csrf" value="{{ csrf_token() }}">
  <button class="link" type="submit">Salir</button>
</form>
</nav>
{% endif %}
</header>
<main>
  {% for category, message in get_flashed_messages(with_categories=true) %}
  <div class="flash {{ category }}" role="status">{{ message }}</div>
  {% endfor %}
  {% block content %}{% endblock %}
</main>
</body>
</html>

```

app/templates/login.html – inicio de sesión

```

{% extends "base.html" %}
{% block title %}Entrar{% endblock %}
{% block content %}
<section class="card narrow">
  <h1>Entrar</h1>
  <form method="post">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <label for="username">Usuario</label>
    <input id="username" name="username" type="text" autocomplete="username" autocapitalize="none"
      ↳ spellcheck="false" autofocus required>
    <label for="password">Contraseña</label>
    <input id="password" name="password" type="password" autocomplete="current-password" required>
    <button class="btn primary" type="submit">Entrar</button>
  </form>
</section>
{% endblock %}

```

app/templates/index.html – panel principal

```

{% extends "base.html" %}
{% block title %}Borradores{% endblock %}

{% macro card(p, eta=None) %}
<a class="post-card k-{{ p.kind }}" href="{{ url_for('post_page', pid=p.id) }}">
  {% if p.n_images %}
  
  {% else %}
  <div class="noimg">Sin imagen</div>
  {% endif %}
  <div class="meta">
    <strong>{{ p.title }}</strong>
    {% if eta %}
    <span class="eta">Sale: {{ eta }}</span>
    {% else %}
    <span class="muted">{{ (p.published_at or p.created_at) | fmt }}</span>
    {% endif %}
    <span class="tags"><span class="pill kind">{{ KIND[p.kind] }}</span><span class="pill {{ p.status }}">{{
      ↳ STATUS[p.status] }}</span></span>
  </div>
</a>
{% endmacro %}

{% block content %}
{% set labels = [
  ('draft', 'Por aprobar'),
  ('approved', 'En cola'),
  ('publishing', 'Publicando'),
  ('failed', 'Fallidos'),
  ('published', 'Publicados'),
  ('rejected', 'Rechazados'),
] %}

{% if queue.schedule_error %}
<div class="flash error">La programación no es válida y no se publicará nada: {{ queue.schedule_error }}</div>
{% elif queue.no_slot %}
<div class="flash error">
  Hay posts aprobados de tipo {{ queue.no_slot | map('title') | join(', ') }} sin ningún hueco en la
  ↳ programación: no se publicarán hasta que añadas uno{% if current_user().role == 'admin' %} (<a href="{{

```

... continuación

```

    ↳ url_for('schedule_page') }}">ir a Programación</a>){% endif %}.
</div>
{% elif groups.approved and queue.tick_stale %}
<div class="flash error">
    El programador no se ha comunicado
    {% if queue.last_tick %}desde {{ queue.last_tick | fmt }}{% else %}nunca{% endif %}:
    los posts en cola no se publicarán hasta que vuelva a funcionar (workflow «Programador de publicaciones» en
    ↳ GitHub Actions).
</div>
{% endif %}

{% if not groups.draft and not groups.approved and not groups.publishing and not groups.failed %}
<section class="card empty">
    <h1>No hay nada pendiente</h1>
    <p class="muted">Pídele a Claude que genere un carrusel: aparecerá aquí para que lo revises.</p>
</section>
{% endif %}

{% for key, label in labels %}
    {% if groups[key] %}
        <section>
            <h2>{{ label }} <span class="count">{{ groups[key]|length }}</span></h2>
            <div class="grid">
                {% for p in groups[key] %}{{ card(p, queue.eta.get(p.id) if key == 'approved' else None) }}{% endfor %}
            </div>
        </section>
    {% endif %}
{% endfor %}

<p class="foot muted">
    {% if queue.schedule_text %}Huecos de publicación: {{ queue.schedule_text }}.<br>{% endif %}
    {% if queue.last_tick %}Último aviso del programador: {{ queue.last_tick | fmt }}.<br>{% endif %}
    {% if refreshed %}Token de Instagram renovado el {{ refreshed | fmt }}.{% else %}Token de Instagram: aún sin
    ↳ renovar desde la app (se renueva solo al iniciar sesión o publicar).{% endif %}
</p>
{% endblock %}

```

app/templates/post.html – detalle de un post

```

{% extends "base.html" %}
{% block title %}{{ post.title }}{% endblock %}
{% block head %}{% if refresh %}<meta http-equiv="refresh" content="4">{% endif %}{% endblock %}

{% block content %}
<p><a class="back" href="{{ url_for('index') }}">- Volver</a></p>

<section class="head">
    <h1>{{ post.title }}</h1>
    <p class="muted">
        <span class="pill kind">{{ KIND[post.kind] }}</span>
        <span class="pill {{ post.status }}">{{ STATUS[post.status] }}</span>
        Creado {{ post.created_at | fmt }}
        {% if post.source_url %} · <span class="url">{{ post.source_url }}</span>{% endif %}
    </p>
</section>

{% if n_images %}
<section aria-label="Slides del carrusel">
    <div class="slides kind-{{ post.kind }}">
        {% for i in range(n_images) %}
            <figure>
                
                <figcaption>{{ i + 1 }} / {{ n_images }}</figcaption>
            </figure>
        {% endfor %}
    </div>
</section>

{% elif post.status == 'published' %}
<p class="notice">Las imágenes se borraron del servidor tras publicar.</p>
{% elif post.status == 'rejected' %}
<p class="notice">Post rechazado: las imágenes se borraron.</p>
{% endif %}

<section class="card">
    {% if post.kind == 'historia' %}
        <h2>Historia</h2>
        <p class="muted">Las historias no llevan texto de publicación: todo el mensaje va dentro de la imagen. Se ven
        ↳ 24 horas.</p>
    {% else %}
        <h2>Texto del post</h2>
    {% endif %}

```

... continuación

```
<pre class="caption">{{ post.caption_full }}</pre>
<p class="muted small">{{ post.caption_full|length }} / 2200 caracteres · {{ post.hashtags|length }}
  ↳ hashtags</p>
{% endif %}
{% if post.slides_text %}
<details>
  <summary>Texto de los slides</summary>
  <ol class="slide-text">
    {% for t in post.slides_text %}<li>{{ t }}</li>{% endfor %}
  </ol>
</details>
{% endif %}
</section>

{% if violations %}
<section class="card danger">
  <h2>No se puede publicar</h2>
  <p>Este contenido incumple la restricción de CTA (no se permiten enlaces por comentarios, DM o bio):</p>
  <ul>{% for v in violations %}<li>{{ v }}</li>{% endfor %}</ul>
</section>
{% endif %}

{% if post.status == 'failed' %}
<section class="card danger">
  <h2>Error al publicar</h2>
  <pre class="caption">{{ post.error }}</pre>
  <p class="muted small">Antes de reintentar, comprueba en Instagram que el post no se haya publicado ya: un
  ↳ reintento podría duplicarlo.</p>
</section>
{% endif %}

{% if post.status == 'published' %}
<section class="card ok">
  <h2>Publicado</h2>
  {% if post.approved_by %}<p class="muted small">Aprobado por {{ post.approved_by }}.</p>{% endif %}
  <p>{{ post.published_at | fmt }}{% if post.permalink %} · <a href="{{ post.permalink }}" rel="noopener
  ↳ noreferrer">Ver en Instagram</a>{% endif %}</p>
</section>
{% endif %}

{% if post.status == 'approved' %}
<section class="card queued">
  <h2>En cola</h2>
  {% if post.approved_by %}<p class="muted small">Aprobado por {{ post.approved_by }}.</p>{% endif %}
  <p>
    {% if queue.eta.get(post.id) %}Se publicará el <strong>{{ queue.eta.get(post.id) }}</strong> (hora de
    ↳ España).
    {% else %}Se publicará en el próximo hueco libre.{% endif %}
  </p>
  <p class="muted small">Un solo post por hueco, por orden de aprobación. Huecos: {{ queue.schedule_text }}.</p>
</section>
{% endif %}

{% if post.status == 'publishing' %}
<section class="card">
  <h2>Publicando en Instagram...</h2>
  {% if is_stuck(post) %}
  <p>Lleva demasiado tiempo. Puede haberse interrumpido; comprueba en Instagram si ya salió.</p>
  <form method="post" action="{{ url_for('unstuck', pid=post.id) }}">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <button class="btn" type="submit">Marcar como fallido</button>
  </form>
  {% else %}
  <p class="muted">Esta página se actualiza sola.</p>
  {% endif %}
</section>
{% endif %}

{% if post.status == 'draft' %}
<section class="card actions">
  <form method="post" action="{{ url_for('approve', pid=post.id) }}">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <label class="check">
      <input type="checkbox" name="confirm" required {% if violations %}disabled{% endif %}>
      He revisado las imágenes y el texto. Autorizo su publicación en el próximo hueco libre.
    </label>
    <button class="btn primary" type="submit" {% if violations %}disabled{% endif %}>Aprobar y poner en
    ↳ cola</button>
  </form>
  <form method="post" action="{{ url_for('reject', pid=post.id) }}">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <button class="btn" type="submit">Rechazar y borrar imágenes</button>
  </form>
</section>
{% endif %}
```

... continuación

```

</form>
</section>
{% elif post.status == 'approved' %}
<section class="card actions">
  <form method="post" action="{{ url_for('unqueue', pid=post.id) }}">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <button class="btn" type="submit">Sacar de la cola</button>
  </form>
  <form method="post" action="{{ url_for('reject', pid=post.id) }}">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <button class="btn" type="submit">Rechazar y borrar imágenes</button>
  </form>
</section>
{% elif post.status == 'failed' %}
<section class="card actions">
  <form method="post" action="{{ url_for('reopen', pid=post.id) }}">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <button class="btn" type="submit">Volver a borrador</button>
  </form>
  <form method="post" action="{{ url_for('reject', pid=post.id) }}">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <button class="btn" type="submit">Rechazar y borrar imágenes</button>
  </form>
</section>
{% endif %}

{% if post.status in ('published', 'rejected', 'failed', 'draft') %}
<form class="delete" method="post" action="{{ url_for('delete', pid=post.id) }}">
  <input type="hidden" name="csrf" value="{{ csrf_token() }}">
  <button class="link danger-link" type="submit">Eliminar este registro</button>
</form>
{% endif %}
{% endblock %}

```

app/templates/account.html – mi cuenta

```

{% extends "base.html" %}
{% block title %}Mi cuenta{% endblock %}
{% block content %}
<p><a class="back" href="{{ url_for('index') }}">- Volver</a></p>
<h1>Mi cuenta</h1>
<section class="card">
  <p><strong>{{ user.username }}</strong> · {{ roles[user.role] }}</p>
</section>

{% if user.builtin %}
<section class="card">
  <h2>Contraseña</h2>
  <p class="muted">La cuenta integrada «admin» usa la variable <code>ADMIN_PASSWORD</code> de Render: se cambia
  ↳ allí, no desde el panel.</p>
</section>
{% else %}
<section class="card">
  <h2>Cambiar contraseña</h2>
  <form method="post" class="stack">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <label for="current_password">Contraseña actual</label>
    <input id="current_password" name="current_password" type="password" autocomplete="current-password"
    ↳ required>
    <label for="new_password">Contraseña nueva <span class="muted">(mínimo {{ min_password }}
    ↳ caracteres)</span></label>
    <input id="new_password" name="new_password" type="password" autocomplete="new-password" minlength="{{
    ↳ min_password }}" required>
    <label for="again">Repite la contraseña nueva</label>
    <input id="again" name="again" type="password" autocomplete="new-password" minlength="{{ min_password }}"
    ↳ required>
    <button class="btn primary" type="submit">Cambiar contraseña</button>
  </form>
</section>
{% endif %}
{% endblock %}

```

app/templates/users.html – gestión de usuarios

```

{% extends "base.html" %}
{% block title %}Usuarios{% endblock %}
{% block content %}
<p><a class="back" href="{{ url_for('index') }}">- Volver</a></p>
<h1>Usuarios</h1>
<p class="muted">Quien tenga una cuenta puede entrar al panel y aprobar o rechazar posts. Los

```

... continuación

```

<strong>administradores</strong> además gestionan los usuarios y la programación.</p>

<section class="card">
  <h2>Con acceso</h2>
  <div class="table-wrap">
    <table>
      <thead><tr><th>Usuario</th><th>Rol</th><th>Creado</th><th><span class="sr">Acciones</span></th></tr></thead>
      <tbody>
        <tr>
          <td><strong>admin</strong>{% if user.builtin %} <span class="muted">(tú)</span>{% endif %}</td>
          <td>{{ roles['admin'] }}</td>
          <td class="muted">Cuenta integrada</td>
          <td class="muted small">No se puede eliminar</td>
        </tr>
        {% for u in users %}
        <tr>
          <td><strong>{{ u.username }}</strong>{% if u.id == user.id %} <span class="muted">(tú)</span>{% endif %}</td>
          <td>{{ roles[u.role] }}</td>
          <td class="muted">{{ u.created_at | fmt }}{% if u.created_by %}<br><span class="small">por {{ u.created_by }}</span>{% endif %}</td>
          <td>
            {% if u.id != user.id %}
            <form method="post" action="{{ url_for('users_delete', uid=u.id) }}">
              <input type="hidden" name="csrf" value="{{ csrf_token() }}">
              <button class="btn small danger" type="submit">Eliminar</button>
            </form>
            {% endif %}
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
  {% if not users %}<p class="muted">Todavía no has creado ningún usuario.</p>{% endif %}
</section>

<section class="card">
  <h2>Crear usuario</h2>
  <form method="post" action="{{ url_for('users_create') }}" class="stack">
    <input type="hidden" name="csrf" value="{{ csrf_token() }}">
    <label for="username">Usuario <span class="muted">(3-32 caracteres: letras, números, . _ -)</span></label>
    <input id="username" name="username" type="text" autocomplete="off" autocapitalize="none" spellcheck="false"
      <minlength="3" maxlength="32" required>
    <label for="password">Contraseña inicial <span class="muted">(mínimo {{ min_password }}
      <caracteres)</span></label>
    <input id="password" name="password" type="password" autocomplete="new-password" minlength="{{ min_password }}"
      <maxlength="32" required>
    <label for="role">Rol</label>
    <select id="role" name="role">
      <option value="editor">{{ roles['editor'] }}: aprueba y rechaza posts</option>
      <option value="admin">{{ roles['admin'] }}: además, usuarios y programación</option>
    </select>
    <button class="btn primary" type="submit">Crear usuario</button>
  </form>
  <p class="muted small">Eliminar un usuario le quita el acceso al instante. Cada persona puede cambiar su
    contraseña en su cuenta.</p>
</section>
{% endblock %}

```

app/templates/schedule.html – gestión de la programación

```

{% extends "base.html" %}
{% block title %}Programación{% endblock %}
{% block content %}
<p><a class="back" href="{{ url_for('index') }}">- Volver</a></p>
<h1>Programación</h1>
<p class="muted">En cada hueco se publica <strong>un</strong> post de ese tipo: el aprobado más antiguo de la
  cola. Si a esa hora no hay ninguno aprobado, el hueco se pierde.</p>

{% if error %}
<div class="flash error">La programación no es válida y no se publicará nada: {{ error }}</div>
{% else %}

<section class="card">
  <h2>Huecos de publicación</h2>
  {% if slots %}
  <div class="table-wrap">
    <table>
      <thead><tr><th>Día</th><th>Hora</th><th>Tipo</th><th><span class="sr">Acciones</span></th></tr></thead>
      <tbody>

```

... continuación

```

    {% for s in slots %}
    <tr>
    <td>{{ s.day | capitalize }}</td>
    <td>{{ s.slot_time }}</td>
    <td>{{ s.kind_label | capitalize }}</td>
    <td>
        <form method="post" action="{{ url_for('schedule_delete', sid=s.id) }}">
        <input type="hidden" name="csrf" value="{{ csrf_token() }}">
        <button class="btn small danger" type="submit">Eliminar</button>
        </form>
    </td>
    </tr>
    {% endfor %}
</tbody>
</table>
</div>
{% else %}
<p class="notice">No hay ningún hueco: no se publicará nada hasta que añadas uno.</p>
{% endif %}
<p class="muted small">Horas de {{ cfg.tz_name }} (con cambio de hora verano/invierno automático).</p>
</section>

<section class="card">
<h2>Añadir hueco</h2>
<form method="post" action="{{ url_for('schedule_add') }}" class="row">
<input type="hidden" name="csrf" value="{{ csrf_token() }}">
<div>
<label for="day">Día</label>
<select id="day" name="day">
    {% for d in days %}<option value="{{ loop.index0 }}">{{ d | capitalize }}</option>{% endfor %}
</select>
</div>
<div>
<label for="time">Hora</label>
<input id="time" name="time" type="time" required>
</div>
<div>
<label for="kind">Tipo</label>
<select id="kind" name="kind">
    {% for value, label in kinds %}<option value="{{ value }}">{{ label }}</option>{% endfor %}
</select>
</div>
<button class="btn primary" type="submit">Añadir</button>
</form>
<p class="muted small">El programador avisa cada 30 minutos, así que un post sale como mucho ~30 min después
↳ de la hora indicada. Un hueco nuevo solo cuenta a partir de ahora: si añades «hoy 10:00» a las 11:00, no
↳ se publica.</p>
</section>

<section class="card">
<h2>Ajustes</h2>
<form method="post" action="{{ url_for('schedule_settings') }}" class="stack">
<input type="hidden" name="csrf" value="{{ csrf_token() }}">
<label for="tz">Zona horaria</label>
<input id="tz" name="tz" type="text" value="{{ cfg.tz_name }}" autocapitalize="none" spellcheck="false"
↳ required>
<label for="tolerance">Tolerancia <span class="muted">(minutos, de {{ tolerance_range[0] }} a {{
↳ tolerance_range[1] }})</span></label>
<input id="tolerance" name="tolerance" type="number" min="{{ tolerance_range[0] }}" max="{{
↳ tolerance_range[1] }}" step="1" value="{{ (cfg.tolerance.total_seconds() // 60) | int }}" required>
<p class="muted small">Tiempo tras la hora del hueco durante el que aún se publica si el aviso del
↳ programador se retrasa. Pasado ese margen, el hueco se pierde.</p>
<button class="btn" type="submit">Guardar ajustes</button>
</form>
</section>
{% endif %}
{% endblock %}

```

10.3 Los estilos

Una sola hoja, `app/static/style.css`, con variables CSS para la paleta y bloques por zonas: base y tipografía, cabecera, tarjetas, formularios, rejilla de posts, etiquetas de estado, tira de slides, acciones y, al final, los estilos de usuarios y programación (navegación, campos, tablas con desplazamiento horizontal en móvil, botones pequeños y de peligro). Las **historias** se muestran con proporción 9:16 y las publicaciones con un ancho algo mayor, para que cada tipo se vea como será en Instagram.

app/static/style.css – estilos

```

:root {
  --bg: #F4EFE1;
  --card: #FBF8EE;
  --ink: #182A20;
  --muted: #6B736C;
  --accent: #2F6B47;
  --accent-ink: #FFFFFF;
  --line: #DDD6C2;
  --danger: #A33A2C;
  --danger-bg: #F8E6E1;
  --ok-bg: #E2EFE6;
  color-scheme: light;
}
* { box-sizing: border-box; }
html { -webkit-text-size-adjust: 100%; }
body {
  margin: 0; background: var(--bg); color: var(--ink);
  font: 16px/1.5 system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
}
main { max-width: 860px; margin: 0 auto; padding: 8px 16px 48px; }
h1 { font-size: 1.5rem; line-height: 1.2; margin: 8px 0; }
h2 { font-size: 1.05rem; margin: 24px 0 10px; }
.card h2 { margin-top: 0; }
a { color: var(--accent); }
.muted { color: var(--muted); }
.small { font-size: .85rem; }

.top {
  display: flex; justify-content: space-between; align-items: center;
  padding: 14px 16px; max-width: 860px; margin: 0 auto;
}
.brand { font-weight: 800; letter-spacing: .02em; text-decoration: none; color: var(--ink); }
.brand span { font-weight: 500; color: var(--muted); }
button.link {
  background: none; border: 0; padding: 8px 0; font: inherit; color: var(--accent);
  cursor: pointer; text-decoration: underline;
}
.back { text-decoration: none; }

.card {
  background: var(--card); border: 1px solid var(--line); border-radius: 12px;
  padding: 16px; margin: 16px 0;
}
.card.narrow { max-width: 380px; margin: 40px auto; }
.card.empty { text-align: center; padding: 32px 16px; }
.card.danger { background: var(--danger-bg); border-color: #E3B8AE; }
.card.queued { background: #E9F1FA; border-color: #BDD3EA; }
.card.ok { background: var(--ok-bg); border-color: #B8D4C0; }
.notice { color: var(--muted); font-style: italic; }

.flash { padding: 12px 14px; border-radius: 10px; margin: 12px 0; background: var(--ok-bg); }
.flash.error { background: var(--danger-bg); color: var(--danger); }

label { display: block; font-weight: 600; margin: 12px 0 6px; }
input[type=password] {
  width: 100%; padding: 12px; font: inherit; border: 1px solid var(--line);
  border-radius: 8px; background: #fff;
}
.btn {
  display: inline-block; padding: 12px 18px; font: inherit; font-weight: 700;
  border-radius: 10px; border: 1px solid var(--line); background: #fff; color: var(--ink);
  cursor: pointer; min-height: 44px;
}
.btn.primary { background: var(--accent); border-color: var(--accent); color: var(--accent-ink); }
.btn.disabled { opacity: .5; cursor: not-allowed; }
.btn:focus-visible, input:focus-visible, button:focus-visible, a:focus-visible {
  outline: 3px solid #7FB394; outline-offset: 2px;
}
.card.narrow .btn { width: 100%; margin-top: 16px; }

.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(150px, 1fr)); gap: 14px; }
.post-card {
  display: block; text-decoration: none; color: inherit; background: var(--card);
  border: 1px solid var(--line); border-radius: 12px; overflow: hidden;
}
.post-card img, .noimg { width: 100%; aspect-ratio: 4 / 5; object-fit: cover; display: block; background:
  #E8E2CF; }
.noimg { display: grid; place-items: center; color: var(--muted); font-size: .85rem; }
.post-card .meta { padding: 10px 12px 12px; display: grid; gap: 4px; }
.post-card strong { line-height: 1.25; }
.count { color: var(--muted); font-weight: 500; }

```

... continuación

```

.pill {
  display: inline-block; padding: 2px 9px; border-radius: 999px; font-size: .78rem;
  font-weight: 700; background: #E8E2CF; width: fit-content;
}
.pill.draft { background: #F3E3A8; }
.pill.publishing { background: #CFE3F5; }
.pill.approved { background: #D9E8F7; color: #1F4E79; }
.eta { font-size: .85rem; font-weight: 600; color: #1F4E79; }
.pill.published { background: var(--ok-bg); color: var(--accent); }
.pill.failed { background: var(--danger-bg); color: var(--danger); }

.slides {
  display: flex; gap: 12px; overflow-x: auto; scroll-snap-type: x mandatory;
  padding: 4px 2px 12px; margin: 8px 0;
}
.slides figure { margin: 0; flex: 0 0 min(78vw, 320px); scroll-snap-align: start; }
.slides img {
  width: 100%; aspect-ratio: 4 / 5; object-fit: cover; display: block;
  border-radius: 10px; border: 1px solid var(--line); background: #E8E2CF;
}
.slides figcaption { text-align: center; color: var(--muted); font-size: .85rem; padding-top: 4px; }

pre.caption {
  white-space: pre-wrap; overflow-wrap: anywhere; margin: 0 0 8px;
  font: inherit; background: #fff; border: 1px solid var(--line); border-radius: 8px; padding: 12px;
}
.url { overflow-wrap: anywhere; }
details { margin-top: 8px; }
summary { cursor: pointer; font-weight: 600; }
.slide-text { padding-left: 20px; }

.actions { display: grid; gap: 14px; }
.actions form { display: grid; gap: 10px; }
.check { display: flex; gap: 10px; align-items: flex-start; font-weight: 500; margin: 0; }
.check input { margin-top: 5px; width: 18px; height: 18px; flex: none; }
.delete { margin-top: 28px; text-align: center; }
.danger-link { color: var(--danger); font-size: .9rem; }
.foot { text-align: center; margin-top: 36px; font-size: .85rem; }

/* tipos de post */
.pill.kind { background: #E4EAF0; color: #2C4A63; }
.tags { display: flex; flex-wrap: wrap; gap: 6px; }
.post-card.k-historia img, .post-card.k-historia .noimg { aspect-ratio: 9 / 16; }
.slides.kind-historia figure { flex: 0 0 min(56vw, 240px); }
.slides.kind-historia img { aspect-ratio: 9 / 16; }
.slides.kind-publicacion figure { flex: 0 0 min(86vw, 360px); }

/* usuarios y programación */
.nav { display: flex; align-items: center; gap: 16px; flex-wrap: wrap; justify-content: flex-end; }
.nav a { text-decoration: none; font-weight: 600; }
.nav form { margin: 0; }
input[type=text], input[type=number], input[type=time], select {
  width: 100%; padding: 12px; font: inherit; border: 1px solid var(--line);
  border-radius: 8px; background: #fff; color: var(--ink);
}
.stack { display: grid; gap: 2px; }
.stack .btn { margin-top: 14px; justify-self: start; }
.row { display: flex; flex-wrap: wrap; gap: 12px; align-items: flex-end; }
.row > div { flex: 1 1 130px; }
.row .btn { flex: none; }
.row label { margin-top: 0; }
table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; }
th, td { text-align: left; padding: 10px 8px; border-bottom: 1px solid var(--line); vertical-align: middle; }
th { font-size: .8rem; color: var(--muted); text-transform: uppercase; letter-spacing: .04em; }
td form { margin: 0; }
.btn.small { min-height: 36px; padding: 6px 12px; font-size: .9rem; }
.btn.danger { color: var(--danger); border-color: #E3B8AE; }
.sr { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; }
code { background: #E8E2CF; padding: 1px 5px; border-radius: 4px; }

```

11. Pruebas automáticas

11.1 Estrategia

Las pruebas (`app/test_server.py`) usan `unittest`, que viene con Python. Ejecutan la app **de verdad**, con su base de datos SQLite temporal, pero con **Instagram simulado**: nunca se llama a la API real. Se lanzan con:

```
cd app
python -m unittest -v
```

Reglas del arnés de pruebas:

- **Las variables de entorno se fijan antes de importar la app.** La app lee `os.environ` al importarse, así que el archivo empieza con `os.environ.update(...)` y solo después hace `import server`.
- **Base de datos temporal.** `SQLITE_PATH` apunta a un archivo en una carpeta temporal, y cada prueba **vacía las tablas** en `setUp`.
- **Instagram falso.** Cada prueba sustituye `instagram.publish` (y `instagram.refresh`) por funciones falsas y las restaura en `tearDown`.
- **Reloj falso.** `server.utcnow` se sustituye por una función que devuelve la hora que quiera la prueba: así se prueba «el martes a las 15:30» sin esperar.
- **Programación controlada.** `schedule.SCHEDULE_FILE` apunta a un archivo de prueba antes de sembrar, para no depender de la programación real.
- **Límite de login limpio.** El diccionario de intentos fallidos es global; `setUp` lo vacía para que las pruebas no se bloqueen unas a otras.

11.2 Qué cubren

Clase de pruebas	Qué comprueba
<code>ApiTests</code>	La API exige clave; borradores válidos; rechazo de PNG y de «falsos» JPEG; rechazo de CTA prohibidos; listado sin exponer texto; reglas por tipo; proporciones por tipo; lectura de la cabecera JPEG; límites
<code>AccessTests</code>	El panel exige login; login correcto e incorrecto; CSRF; redirección <code>next</code> solo relativa; las imágenes solo se ven con sesión; las páginas se pintan
<code>ScheduleTests</code>	Lectura de huecos; hueco vigente en verano; cambio de hora de invierno; próximos huecos; errores de formato; acentos
<code>QueueTests</code>	Aprobar no publica al momento; exige confirmación; un post por hueco y por orden de aprobación; publicar borra imágenes y las expone solo mientras se publica; hueco vacío se pierde; fuera de tolerancia no publica; forzar; fallo conserva imágenes y oculta credenciales; contenido prohibido no se aprueba ni publica; doble aprobación; sacar de la cola
<code>KindQueueTests</code>	Publicación e historia salen juntas el jueves; cada tipo con su cola; un fallo no impide publicar el otro tipo; migración de una base sin columna <code>kind</code>
<code>UserTests</code>	Alta y acceso; credenciales incorrectas; eliminar corta el acceso al instante; no borrar la propia cuenta; el editor no accede a administración; validaciones de alta; cambio de contraseña; sesiones antiguas inválidas; quién aprueba queda registrado
<code>ScheduleAdminTests</code>	Siembra desde <code>schedule.txt</code> ; manda la base de datos; añadir y borrar huecos; validaciones; un hueco nuevo publica en el aviso; un hueco recién creado no rescata una hora pasada; sin huecos no falla; ajustes de zona y tolerancia
<code>TokenTests</code>	Renovación y guardado; un token nuevo de la variable manda; un fallo al renovar no rompe
<code>GuardTests</code>	Textos prohibidos y permitidos de la regla de CTA

COMPRUEBA Al terminar de reconstruir cada capítulo, ejecuta las pruebas. En el estado final del proyecto son **72 pruebas** y todas deben pasar. Si una falla, léela: cada una nombra en español lo que verifica.

11.3 Cómo escribir una prueba nueva

1. Crea un método `test_...` en la clase que corresponda (hereda de `Base`).
2. Usa las ayudas: `self.draft("historia")` sube un borrador válido; `self.approve(pid)` inicia sesión y aprueba; `self.tick("2026-09-22T13:30:00+00:00")` simula un aviso del programador a esa hora (UTC); `self.status(pid)` lee el post de la base de datos.
3. Simula Instagram con `instagram.publish = lambda ...` si la prueba llega a publicar.
4. Comprueba el resultado con `self.assertEqual` / `assertIn`.

El archivo completo está en el **Apéndice A**.

12. El generador de imágenes

12.1 Qué resuelve

Instagram solo acepta **JPEG**, y los textos de estos posts son el contenido principal: un titular grande, un remate y una llamada a la acción. Hacer eso a mano en un editor gráfico es lento y poco repetible. El generador (`carousel_gen.py`) toma un archivo **JSON** con el texto de cada imagen y produce los JPG con el estilo de la marca.

Pillow dibuja el texto y, lo más importante, **mide el ancho real del texto con la fuente exacta** para hacer el salto de línea automático. Es la causa de que casi nunca se salga texto de la imagen (*overflow*).

POR QUÉ Escribir los saltos de línea a mano «a ojo» es la mayor fuente de textos cortados. El script **nunca** parte líneas a mano: prueba palabra a palabra si cabe dentro del ancho útil y, si no, baja de línea. Si un slide queda demasiado largo, avisa con «POSIBLE OVERFLOW»: la solución es **acortar el texto**, no reducir la fuente (rompería la jerarquía visual).

12.2 Los tres formatos

Formato (opción --format)	Imagen	Tema	Elementos
<code>carrusel</code>	1080×1350 (4:5)	Crema, letra oscura	Flecha de deslizar, numeración <code>01 / 05</code> , CTA en el último slide
<code>publicacion</code>	1080×1350 (4:5)	Verde, letra blanca	Sin flecha ni numeración; CTA abajo a la izquierda
<code>historia</code>	1080×1920 (9:16)	Verde, letra blanca	Sin flecha; márgenes de seguridad de ~250 px arriba y abajo

Por qué dos temas. Como la publicación y la historia salen el mismo día, se les dio un fondo verde con letra blanca para que **se distingan a simple vista** del carrusel (crema con letra oscura). El generador elige el tema según el formato (`FORMAT_THEME`); no hay nada que indicar al generar.

Márgenes de seguridad de la historia. La interfaz de Instagram tapa unos 250 px por arriba (perfil) y por abajo (caja de respuesta). Por eso en el formato historia la cabecera está en $y=300$, el texto empieza en $y\approx 760$ y el pie/CTA queda por encima de $y\approx 1620$.

12.3 Cómo está construido

- **Supersampling.** Se dibuja a doble resolución ($S = 2$) y se reduce con `LANCZOS`: los bordes del texto quedan suaves.
- **Constantes de marca.** Colores `BG`, `DARK`, `ACCENT`, `GREY`, `WHITE`, `GREY_ON_GREEN`; márgenes de 90 px; ancho útil = ancho – 2 × margen.
- **THEMES y FORMAT_THEME.** Cada tema define color de fondo, del texto en negrita, de la itálica, del acento (línea, CTA, flecha) y del texto secundario.
- **LAYOUTS.** Posiciones verticales de cada formato (`header_y`, `start_y`, línea, URL, CTA), si lleva flechas o numeración y el límite (`limit`) a partir del cual avisa de *overflow*.
- **Fuentes** (`assets/fonts/`, incluidas para no depender del sistema): *DejaVu Sans Condensed* (regular y negrita) y *DejaVu Serif Condensed Bold Italic* para el remate. Se distribuyen con licencia libre (el archivo `LICENSE-DejaVu.txt` va en la misma carpeta). Descárgalas de dejavu-fonts.github.io y colócalas con esos nombres.
- **wrap / measure.** Miden con `getlength` y parten por palabras.

- `draw_runs`. Dibuja los *runs* del JSON: `"bold"` (84 px; el texto se escribe ya en mayúsculas) y `"italic"` (88 px, el «remate»).
- `build_slide`. Compone una imagen completa: fondo, texto, cabecera (`TRABAJO EN EXCEL®` y la etiqueta a la derecha), línea de pie, URL, CTA, flecha y numeración según el formato.

12.4 El archivo de entrada (JSON)

Una lista con un objeto por imagen:

```
[
  {
    "runs": [
      ["TU EQUIPO HACE SPRINTS.", "bold"],
      ["Pero no hace Scrum.", "italic"]
    ],
    "slide_no": "01 / 05",
    "swipe_hint": true,
    {
      "runs": [
        ["HAZ EL TEST GRATIS.", "bold"]
      ],
      "cta_text": "HAZ EL TEST GRATIS.",
      "swipe_hint": false
    }
  }
]
```

Campo	Uso
<code>runs</code>	Lista de <code>[texto, "bold" o "italic"]</code>
<code>slide_no</code>	Numeración (solo carrusel); <code>null</code> en el slide del CTA
<code>cta_text</code>	Llamada a la acción del último slide o de la imagen única (máx. ~28 caracteres)
<code>swipe_hint</code>	Flecha de deslizar (solo carrusel; <code>false</code> si hay CTA)
<code>start_y</code>	Ajustar la posición del bloque de texto si es muy largo

12.5 Uso

```
python .claude/skills/carruseles-app/scripts/carousel_gen.py spec.json /ruta/temporal/post --format carrusel
```

Escribe `slide-01.jpg`, `slide-02.jpg`... e imprime dónde termina el texto de cada una. Para `publicacion` e `historia`, el JSON debe tener **exactamente una** imagen (el script lo comprueba). Genera **siempre en un directorio temporal**, nunca dentro del repositorio.

.claude/skills/carruseles-app/scripts/carousel_gen.py – generador de imágenes

```
"""
Generador de imágenes de Instagram para trabajoenexcel.com.
Dibuja con Pillow (wrap de texto medido con la fuente real, sin overflow) y guarda JPG,
el único formato que admite la API de Instagram. No necesita rsvg ni fuentes del
sistema: usa las de assets/fonts.

Uso:
    python carousel_gen.py slides.json /tmp/post [--format carrusel|publicacion|historia]

Formatos (--format, por defecto carrusel):
    carrusel      2-10 slides de 1080x1350 (4:5), fondo crema y letra oscura, con flecha de
                  deslizar y numeración
    publicacion   1 imagen de 1080x1350 (4:5), fondo verde y letra blanca, sin flecha ni numeración
    historia      1 imagen de 1080x1920 (9:16), fondo verde y letra blanca, con márgenes de
                  seguridad arriba y abajo (~250 px que Instagram tapa con su interfaz) y sin flecha

Genera slide-01.jpg, slide-02.jpg... en la carpeta de salida.

slides.json: lista de objetos con esta forma:
{
  "runs": [
    ["TEXTO BOLD EN MAYUSCULAS", "bold"],
    ["frase en italica verde", "italic"]
  ],
  "slide_no": "01 / 06",           // opcional, solo carrusel; null en el slide de CTA
  "cta_text": "PLANTILLA DE SPRINT.", // opcional, solo en el slide final / imagen única
  "swipe_hint": true,             // opcional, solo carrusel; false si hay cta_text
  "start_y": 520                  // opcional, ajustar si el bloque de texto es largo
}

Requiere: pip install Pillow
"""
import argparse
import json
import sys
from pathlib import Path
from PIL import Image, ImageDraw, ImageFont
```

... continuación

```
W = 1080
S = 2 # supersampling: se dibuja a 2x y se reduce (bordes suaves)
BG = "#F4EFE1" # cream / hueso (trabajoenexcel.com)
DARK = "#182A20" # texto principal
ACCENT = "#2F6B47" # verde bosque de acento – ver references/style-guide.md
GREY = "#8A9088"
WHITE = "#FFFFFF"
GREY_ON_GREEN = "#B9D3C4" # texto secundario sobre el fondo verde

# Temas: el carrusel va en crema con letra oscura; publicación e historia, en verde con
# letra blanca, para que los dos mensajes de la semana se distingan a simple vista.
THEMES = {
    "crema": {"bg": BG, "bold": DARK, "italic": ACCENT, "accent": ACCENT, "grey": GREY},
    "verde": {"bg": ACCENT, "bold": WHITE, "italic": WHITE, "accent": WHITE, "grey": GREY_ON_GREEN},
}
FORMAT_THEME = {"carrusel": "crema", "publicacion": "verde", "historia": "verde"}

MARGIN = 90
MAX_W = W - 2 * MARGIN

def _layout(h, header_y, start_y, line_y, url_y, cta_y, arrows, numbering, limit):
    return {"H": h, "header_y": header_y, "start_y": start_y, "line_y": line_y, "url_y": url_y,
            "cta_y": cta_y, "arrows": arrows, "numbering": numbering, "limit": limit}

LAYOUTS = {
    "carrusel": _layout(1350, 90, 520, 1350 - 90, 1350 - 60, 1350 - 160, True, True, 1350 - 200),
    "publicacion": _layout(1350, 90, 520, 1350 - 90, 1350 - 60, 1350 - 160, False, False, 1350 - 200),
    # historia: Instagram tapa ~250 px arriba y abajo; nada importante fuera de y=250..1670
    "historia": _layout(1920, 300, 760, 1920 - 330, 1920 - 300, 1920 - 430, False, False, 1920 - 560),
}

FONTS = Path(__file__).resolve().parent.parent / "assets" / "fonts"
BOLD_PATH = str(FONTS / "DejaVuSansCondensed-Bold.ttf")
REGULAR_PATH = str(FONTS / "DejaVuSansCondensed.ttf")
ITALIC_PATH = str(FONTS / "DejaVuSerifCondensed-BoldItalic.ttf")

_font_cache = {}
def get_font(path, size):
    key = (path, size)
    if key not in _font_cache:
        _font_cache[key] = ImageFont.truetype(path, size)
    return _font_cache[key]

def measure(text, path, size):
    return get_font(path, size).getlength(text)

def wrap(text, path, size, max_w):
    words = text.split(" ")
    lines, cur = [], ""
    for w in words:
        trial = (cur + " " + w).strip()
        if measure(trial, path, size) <= max_w:
            cur = trial
        else:
            if cur:
                lines.append(cur)
            cur = w
    if cur:
        lines.append(cur)
    return lines

def draw_text(d, x, y, text, path, size, fill, spacing=0, anchor="l"):
    """Texto con baseline en y. anchor 'l' (izquierda) o 'r' (derecha)."""
    f = get_font(path, size * S)
    if not spacing:
        d.text((x * S, y * S), text, font=f, fill=fill, anchor="l" + "s" if anchor == "l" else "rs")
        return
    widths = [f.getlength(c) for c in text]
    total = sum(widths) + spacing * S * (len(text) - 1)
    cx = x * S if anchor == "l" else x * S - total
    for c, w in zip(text, widths):
        d.text((cx, y * S), c, font=f, fill=fill, anchor="ls")
        cx += w + spacing * S
    return cx # posición donde terminaría el siguiente carácter

def draw_runs(d, runs, start_y, theme, x=MARGIN):
    y = start_y
    for text, style in runs:
        if style == "bold":
```

... continuación

```

        size, path, fill, lh = 84, BOLD_PATH, theme["bold"], 98
    else:
        size, path, fill, lh = 88, ITALIC_PATH, theme["italic"], 102
    for ln in wrap(text, path, size, MAX_W):
        draw_text(d, x, y, ln, path, size, fill)
        y += lh
    return y

def arrow_icon(d, cx, cy, color, r=42):
    d.ellipse([(cx - r) * S, (cy - r) * S, (cx + r) * S, (cy + r) * S], outline=color, width=3 * S)
    lw = int(3.5 * S)
    d.line([(cx - 16) * S, cy * S, (cx + 14) * S, cy * S], fill=color, width=lw)
    d.line([(cx + 2) * S, (cy - 12) * S, (cx + 16) * S, cy * S], fill=color, width=lw)
    d.line([(cx + 2) * S, (cy + 12) * S, (cx + 16) * S, cy * S], fill=color, width=lw)

def build_slide(runs, fmt="carrusel", header_left="TRABAJO EN EXCEL", header_right="SOCIAL AD",
               cta_text=None, footer="TRABAJOENEXCEL.COM", swipe_hint=True,
               slide_no=None, start_y=None):
    L = LAYOUTS[fmt]
    T = THEMES[FORMAT_THEME[fmt]]
    H = L["H"]
    if start_y is None:
        start_y = L["start_y"]
    img = Image.new("RGB", (W * S, H * S), T["bg"])
    d = ImageDraw.Draw(img)

    end_y = draw_runs(d, runs, start_y, T)

    # cabecera: marca con ® en superíndice + etiqueta a la derecha
    hy = L["header_y"]
    x_end = draw_text(d, MARGIN, hy, header_left, BOLD_PATH, 30, T["bold"], spacing=2)
    d.text((x_end, (hy - 14) * S), "®", font=get_font(BOLD_PATH, 18 * S), fill=T["bold"], anchor="ls")
    draw_text(d, W - MARGIN, hy, header_right, REGULAR_PATH, 24, T["grey"], spacing=3, anchor="r")

    # pie: línea, url y, según el formato, CTA / flecha / numeración
    line_y = L["line_y"]
    x2 = 380 if cta_text else 290
    d.rectangle([MARGIN * S, (line_y - 2) * S, x2 * S, (line_y + 2) * S], fill=T["accent"])
    draw_text(d, W - MARGIN, L["url_y"], footer, REGULAR_PATH, 22, T["grey"], spacing=2, anchor="r")
    if cta_text:
        draw_text(d, MARGIN, L["cta_y"], cta_text, BOLD_PATH, 40, T["accent"], spacing=1)
        if L["arrows"]:
            arrow_icon(d, W - 160, L["cta_y"] - 25, T["accent"])
    elif swipe_hint and L["arrows"]:
        arrow_icon(d, W - 160, L["cta_y"] - 25, T["accent"], r=36)
    if slide_no and L["numbering"]:
        draw_text(d, MARGIN, L["cta_y"], slide_no, REGULAR_PATH, 22, T["grey"], spacing=2)

    return img.resize((W, H), Image.LANCZOS), end_y

def main():
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser(description="Genera las imágenes JPG de un post de Instagram")
    parser.add_argument("spec")
    parser.add_argument("out_dir")
    parser.add_argument("--format", dest="fmt", choices=sorted(LAYOUTS), default="carrusel")
    args = parser.parse_args()

    out = Path(args.out_dir)
    out.mkdir(parents=True, exist_ok=True)
    slides = json.loads(Path(args.spec).read_text(encoding="utf-8"))
    if args.fmt != "carrusel" and len(slides) != 1:
        sys.exit(f"El formato {args.fmt} lleva exactamente 1 imagen (el JSON tiene {len(slides)}).")
    H = LAYOUTS[args.fmt]["H"]
    for i, sdef in enumerate(slides, start=1):
        runs = [tuple(r) for r in sdef["runs"]]
        kwargs = {k: v for k, v in sdef.items() if k != "runs"}
        img, end_y = build_slide(runs, fmt=args.fmt, **kwargs)
        name = f"slide-{i:02d}.jpg"
        img.save(out / name, "JPEG", quality=95)
        overflow = " ⚠ POSIBLE OVERFLOW (ajusta start_y o acorta el texto)" if end_y >
        ↳ LAYOUTS[args.fmt]["limit"] else ""
        print(f"{name} {img.width}x{H} (texto termina en y={end_y}){overflow}")

if __name__ == "__main__":
    main()

```

12.6 La guía de estilo

`references/style-guide.md` documenta la paleta, la tipografía, la estructura de cada slide y la regla anti-*overflow*. Su contenido completo está en el **Apéndice B**. Recuerda que los colores son una **aproximación de diseño** de «verde bosque + crema»: si tienes un valor oficial de marca, cámbialo en `carousel_gen.py` y en la guía.

13. Scripts de subida, skill de Claude y rutina semanal

13.1 La cadena de generación

Generar contenido es un trabajo para Claude; recibirlo y publicarlo, para la app. Entre los dos hay dos scripts pequeños y una *skill*.

- `url.txt` — la lista de páginas a promocionar, una por línea.
- `scripts/pick_urls.py` — elige N URLs **al azar** y distintas de `url.txt`. La elección no la hace Claude, sino este script, para que no siempre se elijan las mismas.
- `scripts/upload_draft.py` — sube una carpeta de imágenes y un `meta.yaml` como borrador a la app.
- La **skill** `carruseles-app` — las instrucciones que Claude sigue para generar, revisar y subir.

13.2 url.txt y pick_urls.py

`url.txt` tiene una URL por línea (las líneas que empiezan por `#` son comentarios):

`url.txt` – páginas a promocionar

```
https://www.trabajoenexcel.com/test-nivel-excel.html
https://www.trabajoenexcel.com/
https://www.trabajoenexcel.com/profesor-excel-online.html
https://www.trabajoenexcel.com/certificacion-microsoft-excel-especialista.html
https://www.trabajoenexcel.com/formacion-excel-empresas-fundae.html
https://www.trabajoenexcel.com/prueba-excel-entrevista-trabajo.html
https://www.trabajoenexcel.com/sobre-mi-jhony-escobar.html
```

`pick_urls.py` N imprime N URLs distintas y sin repetir (por defecto 3):

`scripts/pick_urls.py` – elegir URLs al azar

```
#!/usr/bin/env python3
"""Elige N URLs al azar (distintas entre sí) de url.txt.

Uso:  python scripts/pick_urls.py [N]      (por defecto N=3)
"""
import random
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent

def main() -> int:
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 3
    urls = list(dict.fromkeys(
        line.strip() for line in (ROOT / "url.txt").read_text(encoding="utf-8").splitlines()
        if line.strip() and not line.strip().startswith("#")))
    if not urls:
        print("url.txt no tiene ninguna URL.", file=sys.stderr)
        return 1
    for url in random.sample(urls, min(n, len(urls))):
        print(url)
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

13.3 upload_draft.py

Sube un borrador a `POST /api/drafts`. Detalles de funcionamiento:

- **Carpeta de entrada:** `slide-01.jpg`, `slide-02.jpg`... y un `meta.yaml` con `kind`, `title`, `caption`, `hashtags` (lista sin `#`) y `source_url`. Una historia **no** lleva `caption` ni `hashtags`.

- `--spec spec.json`: el JSON con el que se generaron las imágenes; se extrae su texto para subirlo como `slides_text`, de modo que el panel lo muestre y el servidor compruebe la regla de CTA también sobre el texto de las imágenes.
- **Configuración** (`IG_APP_URL` e `IG_APP_API_KEY`): de variables de entorno o de un archivo `.env` en la raíz (ignorado por Git). El script implementa su propio lector de `.env` (sin dependencias).
- **En la nube de Claude no hay clave local.** La clave se guarda como «credencial de API» del entorno y el proxy del entorno la añade a la petición. Por eso, si `IG_APP_API_KEY` no existe, el script simplemente **no envía cabecera** en vez de fallar.
- `--recent` lista los posts recientes (título, tipo, estado y URL): así Claude no repite temas.
- **Limpieza.** Tras subir con éxito, borra las imágenes locales y el `meta.yaml` (salvo `--keep`): **no se guardan imágenes en local** tras subir.
- **Códigos de salida:** 0 correcto; 1 si la app rechaza (muestra el motivo); 2 si falta configuración o archivos.

scripts/upload_draft.py – subida de borradores

```
#!/usr/bin/env python3
"""Sube un carrusel como BORRADOR a la app de aprobación.

Uso:
python scripts/upload_draft.py CARPETA [--spec spec.json] [--keep]
python scripts/upload_draft.py --recent      # lista los posts recientes (para no repetir temas)

CARPETA contiene slide-01.jpg, slide-02.jpg... y meta.yaml con:
  kind (carrusel | publicacion | historia; por defecto carrusel), title, caption,
  hashtags (lista sin #) y, opcional, source_url. Una historia no lleva caption ni hashtags.
--spec es el JSON con el que se generaron las slides; su texto se sube para que
se vea en el panel y se compruebe la restricción de CTA.

Configuración (variables de entorno o fichero .env en la raíz, ignorado por git):
IG_APP_URL      https://tu-app.onrender.com
IG_APP_API_KEY  la API_KEY de la app. Opcional en el entorno de nube de Claude, donde
                 se guarda como «credencial de API» y el proxy la añade a la petición.

Tras subirlo, las imágenes locales se borran (usa --keep para conservarlas).
"""
import argparse
import json
import os
import sys
from pathlib import Path

import requests
import yaml

ROOT = Path(__file__).resolve().parent.parent

def load_env() -> None:
    env_file = ROOT / ".env"
    if not env_file.exists():
        return
    for line in env_file.read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if line and not line.startswith("#") and "=" in line:
            key, value = line.split("=", 1)
            os.environ.setdefault(key.strip(), value.strip().strip("'").strip('"'))

def slides_text(spec_path: Path) -> list[str]:
    texts = []
    for slide in json.loads(spec_path.read_text(encoding="utf-8")):
        parts = [run[0] for run in slide["runs"]]
        if slide.get("cta_text"):
            parts.append(slide["cta_text"])
        texts.append(" ".join(parts))
    return texts

def main() -> int:
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser(description="Sube un borrador a la app de aprobación")
    parser.add_argument("folder", type=Path, nargs="?")
    parser.add_argument("--recent", action="store_true", help="listar los posts recientes y salir")
    parser.add_argument("--spec", type=Path, help="JSON de las slides (para mostrar su texto)")
```

... continuación

```

parser.add_argument("--keep", action="store_true", help="no borrar las imágenes locales tras subir")
args = parser.parse_args()

load_env()
base, key = os.environ.get("IG_APP_URL", "").rstrip("/"), os.environ.get("IG_APP_API_KEY", "")
if not base:
    print("Falta IG_APP_URL (variable de entorno o .env).", file=sys.stderr)
    return 2
# Sin clave local no se envía cabecera: en la nube la añade el proxy del entorno.
headers = {"Authorization": f"Bearer {key}"} if key else {}

if args.recent:
    resp = requests.get(f"{base}/api/posts", headers=headers, params={"limit": 40}, timeout=120)
    if resp.status_code == 401:
        print("[FALLO] 401: la app no acepta la petición. Falta IG_APP_API_KEY (o la credencial "
            "de API del entorno de nube no está bien configurada).", file=sys.stderr)
        return 1
    if resp.status_code != 200:
        print(f"[FALLO] {resp.status_code}: {resp.text[:200]}", file=sys.stderr)
        return 1
    for post in resp.json():
        print(f"{post['created_at'][:10]} {post['status'][:10]} {post.get('kind', 'carrusel')[:11]} "
            f"{post['title']} | {post.get('source_url') or ''}")
    return 0
if not args.folder:
    parser.error("indica CARPETA o usa --recent")

images = sorted(args.folder.glob("slide-*.jpg"))
meta_path = args.folder / "meta.yaml"
if not images or not meta_path.exists():
    print(f"{args.folder}: hacen falta slide-XX.jpg y meta.yaml.", file=sys.stderr)
    return 2
meta = yaml.safe_load(meta_path.read_text(encoding="utf-8")) or {}

data = {
    "kind": meta.get("kind", "carrusel"),
    "title": meta.get("title", ""),
    "caption": meta.get("caption", ""),
    "hashtags": json.dumps(meta.get("hashtags") or [], ensure_ascii=False),
    "source_url": meta.get("source_url", ""),
    "slides_text": json.dumps(slides_text(args.spec) if args.spec else [], ensure_ascii=False),
}
files = [{"slides", (p.name, p.read_bytes(), "image/jpeg")} for p in images]
resp = requests.post(f"{base}/api/drafts", headers=headers, data=data, files=files, timeout=180)
try:
    body = resp.json()
except ValueError:
    body = {"error": resp.text[:200]}
if resp.status_code != 201:
    print(f"[FALLO] {resp.status_code}: {body.get('error')}", file=sys.stderr)
    return 1

print(f"[SUBIDO] {len(images)} slide(s). Revísalo y apruébalo en: {body['url']}")
if not args.keep:
    for path in (*images, meta_path):
        path.unlink(missing_ok=True)
    try:
        args.folder.rmdir() # solo si ha quedado vacía
    except OSError:
        pass
    print("Imágenes locales borradas.")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

13.4 La skill de Claude (SKILL.md)

Una *skill* es una carpeta con un archivo `SKILL.md` (más scripts y referencias) que Claude carga cuando la tarea coincide con su descripción. La de este proyecto vive en `.claude/skills/carruseles-app/`:

Elemento	Función
<code>SKILL.md</code> — cabecera	<code>name</code> y una <code>description</code> larga con las frases que la activan («un carrusel», «una historia», «promociona esta URL...»)

Elemento	Función
<code>SKILL.md</code> — flujo	Los 7 pasos: definir objetivo y URL → sacar el ángulo → escribir el copy → generar imágenes → revisarlas → escribir <code>meta.yaml</code> → subir el borrador
<code>SKILL.md</code> — restricciones	La regla de CTA y la «autonomía total» en la creación (un único resultado, revisado por Claude)
<code>SKILL.md</code> — tanda semanal	El procedimiento de los lunes: 3 piezas, URLs sorteadas, ángulos distintos, sin aprobar ni publicar
<code>scripts/carousel_gen.py</code>	El generador (capítulo 12)
<code>assets/fonts/</code>	Las fuentes
<code>references/style-guide.md</code>	Paleta, tipografía y reglas de layout
<code>references/urls.md</code>	Para cada URL: el dolor del cliente, la palabra clave de SEO, la oferta y el CTA adaptado para Instagram

El contenido completo de `SKILL.md` está en el **Apéndice B**. Ideas clave de diseño:

- **La skill nunca aprueba ni publica.** Termina entregando el enlace del borrador. La aprobación es humana.
- **El copy sale de la página real** de cada URL (y de `urls.md`, ya verificado), no de la memoria: no se inventan cifras ni dolores genéricos.
- **La primera línea del texto de publicación es la palabra clave de SEO** de la URL (Instagram se indexa en buscadores). El gancho de curiosidad va en la primera imagen.
- **Hashtags:** 3 a 6, mezclando genéricos, de nicho y de intención de búsqueda.

13.5 La rutina semanal (los lunes)

Una **rutina programada de Claude en la nube** ejecuta cada lunes (en el proyecto, a las 08:02 hora de Madrid) el procedimiento de la «tanda semanal»: genera **tres borradores**, uno de cada tipo, y los sube. Recibes un resumen con los enlaces del panel y el recordatorio de aprobarlos antes de sus huecos.

Puntos a configurar en la rutina:

- **Entorno de ejecución** con la variable `IG_APP_URL` (la URL de la app) y una **credencial de API** con la clave de la app. **No** guardes la clave como variable de entorno del entorno: las variables las puede ver cualquiera que use ese entorno.
- **El texto de la tarea:** pedir «la tanda semanal» y recordar que no se aprueba ni se publica nada.
- **Sin conectores:** solo hace falta poder ejecutar Python y hacer peticiones HTTP a la app.

NOTA Se pueden crear rutinas desde la propia interfaz de Claude o, con Claude Code, con su función de programación de agentes. Consulta la documentación vigente de tu plan para el procedimiento exacto y sus límites.

CUIDADO En el entorno de la rutina, los dominios externos (incluida la web que promocionas) pueden estar **bloqueados**. En ese caso la rutina no puede leer las páginas y debe apoyarse en `references/urls.md`. Por eso conviene mantener ese archivo con los ángulos de **todas** las URLs de `url.txt`.

13.6 CLAUDE.md: las reglas del proyecto

Claude Code lee un archivo `CLAUDE.md` en la raíz como instrucciones permanentes del proyecto. El de este sistema fija, en pocas líneas: qué es el proyecto y cómo funciona; cuándo usar la skill; las reglas (CTA prohibido, **Claude no publica ni aprueba**, horarios solo los de la app, un post `failed` no se reintenta a ciegas, no se guardan imágenes, el repositorio es público y **no se escriben secretos**, límites de caption y hashtags, tipos e imágenes); los comandos habituales; y una nota de estado. Redáctalo pensando en que Claude lo leerá **antes de cada tarea**: cuanto más concretas las reglas, menos margen para errores.

14. Despliegue: Neon, Render y GitHub Actions

Orden recomendado: base de datos → app → programador → pruebas en producción.

14.1 Neon (Postgres)

1. Crea una cuenta en neon.tech y un proyecto (por ejemplo `ig-rrss`), en una región cercana a la de Render (por ejemplo Frankfurt).
2. En el panel del proyecto, pulsa **Connect** y copia la **cadena de conexión** (`postgresql://usuario:clave@ep-....neon.tech/neondb?sslmode=require`). Es tu `DATABASE_URL`.
3. No tienes que crear tablas: la app las crea sola al arrancar.

CUIDADO La cadena de conexión contiene la contraseña de la base de datos. Trátala como un secreto (solo en Render y, si la necesitas, en tu `.env`).

14.2 Render: el archivo `render.yaml`

Render puede crear el servicio a partir de un archivo de «plano» (*blueprint*) en la raíz del repositorio:

render.yaml – plano de despliegue

```
# Blueprint de Render: New + > Blueprint > este repositorio.
services:
- type: web
  name: ig-rrss
  runtime: python
  plan: free
  rootDir: app
  buildCommand: pip install -r requirements.txt
  # 1 worker: el estado de "publicando" y el límite de intentos de login viven en memoria
  startCommand: gunicorn server:app --workers 1 --threads 4 --timeout 120 --bind 0.0.0.0:$PORT
  healthCheckPath: /healthz
  envVars:
  - key: PYTHON_VERSION
    value: 3.12.7
  - key: SECRET_KEY
    generateValue: true
  - key: API_KEY
    generateValue: true
  - key: ADMIN_PASSWORD
    sync: false
  - key: DATABASE_URL
    sync: false
  - key: IG_USER_ID
    sync: false
  - key: IG_ACCESS_TOKEN
    sync: false
```

Clave	Significado
<code>type: web, runtime: python, plan: free</code>	Un servicio web de Python en el plan gratuito
<code>rootDir: app</code>	Todo se ejecuta desde la carpeta <code>app/</code>
<code>buildCommand</code>	<code>pip install -r requirements.txt</code>
<code>startCommand</code>	<code>gunicorn server:app --workers 1 --threads 4 --timeout 120 --bind 0.0.0.0:\$PORT</code> . Un solo worker (el estado de «publicando» y el límite de login viven en memoria), 4 hilos, y 120 s de margen por petición
<code>healthCheckPath: /healthz</code>	Render sabe que la app está viva si responde
<code>PYTHON_VERSION</code>	Versión de Python del servicio (3.12.7)

Clave	Significado
<code>SECRET_KEY</code> , <code>API_KEY</code> con <code>generateValue: true</code>	Render las genera una vez (y las conserva en los despliegues)
Variables con <code>sync: false</code>	Las rellenas tú a mano en el panel de Render: <code>ADMIN_PASSWORD</code> , <code>DATABASE_URL</code> , <code>IG_USER_ID</code> , <code>IG_ACCESS_TOKEN</code>

Pasos:

1. En dashboard.render.com → **New +** → **Blueprint**, conecta GitHub y elige el repositorio. Render lee `render.yaml`.
2. Rellena las cuatro variables `sync: false`: `DATABASE_URL` (Neon), `ADMIN_PASSWORD` (larga y única), `IG_USER_ID` y `IG_ACCESS_TOKEN`.
3. Pulsa **Apply**. Tras unos minutos, abre <https://<tu-servicio>.onrender.com/healthz>: debe decir **ok**. Después abre la raíz y entra con usuario `admin` y tu `ADMIN_PASSWORD`.

COMPRUEBA Si `/healthz` no responde, mira los registros (*Logs*) del servicio en Render. Los errores típicos: falta una variable, `DATABASE_URL` mal copiada, o una dependencia no instalada.

NOTA El plan gratuito de Render se duerme tras unos 15 minutos sin tráfico y tarda alrededor de un minuto en despertar. No es un problema aquí: los avisos del programador (cada 30 minutos) lo despiertan y el workflow **reintenta** mientras arranca. Su documentación desaconseja el plan gratuito para producción exigente; para esta carga es suficiente, pero conviene saberlo.

CUIDADO En el proyecto original, el **despliegue automático al hacer commit no se disparó**: hubo que pulsar *Manual Deploy* en Render. Después de cada `git push`, comprueba que la versión en línea cambió (por ejemplo, abriendo un archivo estático como `/static/style.css`). Una alternativa es usar el *Deploy Hook* de Render desde un workflow.

14.3 El programador: tick.yml

GitHub Actions ejecuta cada 30 minutos un workflow que **avisa a la app**. La app decide qué toca publicar.

.github/workflows/tick.yml – el programador

```
name: Programador de publicaciones

# Avisa a la app cada 30 minutos. La app decide qué toca publicar según la programación guardada en la app
# (panel → Programación)
# (un post por hueco y tipo: el más antiguo de la cola de ese tipo). GitHub Actions no
# despierta a su hora exacta: puede tardar unos minutos.
on:
  schedule:
    - cron: "* / 30 * * * *"
  workflow_dispatch:
    inputs:
      forzar:
        description: "PRUEBA: publicar YA el post aprobado más antiguo, sin esperar al hueco"
        type: boolean
        default: false
      tipo:
        description: "Solo con «forzar»: qué tipo publicar (cualquiera = el más antiguo de todos)"
        type: choice
        options: [cualquiera, carrusel, publicacion, historia]
        default: cualquiera

permissions:
  actions: write # para reactivar este workflow y que GitHub no lo desactive por inactividad

concurrency:
  group: tick
  cancel-in-progress: false
```

... continuación

```

jobs:
  tick:
    runs-on: ubuntu-latest
    timeout-minutes: 25
    steps:
      - name: Avisar a la app
        env:
          APP_URL: ${ secrets.APP_URL }
          APP_API_KEY: ${ secrets.APP_API_KEY }
          FORZAR: ${ inputs.forzar }
          TIPO: ${ inputs.tipo }
        run: |
          set -euo pipefail
          if [ -z "${APP_URL}" ] || [ -z "${APP_API_KEY}" ]; then
            echo "::warning::Programador sin configurar: faltan los secrets APP_URL y/o APP_API_KEY (ver
            ↪ app/README.md, paso 4)"
            exit 0
          fi
          base="${APP_URL%}/"
          url="${base}/api/cron/tick"
          if [ "${FORZAR:-false}" = "true" ]; then
            url="${url}?force=1"
            if [ -n "${TIPO:-}" ] && [ "${TIPO}" != "cualquiera" ]; then url="${url}&kind=${TIPO}"; fi
          fi

          # el plan gratuito de Render tarda ~1 min en despertar: se reintenta
          resp=$(curl -fsS --retry 5 --retry-delay 20 --retry-all-errors --max-time 180 \
            -X POST -H "Authorization: Bearer ${APP_API_KEY}" "$url")
          echo "$resp"
          ids=$(echo "$resp" | jq -r '.post_ids[]?')
          [ -n "$ids" ] || exit 0

          # hay publicaciones en curso: se espera a cada resultado para que un fallo avise por correo
          fail=0
          for id in $ids; do
            result=""
            for _ in $(seq 1 60); do
              sleep 10
              st=$(curl -fsS --retry 3 --max-time 60 -H "Authorization: Bearer ${APP_API_KEY}"
                ↪ "${base}/api/posts/${id}")
              status=$(echo "$st" | jq -r '.status')
              kind=$(echo "$st" | jq -r '.kind')
              echo "${kind} ${id}: ${status}"
              case "$status" in
                published) echo "Publicado (${kind}): $(echo "$st" | jq -r '.permalink')"; result=ok; break ;;
                failed) echo "::error::La publicación (${kind}) falló: $(echo "$st" | jq -r '.error')";
                  ↪ result=fail; break ;;
              esac
            done
            if [ "$result" != "ok" ]; then
              [ "$result" = "fail" ] || echo "::error::La publicación ${id} no terminó en 10 minutos"
              fail=1
            fi
          done
          exit $fail

      - name: Mantener activo el programador
        if: always()
        env:
          GH_TOKEN: ${ github.token }
        run: gh api -X PUT "repos/${GITHUB_REPOSITORY}/actions/workflows/tick.yml/enable" || true

```

Cómo funciona, paso a paso:

1. **Disparadores.** `schedule` (cron `*/30 * * * *`) y `workflow_dispatch` (ejecución manual, con las opciones `forzar` y `tipo`).
2. **Permisos y concurrencia.** `actions: write` para poder reactivarse a sí mismo; `concurrency` evita que dos ejecuciones se pisen.
3. **Comprobación de secretos.** Si faltan `APP_URL` o `APP_API_KEY`, muestra un aviso y termina **sin fallar** (así el workflow no da errores mientras aún no lo has configurado).
4. **El aviso.** Un `curl` a `POST /api/cron/tick` con la clave en la cabecera y **hasta 5 reintentos** (para cubrir el despertar de Render). Con `forzar` añade `?force=1` y, si se eligió tipo, `&kind=...`.
5. **Esperar el resultado.** Si la respuesta trae `post_ids`, consulta `GET /api/posts/<id>` cada 10 segundos (hasta 10 minutos) hasta ver `published` o `failed`. Si un post **falla**, el workflow termina en

rojo, y GitHub envía un correo: es tu aviso de que algo salió mal.

6. **Mantenerse activo.** GitHub **desactiva los workflows programados tras 60 días sin actividad** en el repositorio. El último paso (`if: always()`) llama a la API de GitHub para volver a habilitar el workflow en cada ejecución.

Crear los secrets (te pide el valor de cada uno):

```
gh secret set APP_URL --repo TU_USUARIO/IG_RRSS      # https://<tu-servicio>.onrender.com
gh secret set APP_API_KEY --repo TU_USUARIO/IG_RRSS  # el mismo valor que API_KEY en Render
```

CUIDADADO La ejecución con **forzar** publica **de verdad** el post aprobado más antiguo, fuera de hueco. Es solo una prueba manual del propietario de la cuenta: nunca la lances por automatismo ni «para probar» con contenido que no quieras publicar.

14.4 Conectar tu equipo

Para subir borradores desde tu ordenador, copia el valor de `API_KEY` desde Render (pestaña *Environment*) y crea en la raíz del repositorio un archivo `.env` (ignorado por Git):

```
IG_APP_URL=https://<tu-servicio>.onrender.com
IG_APP_API_KEY=<valor de API_KEY>
```

14.5 Pruebas en producción, paso a paso

1. **Salud.** `/healthz` responde `ok`; el formulario de login carga; una llamada a la API sin clave responde 401.
2. **Borrador.** Genera un borrador (o súbelo con `upload_draft.py`). Debe aparecer en «Por aprobar» con las imágenes y el texto.
3. **Cola.** Ábrelo, marca la casilla y pulsa «Aprobar y poner en cola». Pasa a «En cola» con la hora prevista («Sale: martes 22 sep, 15:30»).
4. **Programador sin publicar.** GitHub → Actions → *Programador de publicaciones* → *Run workflow* **sin** marcar «forzar». Fuera de un hueco responde `idle` y no publica. El pie del panel muestra «Último aviso del programador».
5. **Publicación real (una vez por tipo).** *Run workflow* marcando **forzar** (y eligiendo el tipo). **Publica ya** en la cuenta real: usa un post que quieras publicar de verdad. Prueba el carrusel, la publicación y la historia por separado.
6. **Verificar.** El post pasa a «Publicado» con el enlace a Instagram; su página dice que las imágenes se borraron; y `/media/<id>/0.jpg` responde 404.
7. **Programación normal.** A partir de ahí, lo aprobado sale solo en cada hueco. Vigila el **primer hueco real**: es la única prueba que no se puede hacer antes.

14.6 Actualizar la app

1. Cambia el código, ejecuta las pruebas (`cd app && python -m unittest`), haz *commit* y *push*.
2. Comprueba en Render que se desplegó la nueva versión (o pulsa *Manual Deploy*).
3. Las migraciones de base de datos se aplican **solas al arrancar**. Si el despliegue no llega a quedar «en línea», revisa los registros: una migración fallida impide arrancar.

15. Operación y mantenimiento

15.1 Una semana tipo

1. **Lunes ~08:02 (Madrid).** La rutina sube tres borradores (carrusel, publicación, historia) y te avisa con los enlaces.
2. **Antes del martes 15:30 y del jueves 19:00.** Revisas en el panel y apruebas. Lo que no apruebes a tiempo espera al **siguiente** hueco de su tipo.
3. **En cada hueco.** La app publica y borra las imágenes. Si algo falla, el workflow acaba en rojo (correo de GitHub) y el panel muestra el error.

15.2 Tareas habituales

Tarea	Cómo
Crear o quitar un usuario	Panel → Usuarios (solo administradores)
Cambiar días y horas de publicación	Panel → Programación (solo administradores). Vale desde el siguiente aviso
Cambiar tu contraseña	Panel → tu nombre arriba → Mi cuenta
Cambiar la contraseña de <code>admin</code>	Variable <code>ADMIN_PASSWORD</code> en Render y redespargar
Sacar un post de la cola	Detalle del post → «Sacar de la cola»
Cambiar el token de Instagram	Pega el nuevo en <code>IG_ACCESS_TOKEN</code> en Render: el valor nuevo manda sobre el guardado
Añadir una URL a promocionar	Añádela a <code>url.txt</code> y, idealmente, sus ángulos a <code>references/urls.md</code>

15.3 Rotar claves

Clave	Dónde está	Cómo cambiarla
<code>API_KEY</code>	Render, secret <code>APP_API_KEY</code> de GitHub, <code>.env</code> local, credencial de API de la rutina	Cambia los cuatro sitios a la vez, o el programador y la rutina dejarán de funcionar
<code>SECRET_KEY</code>	Render	Cámbiala en Render; todas las sesiones se cierran
<code>ADMIN_PASSWORD</code>	Render	Cámbiala y redespiega
Token de Instagram	Render (inicial) y base de datos (renovado)	Genera uno nuevo en Meta y pégalo en Render
<code>DATABASE_URL</code>	Render	Rota la contraseña en Neon y actualiza la variable

15.4 Qué mirar cuando algo va mal

- **El panel avisa solo.** Muestra avisos rojos si: la programación no es válida; hay posts aprobados de un tipo sin ningún hueco; el programador lleva más de 2 horas sin avisar y hay posts en cola.
- **Un post failed.** Su página muestra el error. **Antes de reabrir**, mira el perfil de Instagram por si se publicó.
- **Un post atascado en «Publicando»** más de 10 minutos: el proceso se interrumpió (por ejemplo, un reinicio del servidor). Aparece el botón «Marcar como fallido».
- **Registros.** Render → *Logs* (errores de la app); GitHub → *Actions* (cada aviso y su resultado).

15.5 Lecciones de incidentes reales

- **Secrets cruzados.** Se guardaron `IG_ACCESS_TOKEN` e `IG_USER_ID` intercambiados; el error de la API llevaba el token dentro y se escribió en un archivo público. *Lección:* los errores se redactan en la app (`redact`) y se revisan antes de subir cualquier cosa.
- `git add -A` **subió archivos locales** (permisos de Claude Code). *Lección:* añadir esos archivos a `.gitignore` y no usar `git add -A` a ciegas.
- **El auto-despliegue de Render no se disparaba.** Se detectó porque la versión en línea no cambiaba. *Lección:* verifica el despliegue con un archivo estático público.
- Un `IG_USER_ID` **mal copiado** hizo fallar la primera publicación real. *Lección:* de ahí nace la regla de mirar el perfil antes de reintentar un post fallido.

15.6 Copias de seguridad

En la base de datos solo hay **texto y estados** (las imágenes se borran a propósito) y los usuarios. Lo realmente irrecuperable es poco: los usuarios creados (se recrean) y el historial de posts. Revisa las opciones de recuperación que ofrece tu plan de Neon si te importa el historial; el **código** ya está en Git y la configuración en `render.yaml`.

16. Seguridad

16.1 Qué se protege y cómo

Riesgo	Defensa
Publicar algo no autorizado en la cuenta	Aprobación humana obligatoria; casilla de confirmación; solo aprueban usuarios con sesión
Robo de la clave de API	Comparación en tiempo constante; clave solo en variables y credenciales, nunca en el repositorio
Adivinar contraseñas	Hash con sal (scrypt), mínimo de 8 caracteres, límite de 5 intentos fallidos por IP cada 15 minutos, mensaje genérico, tiempo de respuesta uniforme
CSRF (formularios falsos)	Token por sesión en todos los <code>POST</code> ; cookies <code>SameSite=Lax</code>
XSS (código inyectado en textos)	Autoescapeado de Jinja2 y CSP sin JavaScript
Sesión robada por script	Cookie <code>HttpOnly</code> y <code>Secure</code>
Fijación de sesión	<code>session.clear()</code> al iniciar sesión
Redirección abierta	<code>next</code> solo si es una ruta interna
Exposición de imágenes	Solo públicas mientras el post está «Publicando»; UUID imposible de adivinar; borradas al publicar o rechazar
Fuga del token en errores	<code>redact()</code> antes de guardar o mostrar cualquier error
Contenido prohibido	<code>guards.py</code> al subir, aprobar y publicar
Acceso de un usuario eliminado	Comprobación del usuario en la base de datos en cada petición
Escalada de privilegios	<code>admin_required</code> en cada ruta de administración (ocultar enlaces no basta)
Secretos en el repositorio	<code>.gitignore</code> , credenciales de API para la nube y regla de no escribirlos nunca

16.2 Límites conocidos

- **El límite de intentos vive en memoria** y es **por IP**: se reinicia con cada arranque y no distingue usuarios. Es suficiente para un panel pequeño; para más, usa una tabla o un servicio externo.
- **No hay segundo factor (2FA)** ni recuperación de contraseña por correo.
- **El token de Instagram está en la base de datos en texto** (protegida por Neon, pero no cifrada a nivel de aplicación).
- **Una única clave de API compartida** por todos los clientes de la API.
- **Todos los usuarios ven todos los posts**: no hay separación por usuario.

16.3 Lista de comprobación antes de dar acceso a otra persona

- `ADMIN_PASSWORD` es larga y única, y no se ha compartido.
- `SECRET_KEY` y `API_KEY` están definidas (no vacías).
- La página funciona por **HTTPS**.
- Has probado que un editor **no** entra en `/admin/users` ni en `/admin/schedule` (debe dar 403).
- Has probado que un usuario eliminado pierde el acceso en el acto.
- `git log -p` no contiene tokens, claves ni contraseñas (y `.env` está en `.gitignore`).

17. Solución de problemas

Síntoma	Causa probable	Solución
El login responde «Servidor sin ADMIN_PASSWORD» (503)	Falta la variable	Define <code>ADMIN_PASSWORD</code> en Render y redespiega
«Usuario o contraseña incorrectos» con la contraseña buena	Bloqueo por 5 intentos fallidos, o usuario mal escrito	Espera 15 minutos o reinicia el servicio; recuerda que los usuarios son en minúsculas
«Formulario caducado» (400)	La sesión cambió (p. ej. reinicio con <code>SECRET_KEY</code> aleatoria)	Recarga la página. En producción define <code>SECRET_KEY</code>
Tras cada reinicio hay que volver a entrar	<code>SECRET_KEY</code> sin definir	Define <code>SECRET_KEY</code> fija
<code>/healthz</code> no responde	La app no arranca	Mira los registros de Render: variable ausente, <code>DATABASE_URL</code> mala o dependencia
El programador en rojo con «faltan los secrets»	No se han creado <code>APP_URL</code> y <code>APP_API_KEY</code>	Créalos con <code>gh secret set</code>
El programador responde 401	La clave del secret no coincide con <code>API_KEY</code> de Render	Copia de nuevo el valor exacto
Un post aprobado no sale en su hora	Aprobado tarde; el tipo no tiene hueco; el programador está parado; el hueco ya se usó	Comprueba los avisos del panel y la pantalla de Programación. Lo no aprobado a tiempo espera al siguiente hueco
El panel dice «El programador no se ha comunicado»	El workflow está desactivado o sin secrets	GitHub → Actions: reactívalo y revisa los secrets
Instagram devuelve 400 al publicar	<code>IG_USER_ID</code> o token equivocados, token caducado o permisos insuficientes	Revisa las variables (¿cruzadas?), regenera el token con los dos permisos
Instagram no descarga la imagen	La app estaba dormida o <code>PUBLIC_BASE_URL</code> /proxy mal configurados (URL <code>http</code>)	Comprueba que las URLs de <code>/media/...</code> salen con <code>https</code> y responden públicamente en «Publicando»
Un post queda en «Publicando» para siempre	Reinicio del servidor durante la publicación	«Marcar como fallido», mirar Instagram y, si no salió, reabrir
«No se pudo renovar el token»	Token caducado (más de 60 días sin renovar) o de menos de 24 h	Genera un token nuevo y pégalo en <code>IG_ACCESS_TOKEN</code>
El borrador se rechaza con 422	Imagen no JPEG, proporción errónea, texto largo o CTA prohibido	Lee el mensaje: dice qué regla incumple
Las pruebas fallan con «No module named PIL»	Falta Pillow	<code>pip install pillow</code>
Cambié <code>schedule.txt</code> y no pasa nada	Ese archivo solo se lee la primera vez	Edita los huecos en el panel (Programación)
Añadí un hueco y no publicó lo de «hoy»	Un hueco nuevo no rescata horas ya pasadas	Es intencionado; vale desde la siguiente ocurrencia

18. Ampliaciones y límites

18.1 Añadir un tipo de post nuevo

Por ejemplo, un tipo `reel`. Los puntos a tocar, en orden:

1. `schedule.py` — añadirlo a `KINDS`, `KIND_ALIASES` y `KIND_LABELS`.
2. `server.py` — añadirlo a `KIND_RULES` (imágenes, proporción, si lleva texto) y a las etiquetas `KIND`.
3. `instagram.py` — una rama nueva en `publish` con la llamada correspondiente de la API.
4. **Plantillas y CSS** — la etiqueta en el panel y, si cambia la proporción, sus estilos.
5. **Generador** — un formato nuevo en `LAYOUTS` (y en `FORMAT_THEME`).
6. `upload_draft.py`, **skill y guía** — que sepan del tipo nuevo.
7. **Pruebas** — reglas del tipo, cola y publicación.

18.2 Otras ideas

- **Despliegue automático** con el *Deploy Hook* de Render desde un workflow.
- **Recuperación de contraseña** por correo, o un botón «restablecer» para administradores.
- **Registro de auditoría** (quién hizo qué y cuándo) en una tabla.
- **Segundo factor** (TOTP) para administradores.
- **Varias cuentas de Instagram**, con un `IG_USER_ID` y un token por cuenta en la base de datos.
- **Estadísticas** del post publicado (alcance, guardados) leídas de la API de Instagram.
- **Notificaciones** (correo o mensajería) al publicar o fallar, además del correo de GitHub.
- **Permisos más finos**: por ejemplo, editores que solo pueden aprobar ciertos tipos.

18.3 Límites conocidos del diseño

- **Un solo worker** y estado en memoria: no escala horizontalmente sin rediseñar el límite de login y la publicación en hilo.
- **Plan gratuito de Render**: se duerme y su documentación lo desaconseja para producción exigente.
- **GitHub Actions no es un cron exacto**: hay retrasos de minutos; por eso existe la tolerancia.
- **La primera publicación real de cada tipo debe probarse** con contenido que quieras publicar, y el primer hueco programado real debe vigilarse.
- **El generador usa fuentes fijas** y un único estilo; cambiar la identidad visual implica tocar `THEMES` y `LAYOUTS`.

Apéndice A. Pruebas automáticas: código completo

Este es `app/test_server.py` completo (72 pruebas). Léelo como documentación ejecutable: cada método `test_...` describe en español un comportamiento del sistema.

app/test_server.py – pruebas automáticas

```

"""Pruebas de la app de aprobación (SQLite + Instagram simulado).

Ejecutar desde app/: python -m unittest -v
"""
import datetime as dt
import io
import json
import os
import sqlite3
import tempfile
import time
import unittest

_TMP = tempfile.mkdtemp()
_CARRUSEL_ONLY = os.path.join(_TMP, "solo_carrusel.txt")
with open(_CARRUSEL_ONLY, "w", encoding="utf-8") as _f:
    _f.write("martes 15:30 carrusel\njueves 19:00 carrusel\n")
os.environ.update(
    APP_ENV="dev", ADMIN_PASSWORD="clave-de-prueba", API_KEY="api-key-de-prueba",
    SECRET_KEY="secreto", SQLITE_PATH=os.path.join(_TMP, "test.db"), DATABASE_URL="",
    IG_USER_ID="1784140000000000", IG_ACCESS_TOKEN="IGAA-TOKEN-SECRETO-DE-PRUEBA",
)

import db # noqa: E402
import guards # noqa: E402
import instagram # noqa: E402
import schedule # noqa: E402
import server # noqa: E402
from PIL import Image # noqa: E402

API = {"Authorization": "Bearer api-key-de-prueba"}

# Huecos por defecto: martes 15:30 y jueves 19:00 (Europe/Madrid). En septiembre rige el
# horario de verano (UTC+2) y en diciembre el de invierno (UTC+1).
MON = "2026-09-21T10:00:00+00:00"
TUE_SLOT = "2026-09-22T13:30:00+00:00" # martes 15:30 en Madrid
TUE_LATER = "2026-09-22T13:45:00+00:00"
TUE_TOO_LATE = "2026-09-22T15:31:00+00:00" # 121 min después: fuera de tolerancia
THU_SLOT = "2026-09-24T17:00:00+00:00" # jueves 19:00 en Madrid
TUE_WINTER = "2026-12-01T14:30:00+00:00" # martes 15:30 en Madrid (invierno)

def at(iso: str) -> dt.datetime:
    return dt.datetime.fromisoformat(iso)

def jpeg(color="#F4EFE1", size=(108, 135)) -> bytes:
    buf = io.BytesIO()
    Image.new("RGB", size, color).save(buf, "JPEG")
    return buf.getvalue()

FEED = (108, 135) # 4:5
STORY = (108, 192) # 9:16

def draft_form(kind="carrusel", n=None, size=None, **over):
    """Formulario de subida válido para cada tipo (se puede forzar nº de imágenes y tamaño)."""
    data = {"title": "Prueba", "caption": "Excel para tu día a día.\nHaz el test gratis en trabajoenexcel.com.",
            "hashtags": json.dumps(["excel", "#productividad"]),
            "slides_text": json.dumps(["TU EQUIPO HACE SPRINTS. Pero no hace Scrum."])}
    if kind != "carrusel":
        data["kind"] = kind
    if kind == "historia":
        data["caption"], data["hashtags"] = "", []
    data.update(over)
    n = n or (2 if kind == "carrusel" else 1)
    size = size or (STORY if kind == "historia" else FEED)
    data["slides"] = [(io.BytesIO(jpeg("#F4EFE1", "#2F6B47", "#182A20")[i % 3], size)), f"slide-{i +
↳ 1:02d}.jpg")

```

... continuación

```

        for i in range(n)]

    return data

class Base(unittest.TestCase):
    def setUp(self):
        with db.connect() as conn:
            db.execute(conn, "DELETE FROM images")
            db.execute(conn, "DELETE FROM posts")
            db.execute(conn, "DELETE FROM kv")
            db.execute(conn, "DELETE FROM users")
            db.execute(conn, "DELETE FROM schedule_slots")
        server.attempts.clear() # el límite de intentos de login es global: que no pase de una prueba a otra
        self.client = server.app.test_client()
        self._publish, self._refresh, self._utcnow = instagram.publish, instagram.refresh, server.utcnow
        self._schedule_file = schedule.SCHEDULE_FILE # el de verdad (app/schedule.txt)
        schedule.SCHEDULE_FILE = _CARRUSEL_ONLY
        instagram.refresh = lambda token: (_ for _ in ()).throw(instagram.IGError("no toca"))

    def tearDown(self):
        instagram.publish, instagram.refresh, server.utcnow = self._publish, self._refresh, self._utcnow
        schedule.SCHEDULE_FILE = self._schedule_file

# ayudas
    def upload(self, kind="carrusel", **over):
        return self.client.post("/api/drafts", headers=API, data=draft_form(kind, **over),
                                content_type="multipart/form-data")

    def draft(self, kind="carrusel", **over) -> str:
        r = self.upload(kind, **over)
        self.assertEqual(r.status_code, 201, r.get_data(as_text=True))
        return r.get_json()["id"]

    def login(self, client=None):
        client = client or self.client
        client.get("/login")
        with client.session_transaction() as s:
            token = s["csrf"]
        return client.post("/login", data={"username": "admin", "password": "clave-de-prueba", "csrf": token})

    def csrf(self, client=None):
        """Token CSRF vigente: se genera al pintar cualquier página."""
        client = client or self.client
        client.get("/")
        with client.session_transaction() as s:
            return s["csrf"]

    def approve(self, pid, confirm=True):
        self.login()
        data = {"csrf": self.csrf()}
        if confirm:
            data["confirm"] = "on"
        return self.client.post(f"/post/{pid}/approve", data=data)

    def tick(self, when, force=False, headers=API, kind=None):
        server.utcnow = lambda: at(when)
        url = "/api/cron/tick" + ("?" + ("force=1" if force else "") + (f"&" if force else "?") + f"kind={kind}" if kind
        else "")
        return self.client.post(url, headers=headers)

    def status(self, pid):
        with db.connect() as conn:
            return db.one(conn, "SELECT * FROM posts WHERE id = %s", (pid,))

    def n_images(self, pid):
        with db.connect() as conn:
            return db.one(conn, "SELECT COUNT(*) AS n FROM images WHERE post_id = %s", (pid,))["n"]

    def wait_for(self, pid, wanted, timeout=5):
        end = time.time() + timeout
        while time.time() < end:
            if self.status(pid)["status"] in wanted:
                return
            time.sleep(0.05)
        self.fail(f"el post no llegó a {wanted}: {self.status(pid)['status']}")

class ApiTests(Base):
    def test_sin_api_key_401(self):
        r = self.client.post("/api/drafts", data=draft_form(), content_type="multipart/form-data")
        self.assertEqual(r.status_code, 401)
        r = self.client.post("/api/drafts", headers={"Authorization": "Bearer otra"}, data=draft_form(),

```

... continuación

```

        content_type="multipart/form-data")
self.assertEqual(r.status_code, 401)

def test_borrador_valido(self):
    r = self.upload()
    self.assertEqual(r.status_code, 201, r.get_data(as_text=True))
    pid = r.get_json()["id"]
    post = self.status(pid)
    self.assertEqual(post["status"], "draft")
    self.assertEqual(json.loads(post["hashtags"]), ["excel", "productividad"])
    self.assertEqual(self.n_images(pid), 2)

def test_rechaza_png_y_falsos_jpeg(self):
    buf = io.BytesIO()
    Image.new("RGB", (10, 10)).save(buf, "PNG")
    form = draft_form("publicacion")
    form["slides"] = [(io.BytesIO(buf.getvalue()), "slide-01.jpg")]
    r = self.client.post("/api/drafts", headers=API, data=form, content_type="multipart/form-data")
    self.assertEqual(r.status_code, 422)
    self.assertIn("JPEG", r.get_json()["error"])

def test_rechaza_cta_con_enlace_en_caption_y_slides(self):
    r = self.upload(caption="Aprende Excel.\nComenta NIVEL y te dejo el link.")
    self.assertEqual(r.status_code, 422)
    r = self.upload(slides_text=json.dumps(["COMENTA NIVEL", "y te paso el link"]))
    self.assertEqual(r.status_code, 422)
    with db.connect() as conn:
        self.assertEqual(db.one(conn, "SELECT COUNT(*) AS n FROM posts")["n"], 0)

def test_listado_reciente_exige_api_key_y_no_expone_texto(self):
    self.draft(title="Primero")
    self.draft(title="Segundo")
    self.assertEqual(self.client.get("/api/posts").status_code, 401)
    rows = self.client.get("/api/posts?limit=1", headers=API).get_json()
    self.assertEqual(len(rows), 1)
    self.assertEqual(set(rows[0]), {"id", "title", "status", "kind", "source_url", "created_at",
    ↵ "published_at"})
    self.assertEqual(self.client.get("/api/posts?limit=x", headers=API).status_code, 400)

def test_publicacion_e_historia(self):
    pub = self.upload("publicacion")
    self.assertEqual(pub.status_code, 201, pub.get_data(as_text=True))
    self.assertEqual(self.status(pub.get_json()["id"])[ "kind"], "publicacion")
    his = self.upload("historia")
    self.assertEqual(his.status_code, 201, his.get_data(as_text=True))
    post = self.status(his.get_json()["id"])
    self.assertEqual((post["kind"], post["caption"], post["hashtags"]), ("historia", "", []))

def test_historia_ignora_caption_y_hashtags(self):
    r = self.upload("historia", caption="texto que no se muestra", hashtags=json.dumps(["excel"]))
    self.assertEqual(r.status_code, 201)
    post = self.status(r.get_json()["id"])
    self.assertEqual((post["caption"], post["hashtags"]), ("", []))

def test_reglas_por_tipo(self):
    self.assertEqual(self.upload("carrusel", n=1).status_code, 422) # un carrusel lleva 2 o más
    self.assertEqual(self.upload("publicacion", n=2).status_code, 422) # una publicación, solo 1
    self.assertEqual(self.upload("historia", n=2).status_code, 422)
    self.assertEqual(self.upload("publicacion", caption="").status_code, 422) # la publicación exige caption
    self.assertEqual(self.upload("reel").status_code, 422) # tipo desconocido

def test_proporcion_de_imagen_por_tipo(self):
    r = self.upload("historia", size=FEED) # una imagen 4:5 no vale como historia
    self.assertEqual(r.status_code, 422)
    self.assertIn("9:16", r.get_json()["error"])
    self.assertEqual(self.upload("publicacion", size=STORY).status_code, 422) # 9:16 no vale en el feed
    self.assertEqual(self.upload("publicacion", size=(108, 108)).status_code, 201) # 1:1 sí
    self.assertEqual(self.upload("carrusel", size=(190, 100)).status_code, 201) # 1,9:1: dentro de 1,91
    self.assertEqual(self.upload("carrusel", size=(200, 100)).status_code, 422) # 2:1: fuera del máximo

def test_jpeg_size(self):
    self.assertEqual(server.jpeg_size(jpeg(size=(108, 192))), (108, 192))
    self.assertEqual(server.jpeg_size(jpeg(size=(1080, 1350))), (1080, 1350))
    self.assertIsNone(server.jpeg_size(b"\xff\xd8\xff\xd9"))

def test_limites(self):
    self.assertEqual(self.upload(caption="x" * 2300).status_code, 422)
    self.assertEqual(self.upload(hashtags=json.dumps([f"t{i}" for i in range(31)]))
    .status_code, 422)
    self.assertEqual(self.upload(hashtags=json.dumps(["con espacio"])).status_code, 422)
    self.assertEqual(self.upload(title="").status_code, 422)

```

... continuación

```

class AccessTests(Base):
    def test_panel_requiere_login(self):
        pid = self.draft()
        self.assertEqual(self.client.get("/").status_code, 302)
        self.assertEqual(self.client.get(f"/post/{pid}").status_code, 302)
        self.assertEqual(self.client.get(f"/media/{pid}/0.jpg").status_code, 404)

    def test_login(self):
        c = server.app.test_client()
        c.get("/login")
        with c.session_transaction() as s:
            token = s["csrf"]
        r = c.post("/login", data={"username": "admin", "password": "mala", "csrf": token})
        self.assertEqual(r.status_code, 200)
        self.assertEqual(c.get("/").status_code, 302)
        self.assertEqual(self.login(c).status_code, 302)
        self.assertEqual(c.get("/").status_code, 200)

    def test_login_sin_csrf_400(self):
        self.assertEqual(self.client.post("/login", data={"username": "admin", "password":
        ↵ "clave-de-prueba"}).status_code, 400)

    def test_redireccion_next_solo_relativa(self):
        c = server.app.test_client()
        c.get("/login")
        with c.session_transaction() as s:
            token = s["csrf"]
        r = c.post("/login?next=//evil.com", data={"username": "admin", "password": "clave-de-prueba", "csrf":
        ↵ token})
        self.assertEqual(r.headers["Location"], "/")

    def test_media_visible_con_sesion(self):
        pid = self.draft()
        self.login()
        r = self.client.get(f"/media/{pid}/0.jpg")
        self.assertEqual(r.status_code, 200)
        self.assertEqual(r.mimetype, "image/jpeg")

    def test_paginas_renderizan(self):
        pid = self.draft()
        self.login()
        html = self.client.get("/").get_data(as_text=True)
        self.assertIn("Por aprobar", html)
        self.assertIn(f"/media/{pid}/0.jpg", html)
        page = self.client.get(f"/post/{pid}").get_data(as_text=True)
        self.assertIn("Aprobar y poner en cola", page)
        self.assertIn("#productividad", page)

    def test_sin_csrf_400(self):
        pid = self.draft()
        self.login()
        self.assertEqual(self.client.post(f"/post/{pid}/approve", data={"confirm": "on"}).status_code, 400)
        self.assertEqual(self.client.post(f"/post/{pid}/reject", data={}).status_code, 400)

class ScheduleTests(unittest.TestCase):
    def setUp(self):
        self.cfg = schedule.load(os.path.join(os.path.dirname(os.path.abspath(__file__)), "schedule.txt"))

    def test_archivo_por_defecto(self):
        self.assertEqual(self.cfg.tz_name, "Europe/Madrid")
        self.assertEqual([(d, f"{t:%H:%M}", k) for d, t, k in self.cfg.slots],
        ↵ [(1, "15:30", "carrusel"), (3, "19:00", "historia"), (3, "19:00", "publicacion")])
        self.assertEqual(self.cfg.describe(),
        ↵ "martes 15:30 (carrusel), jueves 19:00 (historia), jueves 19:00 (publicación)")

    def test_hueco_vigente_verano(self):
        self.assertEqual(self.cfg.due_slots(at("2026-09-22T13:29:00+00:00")), {})
        self.assertEqual(self.cfg.due_slots(at(TUE_SLOT)), {"carrusel": at(TUE_SLOT)})
        self.assertEqual(self.cfg.due_slots(at("2026-09-22T15:29:00+00:00")), {"carrusel": at(TUE_SLOT)})
        self.assertEqual(self.cfg.due_slots(at(TUE_TOO_LATE)), {})
        self.assertEqual(self.cfg.due_slots(at(THU_SLOT)),
        ↵ {"publicacion": at(THU_SLOT), "historia": at(THU_SLOT)})
        self.assertEqual(self.cfg.due_slots(at(MON)), {})

    def test_cambio_de_hora_invierno(self):
        self.assertEqual(self.cfg.due_slots(at(TUE_WINTER)), {"carrusel": at(TUE_WINTER)})
        self.assertEqual(self.cfg.due_slots(at("2026-12-01T13:30:00+00:00")), {}) # 14:30 en Madrid: aún no

    def test_proximos_huecos_por_tipo(self):
        now = at("2026-09-22T10:00:00+00:00")
        self.assertEqual(self.cfg.upcoming(now, None, 3, "carrusel"),

```

... continuación

```

        [at(TUE_SLOT), at("2026-09-29T13:30:00+00:00"), at("2026-10-06T13:30:00+00:00")])
self.assertEqual(self.cfg.upcoming(now, at(TUE_SLOT), 1, "carrusel"), [at("2026-09-29T13:30:00+00:00")])
self.assertEqual(self.cfg.upcoming(now, None, 2, "historia"), [at(THU_SLOT),
↳ at("2026-10-01T17:00:00+00:00")])
self.assertEqual(self.cfg.label(at(TUE_SLOT)), "martes 22 sep, 15:30")

def test_errores_de_formato(self):
    for text in ["lunes 25:00 carrusel", "foo 10:00 carrusel", "martes 10:00", "martes carrusel",
                "martes 10:00 reel", "zona: Marte/Olimpo\nmartes 10:00 carrusel",
                "tolerancia_minutos: mucha\nmartes 10:00 carrusel", "# solo comentarios"]:
        with self.assertRaises(schedule.ScheduleError, msg=text):
            schedule.parse(text)

def test_acentos_y_varios_huecos(self):
    cfg = schedule.parse("miércoles 9:05 publicación\nmiercoles 18:00 historia\nsábado 10:00 carrusel")
    self.assertEqual(len(cfg.slots), 3)
    self.assertEqual(cfg.kinds(), {"publicacion", "historia", "carrusel"})

class QueueTests(Base):
    """Aprobar pone en cola; publica el aviso del programador en su hueco."""

    def fake_publish(self, calls=None):
        def fake(urls, caption, user_id, token, **kw):
            if calls is not None:
                calls.append(urls)
            return {"media_id": "123", "permalink": "https://www.instagram.com/p/XYZ/"}
        instagram.publish = fake

    def test_aprobar_no_publica_al_momento(self):
        pid = self.draft()
        calls = []
        self.fake_publish(calls)
        self.approve(pid)
        time.sleep(0.2)
        self.assertEqual(self.status(pid)["status"], "approved")
        self.assertIsNotNone(self.status(pid)["approved_at"])
        self.assertFalse(calls)

    def test_sin_confirmar_no_se_encola(self):
        pid = self.draft()
        self.approve(pid, confirm=False)
        self.assertEqual(self.status(pid)["status"], "draft")

    def test_un_post_por_hueco_por_orden_de_aprobacion(self):
        a, b = self.draft(title="A"), self.draft(title="B")
        calls = []
        self.fake_publish(calls)
        self.approve(a)
        self.approve(b)
        self.assertEqual(self.tick(MON).get_json()["action"], "idle")
        r = self.tick(TUE_SLOT).get_json()
        self.assertEqual(r["action"], r["post_id"], ("publishing", a))
        self.wait_for(a, {"published"})
        self.assertEqual(self.status(b)["status"], "approved")
        # mismo hueco otra vez (el cron avisa cada 30 min): no publica nada más
        self.assertEqual(self.tick(TUE_LATER).get_json()["action"], "slot-used")
        self.assertEqual(self.status(b)["status"], "approved")
        # siguiente hueco: le toca a B
        self.assertEqual(self.tick(THU_SLOT).get_json()["post_id"], b)
        self.wait_for(b, {"published"})
        self.assertEqual(len(calls), 2)

    def test_publica_borra_imagenes_y_las_expone_solo_mientras_publica(self):
        pid = self.draft()
        seen = {}

        def fake_publish(urls, caption, user_id, token, **kw):
            anon = server.app.test_client() # sin sesión, como Instagram
            seen["anon"] = [anon.get(u.replace("http://localhost", "")).status_code for u in urls]
            seen["urls"], seen["caption"], seen["token"] = urls, caption, token
            return {"media_id": "123", "permalink": "https://www.instagram.com/p/XYZ/"}

        instagram.publish = fake_publish
        self.approve(pid)
        # en cola las imágenes NO son públicas
        self.assertEqual(server.app.test_client().get(f"/media/{pid}/0.jpg").status_code, 404)
        self.tick(TUE_SLOT)
        self.wait_for(pid, {"published"})
        post = self.status(pid)
        self.assertEqual(post["permalink"], "https://www.instagram.com/p/XYZ/")
        self.assertEqual(seen["anon"], [200, 200], "públicas mientras se publica")

```

... continuación

```

self.assertTrue(seen["caption"].endswith("#excel #productividad"))
self.assertEqual(seen["token"], "IGAA-TOKEN-SECRETO-DE-PRUEBA")
self.assertEqual(self.n_images(pid), 0, "no se conserva ningún JPG")
self.assertEqual(server.app.test_client().get(f"/media/{pid}/0.jpg").status_code, 404)
self.assertEqual(self.client.get(f"/media/{pid}/0.jpg").status_code, 404)

def test_hueco_sin_posts_se_pierde(self):
    self.fake_publish()
    self.assertEqual(self.tick(TUE_SLOT).get_json()["action"], "empty-queue")
    pid = self.draft()
    self.approve(pid) # aprobado DESPUÉS de que pasara el aviso de ese hueco
    self.assertEqual(self.tick(TUE_LATER).get_json()["action"], "slot-used")
    self.assertEqual(self.status(pid)["status"], "approved")
    self.assertEqual(self.tick(THU_SLOT).get_json()["post_id"], pid)

def test_fuera_de_tolerancia_no_publica(self):
    pid = self.draft()
    self.fake_publish()
    self.approve(pid)
    self.assertEqual(self.tick(TUE_TOO_LATE).get_json()["action"], "idle")
    self.assertEqual(self.status(pid)["status"], "approved")

def test_tick_exige_api_key(self):
    self.assertEqual(self.tick(TUE_SLOT, headers={}).status_code, 401)
    self.assertEqual(self.tick(TUE_SLOT, headers={"Authorization": "Bearer otra"}).status_code, 401)

def test_force_publica_sin_gastar_hueco(self):
    a, b = self.draft(title="A"), self.draft(title="B")
    self.fake_publish()
    self.approve(a)
    self.approve(b)
    self.assertEqual(self.tick(MON, force=True).get_json()["post_id"], a)
    self.wait_for(a, {"published"})
    self.assertEqual(self.tick(TUE_SLOT).get_json()["post_id"], b) # el hueco seguía libre
    self.wait_for(b, {"published"})

def test_fallo_conserva_imagenes_y_oculta_credenciales(self):
    pid = self.draft()

    def failing(urls, caption, user_id, token, **kw):
        raise instagram.IGError(f"API 400 en {user_id}/media con {token}: Invalid OAuth access token")

    instagram.publish = failing
    self.approve(pid)
    self.tick(TUE_SLOT)
    self.wait_for(pid, {"failed"})
    post = self.status(pid)
    self.assertNotIn("IGAA-TOKEN", post["error"])
    self.assertNotIn("1784140000000000", post["error"])
    self.assertIn("****", post["error"])
    self.assertEqual(self.n_images(pid), 2)
    self.client.post(f"/post/{pid}/reopen", data={"csrf": self.csrf()})
    self.assertEqual(self.status(pid)["status"], "draft")

def test_contenido_prohibido_no_llega_a_publicarse(self):
    pid = self.draft()
    self.approve(pid)
    with db.connect() as conn: # simula un registro manipulado tras aprobar
        db.execute(conn, "UPDATE posts SET caption = %s WHERE id = %s", ("Comenta EXCEL y te dejo el link",
            pid))
    called = []
    instagram.publish = lambda *a, **k: called.append(1)
    self.tick(TUE_SLOT)
    self.wait_for(pid, {"failed"})
    self.assertFalse(called)
    self.assertIn("restricción de CTA", self.status(pid)["error"])

def test_aprobar_dos_veces_encola_una_vez(self):
    pid = self.draft()
    calls = []
    self.fake_publish(calls)
    self.approve(pid)
    self.approve(pid)
    self.tick(TUE_SLOT)
    self.wait_for(pid, {"published"})
    time.sleep(0.2)
    self.assertEqual(len(calls), 1)

def test_sacar_de_la_colas_y_rechazar(self):
    pid = self.draft()
    self.approve(pid)
    self.client.post(f"/post/{pid}/unqueue", data={"csrf": self.csrf()})

```

... continuación

```

        self.assertEqual(self.status(pid)["status"], "draft")
        self.assertIsNone(self.status(pid)["approved_at"])
        self.approve(pid)
        self.client.post(f"/post/{pid}/reject", data={"csrf": self.csrf()})
        self.assertEqual(self.status(pid)["status"], "rejected")
        self.assertEqual(self.n_images(pid), 0)

    def test_cta_prohibido_no_se_aprueba(self):
        pid = self.draft()
        with db.connect() as conn:
            db.execute(conn, "UPDATE posts SET caption = %s WHERE id = %s", ("Comenta EXCEL y te dejo el link",
                                                                              pid))
        self.approve(pid)
        self.assertEqual(self.status(pid)["status"], "draft")

    def test_panel_muestraCola_hora_prevista_y_avisos(self):
        pid = self.draft()
        self.approve(pid)
        server.utcnow = lambda: at("2026-09-22T10:00:00+00:00")
        html = self.client.get("/").get_data(as_text=True)
        self.assertIn("En cola", html)
        self.assertIn("Sale: martes 22 sep, 15:30", html)
        self.assertIn("El programador no se ha comunicado", html) # nunca ha habido un aviso
        self.tick("2026-09-22T10:00:00+00:00")
        html = self.client.get("/").get_data(as_text=True)
        self.assertNotIn("El programador no se ha comunicado", html)
        self.assertIn("martes 15:30 (carrusel), jueves 19:00 (carrusel)", html)
        page = self.client.get(f"/post/{pid}").get_data(as_text=True)
        self.assertIn("Sacar de la cola", page)

    def test_programacion_invalida_devuelve_error_y_lo_muestra(self):
        bad = os.path.join(_TMP, "malo.txt")
        with open(bad, "w", encoding="utf-8") as f:
            f.write("martes 99:99\n")
        schedule.SCHEDULE_FILE = bad
        r = self.tick(TUE_SLOT)
        self.assertEqual(r.status_code, 500)
        self.login()
        self.assertIn("no es válida", self.client.get("/").get_data(as_text=True))

class KindQueueTests(Base):
    """Un hueco por tipo (schedule.txt real): carrusel el martes; publicación e historia el jueves."""

    def setUp(self):
        super().setUp()
        schedule.SCHEDULE_FILE = self._schedule_file
        self.calls = []

        def fake(urls, caption, user_id, token, kind="carrusel"):
            self.calls.append({"kind": kind, "n": len(urls), "caption": caption})
            return {"media_id": kind, "permalink": f"https://www.instagram.com/p/{kind}/"}

        instagram.publish = fake

    def test_publicacion_e_historia_salen_juntas_el_jueves(self):
        car, pub, his = self.draft("carrusel"), self.draft("publicacion"), self.draft("historia")
        for pid in (car, pub, his):
            self.approve(pid)
        self.assertEqual(self.tick(MON).get_json()["action"], "idle")
        r = self.tick(THU_SLOT).get_json()
        self.assertEqual(r["action"], "publishing")
        self.assertEqual([p["kind"] for p in r["posts"]], {"publicacion", "historia"})
        self.wait_for(pub, {"published"})
        self.wait_for(his, {"published"})
        self.assertEqual(self.status(car)["status"], "approved") # el carrusel espera a su hueco del martes
        by_kind = {c["kind"]: c for c in self.calls}
        self.assertEqual(by_kind["historia"]["caption"], "") # las historias no llevan texto
        self.assertEqual((by_kind["historia"]["n"], by_kind["publicacion"]["n"]), (1, 1))
        self.assertIn("#excel", by_kind["publicacion"]["caption"])
        for pid in (pub, his):
            self.assertEqual(self.n_images(pid), 0) # se borran las imágenes de ambos
        # la semana siguiente le toca al carrusel
        self.assertEqual(self.tick("2026-09-29T13:30:00+00:00").get_json()["post_id"], car)
        self.wait_for(car, {"published"})
        self.assertEqual([c["kind"] for c in self.calls].count("carrusel"), 1)

    def test_cada_tipo_tiene_suCola_y_su_hueco(self):
        h1, h2 = self.draft("historia", title="H1"), self.draft("historia", title="H2")
        self.approve(h1)
        self.approve(h2)
        self.assertEqual(self.tick(THU_SLOT).get_json()["post_id"], h1) # solo una historia por hueco

```

... continuación

```

        self.wait_for(h1, {"published"})
        self.assertEqual(self.tick("2026-09-24T17:30:00+00:00").get_json()["action"], "slot-used")
        self.assertEqual(self.status(h2)["status"], "approved")

    def test_hueco_de_un_tipo_sin_posts_no_afecta_al_otro(self):
        pub = self.draft("publicacion")
        self.approve(pub)  # no hay historia aprobada
        r = self.tick(THU_SLOT).get_json()
        self.assertEqual([p["kind"] for p in r["posts"]], ["publicacion"])
        self.wait_for(pub, {"published"})
        his = self.draft("historia")
        self.approve(his)  # tarde: su hueco de esta semana ya pasó
        self.assertEqual(self.tick("2026-09-24T17:40:00+00:00").get_json()["action"], "slot-used")
        self.assertEqual(self.status(his)["status"], "approved")

    def test_forzar_un_tipo_concreto(self):
        car, his = self.draft("carrusel"), self.draft("historia")
        self.approve(car)
        self.approve(his)
        self.assertEqual(self.tick(MON, force=True, kind="historia").get_json()["post_id"], his)
        self.wait_for(his, {"published"})
        self.assertEqual(self.status(car)["status"], "approved")
        self.assertEqual(self.tick(MON, force=True, kind="reel").status_code, 400)

    def test_un_fallo_no_impide_publicar_el_otro_tipo(self):
        pub, his = self.draft("publicacion"), self.draft("historia")
        self.approve(pub)
        self.approve(his)

        def fake(urls, caption, user_id, token, kind="carrusel"):
            if kind == "publicacion":
                raise instagram.IGError("API 400: fallo simulado")
            return {"media_id": "1", "permalink": ""}

        instagram.publish = fake
        self.tick(THU_SLOT)
        self.wait_for(pub, {"failed"})
        self.wait_for(his, {"published"})

    def test_panel_hora_prevista_por_tipo_y_avisos_sin_hueco(self):
        car, his = self.draft("carrusel"), self.draft("historia")
        self.approve(car)
        self.approve(his)
        server.utcnow = lambda: at("2026-09-21T10:00:00+00:00")
        html = self.client.get("/").get_data(as_text=True)
        self.assertIn("Sale: martes 22 sep, 15:30", html)
        self.assertIn("Sale: jueves 24 sep, 19:00", html)
        self.assertIn("Historia", html)
        page = self.client.get(f"/post/{his}").get_data(as_text=True)
        self.assertIn("no llevan texto de publicación", page)
        # un tipo aprobado sin hueco definido se avisa
        with db.connect() as conn:
            db.execute(conn, "DELETE FROM schedule_slots WHERE kind = 'historia'")
        self.assertIn("sin ningún hueco en la programación", self.client.get("/").get_data(as_text=True))

    def test_migracion_de_una_base_sin_columna_kind(self):
        old = os.path.join(_TMP, "vieja.db")
        con = sqlite3.connect(old)
        con.execute("CREATE TABLE posts (id TEXT PRIMARY KEY, title TEXT NOT NULL, status TEXT NOT NULL, "
            "caption TEXT NOT NULL, hashtags TEXT NOT NULL, slides_text TEXT NOT NULL, source_url TEXT, "
            "created_at TEXT NOT NULL, updated_at TEXT NOT NULL, approved_at TEXT, published_at TEXT, "
            "media_id TEXT, permalink TEXT, error TEXT)")
        con.execute("INSERT INTO posts (id, title, status, caption, hashtags, slides_text, created_at, "
            "updated_at) "
            "VALUES ('x', 'Viejo', 'published', 'c', '[]', '[]', 'a', 'b')")
        con.commit()
        con.close()
        previous = db.SQLITE_PATH
        db.SQLITE_PATH = old
        try:
            db.init_db()
            db.init_db()  # es idempotente
            with db.connect() as conn:
                self.assertEqual(db.one(conn, "SELECT kind FROM posts WHERE id = 'x'")["kind"], "carrusel")
        finally:
            db.SQLITE_PATH = previous

class UserTests(Base):
    """Varios usuarios: alta, baja, roles y acceso."""

    def post(self, client, path, **data):

```

... continuación

```

        client.get("/login")
        with client.session_transaction() as s:
            token = s["csrf"]
        return client.post(path, data={"csrf": token, **data})

def as_admin(self):
    self.login()
    return self.client

def create(self, name="ana", password="clave-larga-1", role="editor"):
    r = self.post(self.as_admin(), "/admin/users", username=name, password=password, role=role)
    self.assertEqual(r.status_code, 302)
    with db.connect() as conn:
        return db.one(conn, "SELECT * FROM users WHERE username = %s", (name.lower(),))

def login_as(self, name, password="clave-larga-1"):
    c = server.app.test_client()
    r = self.post(c, "/login", username=name, password=password)
    return c, r

def test_crear_usuario_y_entrar(self):
    row = self.create("Ana") # el nombre se guarda en minúsculas
    self.assertEqual(row["username"], "ana")
    self.assertNotIn("clave-larga-1", row["password_hash"]) # cifrada, nunca en claro
    c, r = self.login_as("ANA")
    self.assertEqual(r.status_code, 302)
    self.assertEqual(c.get("/").status_code, 200)

def test_credenciales_incorrectas(self):
    self.create("ana")
    for user, pw in (("ana", "otra-clave-x"), ("nadie", "clave-larga-1"), ("ana", "")):
        c, r = self.login_as(user, pw)
        self.assertEqual(r.status_code, 200, (user, pw))
        self.assertEqual(c.get("/").status_code, 302)

def test_eliminar_usuario_le_quita_el_acceso_al_instante(self):
    row = self.create("ana")
    c, _ = self.login_as("ana")
    self.assertEqual(c.get("/").status_code, 200)
    r = self.post(self.client, f"/admin/users/{row['id']}/delete")
    self.assertEqual(r.status_code, 302)
    self.assertEqual(c.get("/").status_code, 302) # su sesión ya no vale
    c2, _ = self.login_as("ana")
    self.assertEqual(c2.get("/").status_code, 302) # ni puede volver a entrar

def test_no_se_puede_eliminar_la_propia_cuenta(self):
    row = self.create("ana", role="admin")
    c, _ = self.login_as("ana")
    self.post(c, f"/admin/users/{row['id']}/delete")
    self.assertEqual(c.get("/admin/users").status_code, 200)

def test_editor_no_gestiona_usuarios_ni_programacion(self):
    self.create("ana", role="editor")
    c, _ = self.login_as("ana")
    for path in ("/admin/users", "/admin/schedule"):
        self.assertEqual(c.get(path).status_code, 403, path)
    self.assertEqual(self.post(c, "/admin/users", username="otro", password="clave-larga-1",
        role="admin").status_code, 403)
    self.assertEqual(self.post(c, "/admin/schedule/slots", day="0", time="10:00",
        kind="carrusel").status_code, 403)
    html = c.get("/").get_data(as_text=True)
    self.assertNotIn("/admin/users", html)
    self.assertEqual(c.get("/").status_code, 200) # pero sí puede usar el panel

def test_admin_de_la_base_de_datos_gestiona_usuarios(self):
    self.create("jefe", role="admin")
    c, _ = self.login_as("jefe")
    self.assertEqual(c.get("/admin/users").status_code, 200)
    self.assertEqual(c.get("/admin/schedule").status_code, 200)

def test_validaciones_de_alta(self):
    admin = self.as_admin()
    for name, pw, role in (("ab", "clave-larga-1", "editor"), ("con espacio", "clave-larga-1", "editor"),
        ("admin", "clave-larga-1", "editor"), ("ana", "corta", "editor"),
        ("ana", "clave-larga-1", "superhéroe")):
        self.post(admin, "/admin/users", username=name, password=pw, role=role)
    with db.connect() as conn:
        self.assertEqual(db.query(conn, "SELECT * FROM users"), [])
    self.create("ana")
    self.post(admin, "/admin/users", username="ANA", password="otra-clave-larga", role="editor")
    with db.connect() as conn:
        self.assertEqual(len(db.query(conn, "SELECT * FROM users")), 1) # sin duplicados

```

... continuación

```

def test_cambiar_contrasena_propia(self):
    self.create("ana")
    c, _ = self.login_as("ana")
    r = self.post(c, "/account", current_password="incorrecta", new_password="nueva-clave-99",
    ↪ again="nueva-clave-99")
    self.assertEqual(r.status_code, 302)
    self.assertEqual(self.login_as("ana", "nueva-clave-99")[0].get("/").status_code, 302) # no cambió
    self.post(c, "/account", current_password="clave-larga-1", new_password="nueva-clave-99",
    ↪ again="distinta-99999")
    self.assertEqual(self.login_as("ana", "nueva-clave-99")[0].get("/").status_code, 302)
    self.post(c, "/account", current_password="clave-larga-1", new_password="nueva-clave-99",
    ↪ again="nueva-clave-99")
    self.assertEqual(self.login_as("ana", "nueva-clave-99")[0].get("/").status_code, 200)
    self.assertEqual(self.login_as("ana", "clave-larga-1")[0].get("/").status_code, 302)

def test_cuenta_integrada_no_cambia_contrasena_desde_el_panel(self):
    self.login()
    self.assertIn("ADMIN_PASSWORD", self.client.get("/account").get_data(as_text=True))

def test_usuarios_sin_csrf_400(self):
    self.login()
    self.assertEqual(self.client.post("/admin/users", data={"username": "ana"}).status_code, 400)
    self.assertEqual(self.client.post("/admin/schedule/slots", data={}).status_code, 400)

def test_sesion_antigua_sin_usuario_no_vale(self):
    with self.client.session_transaction() as s:
        s["auth"] = True # formato de sesión anterior a los usuarios
    self.assertEqual(self.client.get("/").status_code, 302)

def test_quien_aprueba_queda_registrado(self):
    self.create("ana")
    pid = self.draft()
    c, _ = self.login_as("ana")
    c.get("/")
    with c.session_transaction() as s:
        token = s["csrf"]
    c.post(f"/post/{pid}/approve", data={"csrf": token, "confirm": "on"})
    self.assertEqual(self.status(pid)["status"], "approved")
    with db.connect() as conn:
        self.assertEqual(db.one(conn, "SELECT approved_by FROM posts WHERE id = %s", (pid,))["approved_by"],
        ↪ "ana")
    self.assertIn("Aprobado por ana", c.get(f"/post/{pid}").get_data(as_text=True))

class ScheduleAdminTests(Base):
    """Huecos de publicación editables desde el panel."""

    def setUp(self):
        super().setUp()
        schedule.SCHEDULE_FILE = self._schedule_file # el de verdad: sirve de programación inicial

    def slots(self):
        with db.connect() as conn:
            return [(r["weekday"], r["slot_time"], r["kind"]) for r in db.query(
                conn, "SELECT * FROM schedule_slots ORDER BY weekday, slot_time, kind")]

    def act(self, path, **data):
        self.login()
        return self.client.post(path, data={"csrf": self.csrf(), **data})

    def test_primera_carga_copia_schedule_txt(self):
        self.login()
        html = self.client.get("/admin/schedule").get_data(as_text=True)
        self.assertEqual(self.slots(), [(1, "15:30", "carrusel"), (3, "19:00", "historia"), (3, "19:00",
        ↪ "publicacion")])
        for text in ("Martes", "15:30", "Jueves", "19:00", "Europe/Madrid"):
            self.assertIn(text, html)

    def test_manda_la_base_de_datos_no_el_archivo(self):
        self.login()
        self.client.get("/admin/schedule")
        bad = os.path.join(_TMP, "ignorado.txt")
        with open(bad, "w", encoding="utf-8") as f:
            f.write("esto ni es un horario\n")
        schedule.SCHEDULE_FILE = bad
        self.assertEqual(self.client.get("/admin/schedule").status_code, 200)
        self.assertEqual(self.tick(MON).get_json()["action"], "idle")

    def test_anadir_y_eliminar_hueco(self):
        self.act("/admin/schedule/slots", day="4", time="9:05", kind="carrusel")
        self.assertIn((4, "09:05", "carrusel"), self.slots())

```

... continuación

```

self.act("/admin/schedule/slots", day="4", time="09:05", kind="carrusel") # duplicado
self.assertEqual(self.slots().count((4, "09:05", "carrusel")), 1)
with db.connect() as conn:
    sid = db.one(conn, "SELECT id FROM schedule_slots WHERE weekday = 4")["id"]
self.act(f"/admin/schedule/slots/{sid}/delete")
self.assertNotIn((4, "09:05", "carrusel"), self.slots())

def test_rechaza_huecos_no_validos(self):
    self.login()
    self.client.get("/admin/schedule") # carga inicial
    before = self.slots()
    self.assertEqual(len(before), 3)
    for day, time_, kind in (("9", "10:00", "carrusel"), ("x", "10:00", "carrusel"), ("1", "25:00",
    ↵ "carrusel"),
                                ("1", "10:99", "carrusel"), ("1", "", "carrusel"), ("1", "10:00", "reel")):
        self.act("/admin/schedule/slots", day=day, time=time_, kind=kind)
    self.assertEqual(self.slots(), before)

def test_el_hueco_nuevo_publica_en_el_tick(self):
    pid = self.draft("carrusel")
    self.approve(pid)
    self.act("/admin/schedule/slots", day="2", time="12:00", kind="carrusel") # miércoles 12:00
    instagram.publish = lambda urls, caption, user_id, token, **kw: {
        "media_id": "1", "permalink": "https://www.instagram.com/p/A/"
    }
    # creado a las 10:05 UTC, cuando las 12:00 de Madrid (10:00 UTC) ya habían pasado: no cuenta
    with db.connect() as conn:
        db.execute(conn, "UPDATE schedule_slots SET created_at = '2026-09-23T10:05:00+00:00' WHERE weekday =
        ↵ 2")
    self.assertEqual(self.tick("2026-09-23T10:10:00+00:00").get_json()["action"], "idle")
    with db.connect() as conn:
        db.execute(conn, "UPDATE schedule_slots SET created_at = '2026-09-22T09:00:00+00:00' WHERE weekday =
        ↵ 2")
    r = self.tick("2026-09-23T10:10:00+00:00").get_json() # creado el día anterior: sí
    self.assertEqual(r["action"], "publishing")
    self.assertEqual(r["posts"][0]["kind"], "carrusel")

def test_hueco_recien_creado_no_publica_una_hora_pasada(self):
    cfg = schedule.build("Europe/Madrid", 120, [
        {"weekday": 1, "slot_time": "15:30", "kind": "carrusel", "created_at": "2026-09-22T13:40:00+00:00"}])
    self.assertEqual(cfg.due_slots(at(TUE_LATER)), {}) # las 15:30 ya pasaron cuando se creó el hueco
    self.assertEqual(cfg.upcoming(at(TUE_LATER), None, 1, "carrusel"), [at("2026-09-29T13:30:00+00:00")])

def test_eliminar_un_hueco_deja_de_publicar_ahi(self):
    pid = self.draft("carrusel")
    self.approve(pid)
    with db.connect() as conn:
        sid = db.one(conn, "SELECT id FROM schedule_slots WHERE kind = 'carrusel'")["id"]
    self.act(f"/admin/schedule/slots/{sid}/delete")
    self.assertEqual(self.tick(TUE_SLOT).get_json()["action"], "idle")
    self.assertIn("sin ningún hueco en la programación", self.client.get("/").get_data(as_text=True))

def test_sin_ningun_hueco_no_falla(self):
    with db.connect() as conn:
        db.execute(conn, "DELETE FROM schedule_slots")
        db.kv_set(conn, "schedule_seeded", db.now())
        db.kv_set(conn, "schedule_tz", "Europe/Madrid")
        db.kv_set(conn, "schedule_tolerance", "120")
    self.login()
    self.assertIn("No hay ningún hueco", self.client.get("/admin/schedule").get_data(as_text=True))
    self.assertEqual(self.tick(TUE_SLOT).get_json()["action"], "idle")
    self.assertEqual(self.client.get("/").status_code, 200)
    self.assertEqual(self.slots(), []) # no se vuelve a copiar schedule.txt

def test_ajustes_zona_y_tolerancia(self):
    self.act("/admin/schedule/settings", tz="America/Bogota", tolerance="60")
    with db.connect() as conn:
        self.assertEqual(db.kv_get(conn, "schedule_tz"), "America/Bogota")
        self.assertEqual(db.kv_get(conn, "schedule_tolerance"), "60")
    # 15:30 en Bogotá (UTC-5) = 20:30 UTC
    self.assertEqual(self.tick("2026-09-22T13:30:00+00:00").get_json()["action"], "idle") # ya no es la hora
    self.assertEqual(self.tick("2026-09-22T20:30:00+00:00").get_json()["action"], "empty-queue")
    for tz, tol in (("Marte/Olimpo", "60"), ("Europe/Madrid", "5"), ("Europe/Madrid", "99999"),
    ↵ ("Europe/Madrid", "abc")):
        self.act("/admin/schedule/settings", tz=tz, tolerance=tol)
    with db.connect() as conn:
        self.assertEqual(db.kv_get(conn, "schedule_tz"), "America/Bogota")
        self.assertEqual(db.kv_get(conn, "schedule_tolerance"), "60")

class TokenTests(Base):
    def test_renueva_y_guarda(self):
        instagram.refresh = lambda token: ("TOKEN-NUEVO", 5184000)

```

... continuación

```
with db.connect() as conn:
    server.maybe_refresh_token(conn)
    self.assertEqual(server.get_token(conn), "TOKEN-NUEVO")

def test_token_nuevo_del_entorno_manda_sobre_el_guardado(self):
    instagram.refresh = lambda token: ("TOKEN-RENOVADO", 5184000)
    with db.connect() as conn:
        server.maybe_refresh_token(conn)
    os.environ["IG_ACCESS_TOKEN"] = "TOKEN-EXTERNO-NUEVO"
    try:
        with db.connect() as conn:
            self.assertEqual(server.get_token(conn), "TOKEN-EXTERNO-NUEVO")
    finally:
        os.environ["IG_ACCESS_TOKEN"] = "IGAA-TOKEN-SECRETO-DE-PRUEBA"

def test_fallo_al_renovar_no_rompe(self):
    with db.connect() as conn:
        server.maybe_refresh_token(conn) # refresh lanza IGError; no debe propagarse
    self.assertEqual(server.get_token(conn), "IGAA-TOKEN-SECRETO-DE-PRUEBA")

class GuardTests(unittest.TestCase):
    def test_prohibidos(self):
        for text in ["Comenta NIVEL y te dejo el link", "COMENTA CLASES y te paso el enlace",
                    "Link en la bio", "enlace en la descripción", "te dejo el link del test",
                    "escribeme por DM", "Comenta \"TEST\" abajo", "comenta la palabra y recibe el link"]:
            self.assertTrue(guards.check(text), text)

    def test_permitidos(self):
        for text in ["Haz el test gratis de 5 minutos en trabajoexcel.com.",
                    "Reserva tu clase diagnóstico gratis en trabajoexcel.com",
                    "TU EQUIPO HACE SPRINTS. Pero no está haciendo Scrum.",
                    "¿Qué te parece? Cuéntanos tu experiencia con Excel."]:
            self.assertFalse(guards.check(text), text)

if __name__ == "__main__":
    unittest.main()
```

Apéndice B. La skill de Claude: archivos completos

B.1 SKILL.md

.claude/skills/carruseles-app/SKILL.md – instrucciones de la skill

```
---
name: carruseles-app
description: Crea contenido de Instagram (carruseles, publicaciones de una imagen e historias) de alto impacto y
↳ alcance para conseguir seguidores, promocionando las páginas de trabajoenexcel.com (homepage,
↳ profesor-excel-online.html, test-nivel-excel.html), y los sube como borrador a la app de aprobación, donde
↳ el usuario los revisa y autoriza su publicación en @trabajoenexcel. Úsala SIEMPRE que el usuario pida "un
↳ carrusel", "carrusel de Instagram", "un post", "una historia", "una publicación", "promociona esta URL en
↳ Instagram", mencione alcance/seguidores/engagement para trabajoenexcel.com, o pida contenido de Instagram
↳ sobre clases de Excel, el profesor particular, o el test de nivel. También úsala si pide ajustar colores,
↳ tipografía o número de slides de un carrusel ya generado con esta skill.
---
```

Carruseles App – trabajoenexcel.com

Genera contenido de Instagram en JPG, en tres formatos, con copy optimizado para alcance orgánico con pocos o cero seguidores, y lo sube como **borrador** a la app de aprobación (`app/`, publicada en Render; ver `CLAUDE.md`). El usuario lo revisa allí y, si lo aprueba, la app lo publica en el siguiente hueco de `app/schedule.txt` para su tipo y borra las imágenes.

Formatos (tipos de post)

```
| Tipo (`kind`) | Qué es | Imagen | Texto de publicación |
|---|---|---|---|
| `carrusel` | 3-5 slides que promocionan una URL (servicios) | 1080x1350 (4:5), `--format carrusel`, fondo
↳ crema y letra oscura | caption + hashtags |
| `publicacion` | 1 sola imagen: *un truco o dato útil de Excel*, con la web al final | 1080x1350 (4:5),
↳ `--format publicacion`, **fondo verde y letra blanca** | caption + hashtags |
| `historia` | 1 sola imagen: *promoción breve* de una URL | 1080x1920 (9:16), `--format historia`, **fondo
↳ verde y letra blanca** | **ninguno** |
```

Las historias por API son solo una imagen: no admiten stickers, enlaces, encuestas ni texto de publicación, y duran 24 horas. **Todo el mensaje va dentro de la imagen** (corto: un gancho y un remate) y el CTA nombra la web. El generador ya respeta los márgenes de seguridad de Instagram (~250 px arriba y abajo).

Principio de trabajo: AUTONOMÍA TOTAL en la creación

El usuario no quiere elegir entre variantes ni aprobar cada slide. **Genera un único resultado final por slide, revisalo tú mismo visualmente (paso 5) y entrega el carrusel completo de una vez.** Solo pregunta (con AskUserQuestion) por datos que de verdad falten (ver paso 1).

La **publicación** es otra cosa: es una acción sobre la cuenta real del usuario y la autoriza él en la app de aprobación. Esta skill nunca publica por su cuenta: termina subiendo el borrador (paso 7).

Restricciones (obligatorias)

Prohibido, en slides y en caption, cualquier CTA que dependa de enviar un enlace por comentarios, mensajes o la bio:

- «Comenta X y te dejo/paso/mando el link» (y cualquier variante de «Comenta [PALABRA]...»).
- «Link en la bio», «link en la descripción», «te lo envío por DM».

El CTA siempre nombra la web (trabajoenexcel.com) con una acción concreta, por ejemplo «Haz el test gratis de 5 minutos en trabajoenexcel.com». Antes de generar o publicar, relee el copy de todos los slides y del caption y descarta cualquier texto que incumpla esto. Si un carrusel lo incumple, no se publica.

Flujo

1. Definir objetivo y URL

- **URL**: las páginas a promocionar están en `url.txt` (raíz del repo); léelo siempre. Si el usuario no especifica cuál, y no hay contexto para inferirlo, pregunta con opciones sacadas de ese archivo. Si menciona una URL de trabajoenexcel.com que no está en `url.txt`, úsala igualmente.
- **Tema**: haz `WebFetch` de la URL elegida y extrae de la página real el dolor, la keyword y la oferta. No te bases solo en memoria.
- **Objetivo** (alcance / conexión / venta): si no lo dice, infiérelo del contexto («quiero seguidores» → alcance; «quiero que compren clases» → venta)

... continuación

y dilo en una frase; no lo preguntes salvo que sea genuinamente ambiguo.

- **Tipo y nº de slides:** si no lo dice, carrusel. Un carrusel lleva 3–5 slides (por defecto 5); una publicación o una historia, exactamente 1 imagen.

2. Sacar el ángulo real

Combina lo que extrajiste de la página con `references/urls.md`, que ya tiene el dolor, la keyword SEO y el CTA verificados de cada página (si difieren, manda la página). No inventes dolor genérico ni cifras que no estén en ninguna de las dos.

3. Escribir el copy de cada slide

Estructura por slide: `runs` = lista de (texto, "bold"|"italic").

- Slide 1: hook específico, 1-2 líneas bold + remate en itálica.
- Slides intermedios: una idea cada uno, con progresión lógica.
- Slide final: SIEMPRE termina con el CTA de `references/urls.md` para esa URL (acción concreta + trabajoenexcel.com). Lleva `cta\_text` (frase corta, en mayúsculas, máx. 28 caracteres) y `swipe\_hint`: false. Respeta la sección «Restricciones» de más abajo.
- Todos los slides salvo el último llevan `slide\_no`: "0N / 0T".

Publicación de una imagen (`publicacion`): un truco, atajo, función o dato útil de Excel, correcto y comprobable (nombres de funciones en español: `BUSCARV`, `SI.ERROR`, `HOY`); si el truco depende de la versión, dilo). No inventes cifras ni estadísticas. Una sola imagen: gancho en bold (el truco) + remate en itálica (por qué importa), y `cta\_text` tipo «MÁS EN TRABAJOENEXCEL.COM» (máx. 28 caracteres). Sin `slide\_no`. `source\_url`: `https://www.trabajoenexcel.com/`.

Historia (`historia`): una sola imagen 9:16 con la promoción breve de una URL: gancho corto en bold + remate en itálica + `cta\_text` con la acción («HAZ EL TEST GRATIS.»). Máximo unas 3-4 líneas de texto en total: se lee en 5 segundos. Sin caption.

No escribas saltos de línea a mano: el script hace wrap midiendo la fuente real (ver la sección anti-overflow de `references/style-guide.md`).

4. Generar las imágenes

Escribe el spec JSON y genera las imágenes en el **directorio temporal** (scratchpad), nunca dentro del repo: las imágenes no deben pasar por git.

```
``bash
python .claude/skills/carruseles-app/scripts/carousel_gen.py <tmp>/spec.json <tmp>/post --format
└─ carousel|publicacion|historia
````
```

(`--format` por defecto es `carousel`. Con `publicacion` o `historia` el JSON lleva exactamente 1 slide; el script lo comprueba.)

- Carpeta: `/post/` (temporal). Solo existe hasta que se sube el borrador.
- El script escribe `slide-01.jpg`, `slide-02.jpg`... (Instagram solo acepta JPEG) e imprime dónde termina el texto de cada slide. Si avisa con `△ POSIBLE OVERFLOW`, acorta el copy de ese slide y vuelve a ejecutarlo.
- Requiere Pillow (`pip install pillow`). Las fuentes van incluidas en `assets/fonts/`; no hace falta instalar nada más.

## ### 5. Revisar

Abre con Read el slide 1, el último y cualquiera que haya dado aviso de overflow, y comprueba que no hay cortes de texto. Nunca entregues sin revisar al menos el hook y los slides largos.

## ### 6. Escribir `meta.yaml` (copy de Instagram)

Crea `/post/meta.yaml` con estos campos:

- `kind`: `carousel`, `publicacion` o `historia` (si falta, se entiende carousel).
- `title`: nombre corto del post para el panel (máx. 120 caracteres).
- `source\_url`: la URL de trabajoenexcel.com que promociona.
- `caption` y `hashtags` (lo que sigue). **Una historia no lleva `caption` ni `hashtags`**: solo `kind`, `title` y `source\_url`.
- No hay `status` ni fecha: el estado lo gestiona la app y se publica cuando el usuario lo aprueba.
- **Primera línea del caption** = keyword SEO exacta de la URL (Instagram indexa en Google; está en `references/urls.md`). El hook de curiosidad va en el slide 1, no en el caption.
- **Cuerpo**: 2-3 líneas máximo, directo, sin relleno. En una publicación con un truco de Excel, el cuerpo lo explica en 2-3 líneas con un ejemplo breve.
- **CTA**: el mismo del slide final, apuntando a trabajoenexcel.com (ver `references/urls.md`). Respeta la sección «Restricciones».
- **Hashtags** (`hashtags`: sin `#`): 3-6 mezclando tres tipos: genéricos de volumen alto (`excel`, `productividad`), de nicho medio (`buscarv`, `tablasdinamicas`, `powerquery`) y de intención de búsqueda (`buscandotrabajo`, `entrevistadetrabajo`, `cursodeexcel`).
- Si preguntan por música/audio: el audio en tendencia impulsa Reels, no carruseles estáticos; para máximo alcance, un Reel con las mismas slides rinde más (este repo solo publica carruseles/imágenes).

## ... continuación

```
7. Subir el borrador a la app de aprobación
```bash
python scripts/upload_draft.py <tmp>/post --spec <tmp>/spec.json
```
```

- Sube las slides, el caption, los hashtags y el texto de los slides. El servidor vuelve a comprobar la restricción de CTA y rechaza el borrador si la incumple: corrige el copy y vuelve a subirlo.
- Necesita `IG\_APP\_URL` e `IG\_APP\_API\_KEY` (variables de entorno o `.env` en la raíz, ignorado por git). En la rutina de la nube solo existe `IG\_APP\_URL`: la clave es una «credencial de API» del entorno que añade el proxy, así que el script funciona sin ella. Si el servidor responde 401 o falta `IG\_APP\_URL`, dilo al usuario; no inventes valores.
- Tras subirlo, el script borra las imágenes locales y muestra el enlace del borrador. **\*\*Dáselo al usuario y termina ahí\*\***: la aprobación la hace él en el panel y la publicación sale sola en el siguiente hueco de `app/schedule.txt` para su tipo (por defecto: carrusel el martes 15:30; publicación e historia el jueves 19:00, hora de España). Recuérdale que debe aprobarlo **\*\*antes\*\*** de esa hora. No apruebes ni fuerces la publicación.

Si el post queda `failed` en el panel, el usuario ve el error. Un fallido no se reintenta a ciegas (podría duplicar la publicación): primero se comprueba en el perfil de Instagram si llegó a publicarse.

## ## Tanda semanal (los lunes)

Cuando se pida «la tanda semanal», «los 3 posts de la semana», o la ejecute la rutina programada de los lunes, genera **\*\*3 piezas, una de cada tipo\*\***, siguiendo el flujo 1-7 para cada una, más estas reglas:

1. **\*\*Contexto\*\*** ejecuta `python scripts/upload\_draft.py --recent` para ver los posts de las últimas semanas (tipo, título y URL). Necesita `IG\_APP\_URL` (y la clave, ver paso 7).
2. **\*\*URLs al azar\*\*** obtén 2 con `python scripts/pick\_urls.py 2` (no las elijas tú: el script es el que sortea). La primera es para el **\*\*carrusel\*\*** y la segunda para la **\*\*historia\*\***. Si una URL no está en `references/urls.md`, saca el dolor, la keyword y el CTA real de la propia página (`WebFetch`) y aplica la fórmula «[acción concreta] en trabajoenexcel.com». En la tanda semanal no modifiques archivos del repo.
3. **\*\*Las 3 piezas\*\***:
  - **\*\*Carrusel\*\*** (`carrusel`, 3-5 slides): servicios/URL sorteada.
  - **\*\*Publicación de una imagen\*\*** (`publicacion`): un truco o dato útil de Excel, con la web al final.
  - **\*\*Historia\*\*** (`historia`): promoción breve de la segunda URL.
4. **\*\*Ángulos distintos\*\*** entre las 3 de la tanda y respecto a los recientes: no repitas el mismo dolor, gancho, truco ni ejemplo de las últimas 3-4 semanas (en los trucos de Excel, no repitas la función o el atajo). Varía el objetivo (alcance / conexión / venta).
5. **\*\*Cada pieza en su propia carpeta temporal\*\*** (`<tmp>/post1`, `post2`, `post3`), con su formato (`--format`), su revisión visual del paso 5 y las «Restricciones» de siempre.
6. **\*\*Sube las 3\*\*** con `scripts/upload\_draft.py`. Si el servidor rechaza una, corrígela y vuelve a subirla; si al final no se pueden subir las 3, dilo claramente en el resumen.
7. **\*\*No apruebes ni publiques nada\*\***. Termina con un resumen corto: las 3 piezas (tipo, título y enlace del panel) y el recordatorio de las horas de `app/schedule.txt` (por defecto: el carrusel sale el martes 15:30; la publicación y la historia, el jueves 19:00): que las aprueben **\*\*antes\*\*** de esa hora.

## ## Referencias

- `references/urls.md` – dolor, keyword y CTA reales de las URLs verificadas. Léelo siempre en el paso 2, no lo repitas de memoria de conversaciones pasadas.
- `references/style-guide.md` – paleta, tipografía, estructura de slide y la regla anti-overflow. Léelo si vas a tocar colores/fuentes o si el script da overflow en varios slides seguidos.
- `scripts/carousel\_gen.py` – el generador. No lo reescribas desde cero; si necesitas un ajuste de layout, edita este archivo.

## B.2 Guía de estilo

.claude/skills/carruseles-app/references/style-guide.md – paleta, tipografía y layout

# Guía de estilo – carruseles trabajoenexcel.com

## Patrón visual (inspirado en Much Media, adaptado a la marca)  
Fondo sólido + tipografía condensada bold en mayúsculas + una frase corta en itálica de acento como "punchline" + línea divisora fina + ícono de flecha en círculo + marca en cabecera/pie. Editorial, minimalista, mucho espacio en blanco arriba y abajo del bloque de texto.

## Paleta (aproximación verificada por el usuario como "verde bosque + crema")  
- Fondo: `#F4EFE1` (cream/hueso)  
- Texto principal: `#182A20` (casi negro, tono verde oscuro)  
- Acento (itálica, líneas, ícono, CTA): `#2F6B47` (verde bosque)  
- Texto secundario/footer: `#8A9088` (gris)

△ Este hex es una aproximación de diseño, no extraído verificado del CSS del sitio (el fetch no expone estilos). Si el usuario da en algún momento un hex oficial de marca, actualiza esta tabla inmediatamente y trátalo como el estándar desde ese momento.

## Tema verde (publicación e historia)  
El carrusel usa la paleta crema de arriba. La **publicación** y la **historia** usan otro tema, para que los mensajes de la semana se distingan a simple vista (lo aplica el script según `--format`, no hay que elegirlo):  
- Fondo: `#2F6B47` (el verde de acento)  
- Texto bold e itálica, marca, línea y CTA: `#FFFFFF` (blanco)  
- Texto secundario (etiqueta y URL del pie): `#B9D3C4` (verde claro)

## Tipografía  
Fuentes incluidas en `assets/fonts/` (sin depender de Canva, de internet ni de fuentes del sistema):  
- Bold/mayúsculas: `DejaVu Sans Condensed` bold  
- Texto secundario (cabecera derecha, numeración, pie): `DejaVu Sans Condensed` regular  
- Itálica de acento: `DejaVu Serif Condensed` bold italic

Alternativas comunes si se quiere replicar en Canva/Figma más adelante:  
- Bold condensada: **0swald** (o Bebas Neue / Anton para más impacto)  
- Itálica de acento: **Playfair Display** bold italic

## Estructura de cada slide  
1. Header: `TRABAJO EN EXCEL®` (izq) / `SOCIAL AD` (der), pequeño, letter-spacing.  
2. Bloque de texto principal, empieza ~y=460–560 según cuánto texto tenga:  
- 1-2 líneas bold en mayúsculas (la idea)  
- 1 línea itálica en verde (el "punch" – la frase que se recuerda)  
- opcional: 1 línea bold más cerrando la idea  
3. Footer: línea divisora fina verde + `TRABAJOENEXCEL.COM` (der).  
4. Esquina inferior derecha: ícono de flecha en círculo (indicador de swipe) en todos los slides excepto el último, que en su lugar lleva el CTA\_TEXT + el mismo ícono.  
5. Esquina inferior izquierda: numeración `01 / 0N` en todos menos el CTA.

## Reglas de contenido (heredadas del sistema de carruseles)  
- 4 a 9 slides. Slide 1 = hook específico (nunca genérico). Última = CTA con fórmula "[Acción concreta] en trabajoenexcel.com" (el `cta\_text` ≤ 28 caracteres para no chocar con la flecha) – nunca "sígueme" ni "Comenta [PALABRA] y te dejo el link" (ver la restricción de CTA en `urls.md`).  
- Una idea por slide. Si un slide no avanza la idea, no va.  
- Progresión lógica: cada slide construye sobre el anterior.

## Anti-overflow (lección aprendida – no repetir el error)  
NUNCA escribas saltos de línea a mano adivinando dónde corta el texto. El script mide el ancho real con la fuente exacta (Pillow + `getbbox`) y hace wrap automático. Si un slide queda muy denso (el script avisa con "△ POSIBLE OVERFLOW"), la solución es acortar el copy o subir `start\_y`, no forzar el tamaño de fuente hacia abajo (rompe la jerarquía visual).

## Formatos  
- **carrusel** (crema) y **publicacion** (verde, letra blanca): 1080x1350 (4:5). La publicación va sin flecha de L deslizar ni numeración; el CTA (`cta\_text`) va abajo a la izquierda.  
- **historia** (verde, letra blanca): 1080x1920 (9:16). Instagram superpone su interfaz en los ~250 px de arriba y de abajo: cabecera en y=300, texto desde y=760 y pie/CTA por encima de y=1620. Sin flecha. Es una sola imagen sin caption: el mensaje va entero en la imagen y debe leerse en segundos.

## Apéndice C. Glosario

| Término                             | Significado                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------|
| <b>Borrador</b>                     | Un post subido a la app y aún sin aprobar ( <b>draft</b> )                           |
| <b>Cola</b>                         | Los posts aprobados de un tipo, ordenados por fecha de aprobación                    |
| <b>Hueco (slot)</b>                 | Combinación día + hora + tipo en la que se publica un post                           |
| <b>Tolerancia</b>                   | Minutos tras la hora de un hueco durante los que aún se puede usar                   |
| <b>Tick / aviso del programador</b> | La llamada periódica (cada 30 min) que despierta a la app para publicar lo que toque |
| <b>Reclamar un hueco</b>            | Marcarlo como usado de forma atómica para que solo un aviso lo aproveche             |
| <b>Contenedor (Instagram)</b>       | Objeto intermedio que crea la API con la imagen descargada, antes de publicar        |
| <b>Token de larga duración</b>      | Credencial de Instagram válida 60 días y renovable                                   |
| <b>CSRF</b>                         | Ataque que engaña al navegador para enviar un formulario con tu sesión               |
| <b>CSP</b>                          | Política de seguridad de contenido: qué recursos puede cargar una página             |
| <b>Hash con sal</b>                 | Forma irreversible de guardar contraseñas                                            |
| <b>Migración</b>                    | Cambio del esquema de la base de datos en una base que ya existe                     |
| <b>Blueprint (Render)</b>           | Archivo <b>render.yaml</b> que describe el servicio a crear                          |
| <b>Redactar</b>                     | Sustituir un secreto por <b>***</b> en un texto antes de guardarlo o mostrarlo       |
| <b>Skill</b>                        | Carpeta de instrucciones y herramientas que Claude carga para una tarea              |
| <b>Rutina</b>                       | Tarea programada de Claude en la nube                                                |
| <b>CTA</b>                          | <i>Call to action</i> : llamada a la acción («Haz el test gratis en...»)             |

# Apéndice D. Lista de comprobación final

Marca cada punto cuando lo hayas verificado de verdad:

- Las 72 pruebas pasan (`cd app && python -m unittest`).
- `/healthz` responde `ok` en la URL de Render.
- Entrar con `admin` y `ADMIN_PASSWORD`.
- Creas un usuario editor y compruebas que entra al panel pero recibe 403 en Usuarios y Programación.
- Eliminas ese usuario y su sesión abierta deja de funcionar.
- Añades y eliminas un hueco en Programación y el panel refleja los cambios.
- Subes un borrador y aparece en «Por aprobar».
- Lo apruebas y aparece «Sale: ...» con la hora prevista.
- El workflow sin «forzar» responde `idle` fuera de hueco.
- Una publicación **forzada** de cada tipo aparece en Instagram con el aspecto esperado (especialmente la historia en el móvil).
- Tras publicar, el panel dice que las imágenes se borraron y `/media/...` da 404.
- Un fallo provocado (por ejemplo, un `IG_USER_ID` erróneo) deja el post en «Falló», sin exponer credenciales y conservando las imágenes.
- `git log -p` no contiene secretos.
- Has observado el **primer hueco programado real**.